Formalization of Scott's Measurement Structures and Linear Inequalities in Lean 4

Lars Warren Ericson[†], **Dana S. Scott, Vijay D'Silva**[‡], **Brian Milnes**[§]

September 2026
CMU-CS-26-133

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

[†] Catskills Research Company, [‡] Google Research, [§] Unaffiliated

Keywords: Lean 4; formal verification; measurement theory; linear inequalities; cancellation conditions; utility theory; subjective probability

Abstract

Scott's 1964 paper *Measurement Structures and Linear Inequalities* gives a unified finite separation criterion for qualitative systems represented by homogeneous linear inequalities, with applications to intransitive indifference, ordered utility differences, and subjective probability. This report presents a Lean 4 / Mathlib formalization of all eight numbered theorems (1.1–1.4, 2.1, 3.1, 3.2, and 4.1), together with Scott's ordered-group and signed-charge remarks. The development comprises roughly 4,200 lines across 21 modules. It makes cancellation hypotheses explicit, formalizes the rational-to-integer reduction underlying finite sequence conditions, and verifies the incidence-vector constructions used by the three applications. Supplementary results include the Kraft–Pratt–Seidenberg counterexample to de Finetti's axioms and a clearly labelled modern infinite if-and-only-if characterization, adapting Kelley's 1959 method, under a generalized Kelley condition. Based on the literature audit in Section 3.5.10, that characterization provides the first explicit, self-contained realization of Scott's announced infinite extension in the published literature. The Lean library is sorry-free and introduces no project axioms beyond Mathlib's classical footprint. Large-language-model assistance was used in drafting and proof development, but every accepted declaration is checked by Lean's kernel under the pinned toolchain. The complete source and reproducible build pipeline are publicly available with this report.

Contents

1 Introduction and Historical Context **5**
- 1.1 Historical context . . . 5
- 1.2 Retrospective Remarks by Dana S. Scott . . . 5

2 Mathematical Background **6**
- 2.1 Finite linear inequalities and cancellation . . . 6
- 2.2 Solvable measurement structures . . . 7
- 2.3 A proof-development lifecycle . . . 7

3 Lean 4 Architecture and Design Decisions **8**
- 3.1 Scope and module layout . . . 8
- 3.2 Mathlib integration . . . 9
- 3.3 Proof dependency structure . . . 10
- 3.4 Verified theorem inventory . . . 10
- 3.5 Formal proof architecture . . . 11
  - 3.5.1 Theorem 1.1 — finite separation . . . 11
  - 3.5.2 Theorem 1.2 — rationalization . . . 12
  - 3.5.3 Theorems 1.3 and 1.4 — relations and additive closure . . . 13
  - 3.5.4 Theorem 2.1 — intransitive indifference . . . 14
  - 3.5.5 Theorems 3.1 and 3.2 — pairs and differences . . . 14
  - 3.5.6 Theorem 4.1 — finite subjective probability . . . 15
  - 3.5.7 Ordered groups and the lexicographic boundary . . . 16
  - 3.5.8 The Kraft–Pratt–Seidenberg counterexample . . . 16
  - 3.5.9 Modern reconstruction of an infinite analogue . . . 16
  - 3.5.10 Literature and priority . . . 17
- 3.6 Source fidelity . . . 18
  - 3.6.1 Formal conventions . . . 18
  - 3.6.2 Recorded divergences and limitations . . . 18

4 Source–Lean Concordance and Mathematical Reconstruction **19**
- 4.1 Front matter, abstract, and the three problems . . . 20
- 4.2 I. The general method . . . 24
- 4.3 II. Intransitive indifference . . . 35
- 4.4 III. Ordered differences . . . 43
- 4.5 IV. Subjective probability . . . 50
- 4.6 References in Scott 1964 . . . 55

5 Verification and Automated Pipeline **57**
- 5.1 Kernel verification and reproducible build . . . 57
- 5.2 LLM-assisted drafting and interactive proof verification . . . 58

6 Discussion and Future Work **58**

7 Code Availability and Archival **59**
7.1 GitHub repository . . . 59
7.2 Palomar registry . . . 59
7.3 License and source PDF . . . 59

8 Acknowledgments **59**
8.1 AI-assisted development . . . 59

9 References **60**

A Concordance methodology **61**
A.1 Report form and controls . . . 61
A.2 Exhaustive alignment method . . . 61
A.3 Paper-order drafting passes . . . 62
A.4 Traceability and verification . . . 62

B Complete Lean source **62**
B.1 FinHead.lean . . . 63
B.2 LinearInequalities . . . 63
B.2.1 Definitions.lean . . . 63
B.2.2 Separation.lean . . . 66
B.2.3 Rationalization.lean . . . 67
B.2.4 Sequences.lean . . . 71
B.2.5 ScottTheorems.lean . . . 73
B.2.6 OrderedGroup.lean . . . 80
B.3 Preference . . . 86
B.3.1 Direct.lean . . . 86
B.3.2 Cycle.lean . . . 91
B.3.3 Intransitive.lean . . . 92
B.4 Differences . . . 97
B.4.1 Pair.lean . . . 97
B.4.2 Ordered.lean . . . 102
B.5 Probability . . . 104
B.5.1 Basic.lean . . . 104
B.5.2 Atoms.lean . . . 107
B.5.3 Finite.lean . . . 111
B.5.4 KPSCounterexample.lean . . . 120
B.5.5 Infinite . . . 124
B.5.5.1 EventSpace.lean . . . 124
B.5.5.2 HahnBanach.lean . . . 132
B.5.5.3 Kelley.lean . . . 133
B.5.5.4 Reconstructed.lean . . . 135
B.6 Basic.lean . . . 139

C Scott 1964 source paper **140**

List of Figures

1 Main published dependency structure: finite separation core and the three applications. . . . 10
2 Finite subjective probability pipeline and the supplementary infinite reconstruction. 12

1 Introduction and Historical Context

1.1 Historical context

Measurement theory studies how observed or judged structure can be expressed numerically. Here "measurement" need not involve a physical instrument. The starting data may instead be statements such as "x is definitely preferred to y," "the change from y to x is at least as large as the change from w to z," or "event E is at least as likely as event F." A numerical representation assigns real numbers while preserving exactly those comparisons. This question is broad: it connects the foundations of measurement with utility theory, mathematical psychology, decision theory, and probability.

The three applications in Scott's paper illustrate different forms that such a representation can take. For alternatives, imagine that two objects count as discernibly different only when their scores differ by at least one unit: xPy is represented by $f(x) \geq f(y) + 1$. Nearby scores can then model indifference even when that indifference is not transitive. For pairs, imagine bundles containing one item of each of two kinds; comparing (x, x') with (y, y') by $f(x) + f'(x') \geq f(y) + f'(y')$ says that the two components contribute additively. For events, a judgment such as "rain is at least as likely as a train delay" is represented by $\mu(E) \geq \mu(F)$, where μ is a finitely additive probability. In each case qualitative data are being translated into arithmetic, but the arithmetic has a different interpretation.

Representation theorems have two directions. **Necessity** asks which qualitative laws must hold whenever a numerical representation already exists; this direction is usually checked by calculating with the proposed numbers. **Sufficiency** starts only from those qualitative laws and proves that some suitable numerical assignment must exist. Sufficiency is the harder and more consequential direction: without it, a list of axioms describes properties of a model one hopes to have, but does not show that the model can be constructed. A necessary-and-sufficient theorem therefore identifies exactly when qualitative observations admit the intended numerical reading.

Scott's article belongs to a developing program in representational measurement. Luce's work on semiorders supplied the initial setting for intransitive indifference [1], and Scott and Suppes had already given a complete finite representation theorem for that problem [2]. Kraft, Pratt, and Seidenberg had treated finite qualitative probability [3], while work on additive and conjoint measurement provided the setting for comparisons of pairs and differences [4]. Published in the first volume of the *Journal of Mathematical Psychology* in 1964 [5], Scott's paper did more than collect these questions: it showed that a common finite linear-inequality method could generate their representation conditions. Cancellation axioms, additive representations, and separation arguments subsequently became standard themes in systematic treatments of measurement theory [6].

That unification makes the article a particularly informative formalization target. It has a reusable mathematical core, three applications with familiar interpretations, and proofs that move between qualitative relations, finite-dimensional geometry, and explicit numerical models. A proof assistant must make every move in that passage precise, while a useful report must also recover the human meaning of the resulting definitions and proof terms. This formalization was undertaken after Dana S. Scott suggested the paper as a target for mechanization.

1.2 Retrospective Remarks by Dana S. Scott

For some time a group of colleagues of mine have been helping get PDFs of my old papers in shape to archive in the CMU Library. Two of them, Brian Milnes and Lars Ericson, are long ago CMU students from the 1980s. As a result of hearing a recent Milnes' Zoom talk from CMU, I arranged for him to give his talk

in Berkeley at the Topos Institute. Subsequently he and I started a collaboration so I could learn to use an AI Agent, and that is how we discovered recent work of Ericson when the agent did a literature search.

As an example of some easy-to-understand mathematics, I asked Ericson to use AI Agents to formalize in Lean 4 this old paper of mine from 1964. This report is on how the formalization was done and verified, and — more importantly — on how a human can read it and understand the results and the whole process. We hope this can then be a good model for presenting other formalization work.

2 Mathematical Background

2.1 Finite linear inequalities and cancellation

The common question behind the examples is this: when does a finite qualitative relation agree *exactly* with inequalities produced by one real-valued function? "Exactly" requires both directions. Every declared comparison must become a valid inequality, and every valid inequality among the encoded objects must correspond to a declared comparison. Solving the question directly and separately for each kind of relation would hide their shared structure.

Scott's answer is to encode each qualitative comparison as a vector and ask for one linear functional that gives every vector the prescribed sign. A utility comparison between a and b, for example, can be encoded by the difference of their coordinate vectors; applying a functional produces $f(a) - f(b)$. A comparison of two-component bundles is encoded by adding the coordinates of their components, and an event is encoded by its $0/1$ incidence vector on the underlying atoms. After these translations, preference, additive utility, and comparative probability all become finite systems of homogeneous linear inequalities.

The obstruction also has a qualitative interpretation. If positively judged comparisons can be added with positive weights to obtain the zero vector, then any representing functional must assign zero to the whole sum. It cannot make one summand strictly positive while keeping all the others nonnegative. **Cancellation** rules out precisely such contradictory ledgers of comparisons. A finite separation theorem then turns the absence of a contradictory ledger into a functional that separates strict comparisons from neutral ones. Thus necessity is obtained by applying a functional to a vanishing sum, while sufficiency is supplied by geometric separation.

Formally, for a finite symmetric set X in a real vector space and a chosen "nonnegative" part $N \subseteq X$, a realization is a linear functional φ satisfying

$$x \in N \quad \Longleftrightarrow \quad 0 \le \varphi(x) \qquad (x \in X).$$

Sign completeness ensures that every $x \in X$ receives at least one weak sign. Cancellation prevents a positive combination of strictly positive vectors from collapsing into the neutral span. Together these conditions separate the strict cone from the neutral subspace and produce φ. Rational coordinates permit positive real coefficients to be replaced by integer multiplicities, yielding Scott's unweighted sequence conditions. The core predicates are as follows:

Lean 4 source

```
def Realizable (X N : Set L) : Prop :=
  ∃ φ : Module.Dual ℝ L, ∀ x ∈ X, x ∈ N ↔ 0 ≤ φ x

def SignComplete (X N : Set L) : Prop :=
  ∀ {{x}}, x ∈ X → x ∈ N ∨ -x ∈ N
```

```
def WeightedSequenceCancellation (X N : Set L) : Prop :=
  ∀ (n : ℕ) (x : Fin (n + 1) → L) (c : Fin (n + 1) → ℝ),
    (∀ i, x i ∈ X) → (∀ i, x i ∈ N) → (∀ i, 0 < c i) →
    ∑ i, c i • x i = 0 → ∀ i, -x i ∈ N
```

2.2 Solvable measurement structures

The three applications use different incidence vectors but the same engine. The informal questions and their numerical answers are:

1. **Intransitive indifference.** A strict relation P on a finite set is interpreted as "noticeably preferred." It is represented by $xPy \iff f(x) \geq f(y) + 1$. Two alternatives less than one unit apart are indifferent; because "within one unit" need not be transitive, the model captures limited discrimination rather than forcing indifference classes.

2. **Ordered differences.** Comparisons of pairs are represented first by an additive score $f(x) + f'(x')$. When a four-place relation instead asks whether the change from y to x is at least the change from w to z, reversal combines the two scales into the single utility difference $f(x) - f(y)$.

3. **Subjective probability.** An event in a finite Boolean algebra is sent to its $0/1$ incidence vector on atoms. Qualitative comparisons of events become comparisons of sums of atom weights. A separating functional gives those weights, which normalize to a finitely additive probability.

The formalization follows this mathematical decomposition rather than treating the eight theorems as unrelated endpoints. It also records where a proof route differs from Scott's exposition: most importantly, Theorem 2.1 currently uses a direct finite Scott–Suppes staircase construction, while Scott's local cycle reductions are formalized separately.

2.3 A proof-development lifecycle

This case study also illustrates why a mathematically inclined reader might formalize an existing proof. Formalized mathematics can be viewed as an emerging **proof-development lifecycle**, analogous—but not identical—to the software development lifecycle. A theorem must be stated against precise interfaces, decomposed into reusable components, checked continuously as those components change, and delivered with enough documentation and provenance for another person to audit it. Proof development differs from ordinary programming in two important respects: discovering a proof can be the central difficulty, and a successful kernel check gives unusually strong assurance about the formal statement while still leaving humans responsible for whether that statement faithfully expresses the intended mathematics.

Several layers make such a lifecycle practical here:

1. Lean provides the basic language of types, propositions, functions, quantifiers, and equality.

2. The elaborator and common proof tools provide routine reasoning support, including rewriting, simplification, and certified arithmetic automation.

3. Mathlib provides domain-independent mathematical infrastructure: finite sums and permutations, finite-dimensional linear algebra, convexity and separation, quotient orders, and finite Boolean algebras.

4. This repository supplies the measurement-specific layer absent from the general library: cancellation predicates, incidence-vector encodings, Scott–Suppes preference constructions, and the interfaces for utility and probability representations.

The boundaries between these layers are mathematically informative. A reference used in the 1964 proof need not already exist in a library under the same name or formulation. The formalizer can identify the property actually needed, derive it from available results, and isolate the new domain-specific argument. In this development, for example, Mathlib's compact-closed separation theorem replaces Scott's cited finite-polyhedral separation result, while the measurement-theoretic reductions are proved locally.

AI-assisted drafting can reduce the cost of searching libraries, proposing encodings, and translating informal proof steps, but it does not remove the need for source comparison, theorem design, or kernel verification. Moreover, **autoformalization**—moving from prose to formal statements and proofs—is only half of the communication problem. The reverse movement, sometimes called **deformalization**, explains machine-checked objects in ordinary mathematical language and relates them back to the source question. This report makes that return path explicit: its architecture and theorem inventory expose the formal interfaces, its proof narratives explain the mathematics, and the source–Lean concordance places Scott's text, exact Lean declarations, and human-readable reconstructions side by side. That combination of formal certificate and readable reconstruction is the model for presenting formalization work tested by this article.

3 Lean 4 Architecture and Design Decisions

3.1 Scope and module layout

The library has 21 Lean modules and approximately 4,200 lines. The root re-export module is `Scott1964/MeasurementStructures/Basic.lean`, which `Solution.lean` imports.

Module cluster	Role
`FinHead`	Stable distinguished index in `Fin (n + 1)` cancellation schemes
`LinearInequalities/Definitions`, `Separation`, `Sequences`	Geometric and sequence forms of Scott's finite inequality criterion
`LinearInequalities/Rationalization`, `ScottTheorems`	Rational coefficient clearing and Theorems 1.1–1.4
`LinearInequalities/OrderedGroup`	Finite local real embedding and lexicographic obstruction
`Preference/Direct`, `Intransitive`, `Cycle`	Scott–Suppes construction, Theorem 2.1, and source proof reductions
`Differences/Pair`, `Ordered`	Incidence vectors and Theorems 3.1–3.2
`Probability/Basic`, `Atoms`, `Finite`	Finitely additive probabilities, atom vectors, signed charges, and Theorem 4.1
`Probability/KPSCounterexample`	Exact five-atom KPS qualitative order and nonrepresentability
`Probability/Infinite/*`	Universal event space, Hahn–Banach/Kelley machinery, and the modern infinite reconstruction

Module cluster	Role
`Challenge` / `Solution`	Optional stub module and sorry-free re-export of the library

The published scope is exactly the eight numbered theorems. Supplementary results are clearly separated from that inventory:

- `finite_local_real_embedding` formalizes Scott's ordered-group observation;
- `scott_p15_signed_charge` and its vector form formalize the p. 15 signed-measure characterization;
- `deFinetti_axioms_insufficient` integrates the KPS five-atom obstruction;
- `reconstructed_infinite_theorem_4_1` is a modern theorem adapting Kelley's method and motivated by, but not identified with, Scott's closing announcement. The literature basis for the priority sentence is the audit in Section 3.5.10.

The project is standalone. It imports none of the sibling formalizations of Scott's later domain-theory papers.

3.2 Mathlib integration

Mathlib supplies the finite-dimensional analytic and algebraic substrate. The predicates and reductions specific to Scott's paper are defined in this repository.

Finite-dimensional separation. The key imported result is `geometric_hahn_banach_compact_closed` from mathlib's locally convex separation library. The formalization specializes it to a finite convex hull and a finite-dimensional subspace. It then proves that the separator vanishes on the subspace and is strictly positive on the finite set:

Lean 4 source

```
theorem finite_strict_separation {A B : Set L} (hA : A.Finite) :
    Disjoint (convexHull ℝ A) (Submodule.span ℝ B : Set L) ↔
      ∃ φ : Module.Dual ℝ L,
        (∀ x ∈ A, 0 < φ x) ∧ ∀ x ∈ B, φ x = 0
```

Finite combinatorics. Cancellation schemes are indexed by `Fin (n + 1)` so that a distinguished comparison always exists. `Equiv.Perm` expresses Scott's two independent permutations in the pair and difference problems. `Finset` sums, subtype cardinalities, and `Equiv.ofFiberEquiv` turn equality of incidence-vector sums into actual permutations.

Linear algebra over function spaces. Scott's vectors are represented as functions into $\mathbb{R}$. Pair incidence vectors live in $\mathrm{Sum}\, A\, A' \to \mathbb{R}$; event vectors live in $\{a : B \mid \mathrm{IsAtom}\, a\} \to \mathbb{R}$. Mathlib linear maps provide the representing functionals, while `Pi.single` and finite sums implement the coordinate encodings.

Finite Boolean algebras. Mathlib's `BooleanAlgebra`, `IsAtom`, and finite-lattice API support decomposition of each event as the supremum of its atoms. The formalization proves both the atom-count and vector-sum versions of cancellation and constructs finitely additive functions by integrating atom weights.

Classical quotient orders. The direct proof of Theorem 2.1 uses `Antisymmetrization` of a finite total preorder. Substitutability classes remove duplicate preference profiles, and a recursively constructed staircase gives the numerical representation.

Functional analysis for the supplementary infinite theorem. The universal event space is a normed span of evaluation functions on all finitely additive probabilities. Continuous dual separation and a countable Kelley cover produce one functional that is nonnegative on weak comparisons and positive on strict comparisons.

What mathlib does not supply is Scott's measurement-theoretic interface: the sign and cancellation conditions, relation-difference encoding, Scott–Suppes weak order, pair permutation principle, atom-vector transport, or generalized Kelley condition. Those constructions are developed here. This division of labor is also reflected in the exposition. Each major result is presented first as a question about qualitative structure, then as a mathematical reduction, and finally as a checked Lean declaration. The code shows what the machine verifies; the surrounding proof narrative explains why the definitions express Scott's question and why the reduction works. Section 4 closes the loop by aligning both views with the source text.

3.3 Proof dependency structure

The main published development follows one linear-inequality core and three applications.

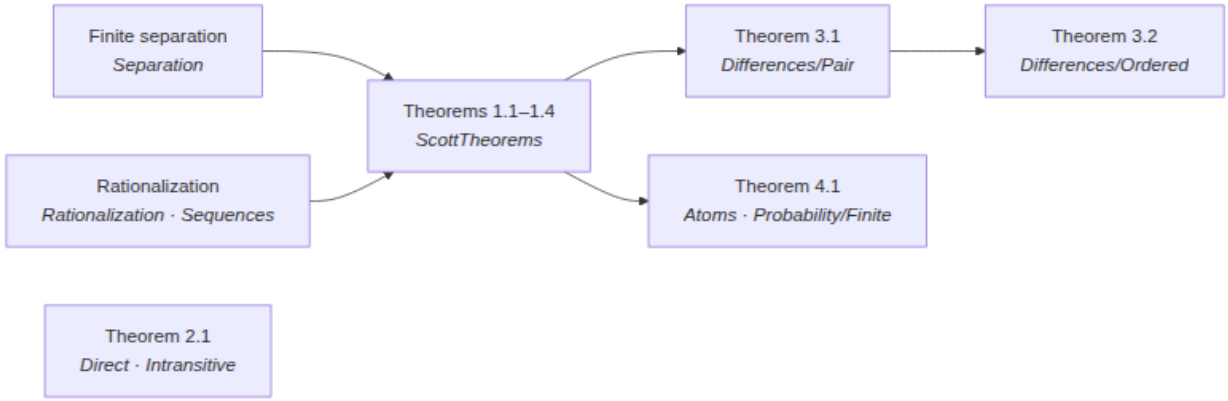


Figure 1: Main published dependency structure: finite separation core and the three applications.

Theorem 2.1 is shown as an independent branch because its completed proof is the direct Scott–Suppes construction. The source-facing cycle lemmas remain available for a future proof routed through the section-1 machinery.

The finite probability and modern infinite developments have distinct foundations:

3.4 Verified theorem inventory

Paper result	Lean theorem	Content
Theorem 1.1	`LinearInequalities.scott_theorem_1_1`	Finite symmetric sign systems: realizability iff sign completeness and positive-weight cancellation

Paper result	Lean theorem	Content
Theorem 1.2	`LinearInequalities.scott_theorem_1_2`	Rational-coordinate version with unweighted repeated summands
Theorem 1.3	`LinearInequalities.scott_theorem_1_3`	Complete relations on finite rational vectors: realizability iff paired equal-sums cancellation
Theorem 1.4	`LinearInequalities.scott_theorem_1_4`	Realizability iff extension to a strictly monotonic relation on the additive closure
Theorem 2.1	`theorem_2_1`	Intransitive-indifference representation with unit discrimination threshold
Theorem 3.1	`theorem_3_1`	Additive representation of pair comparisons by two utilities
Theorem 3.2	`theorem_3_2`	Ordered-difference representation by one utility
Theorem 4.1	`theorem_4_1`	Finite qualitative probability representation

Further source-facing or diagnostic results:

Topic	Lean theorem	Status
Literal form of `(4_B)`	`theorem_4_1_vector`	Equivalent vector-sum version of Theorem 4.1
Scott's p. 15 signed charge	`scott_p15_signed_charge` (+ vector form)	Proved without normalization or nonnegativity
Ordered-group remark	`finite_local_real_embedding`	Proved for each finite subset
Global obstruction	`no_global_real_additive_lex_realization`	Lexicographic $\mathbb{Z} \times \mathbb{Z}$ has no global additive real representation
KPS counterexample	`deFinetti_axioms_insufficient`	Exact five-atom order satisfies bundled de Finetti axioms but has no probability realization
Infinite analogue	`reconstructed_infinite_theorem_4_1`	Modern if-and-only-if result adapting Kelley; priority rests on the Section 3.5.10 audit; not attributed to Scott

3.5 Formal proof architecture

3.5.1 Theorem 1.1 — finite separation

Let X be finite and symmetric, and let $N \subseteq X$ denote the vectors declared nonnegative. The strict vectors are those $x \in N$ for which $-x \notin N$; the neutral vectors have both signs. Scott's cancellation hypothesis

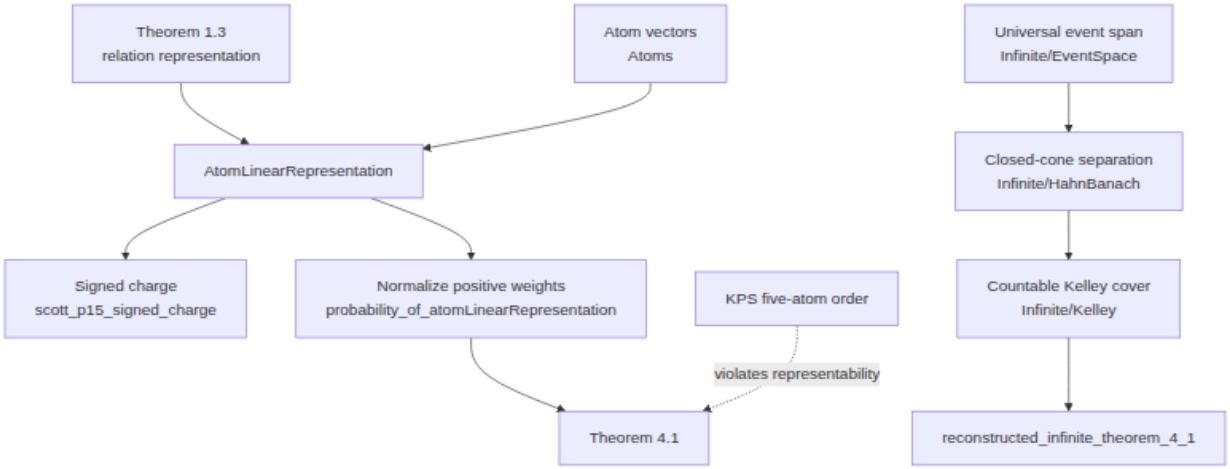


Figure 2: Finite subjective probability pipeline and the supplementary infinite reconstruction.

states geometrically that the convex hull of the strict set is disjoint from the linear span of the neutral set.

`weightedCancellation_of_sequence` derives this geometric disjointness from the literal positive-weight sequence condition. It expands a hypothetical intersection point twice: as a convex combination of strict vectors and as a linear combination of neutral vectors. Signs of the latter coefficients are absorbed using symmetry, producing one positive finite dependence. Sequence cancellation would force the negative of a strict vector into N, a contradiction.

`finite_strict_separation` then produces a linear functional positive on strict vectors and zero on neutral vectors. Sign completeness proves that the functional realizes N:

Lean 4 source

```
theorem scott_theorem_1_1 {X N : Set L}
    (hX : X.Finite) (hsym : Symmetric X) :
    Realizable X N ↔
      SignComplete X N ∧ WeightedSequenceCancellation X N
```

The two sides are exact. `Realizable X N` asks for one linear functional that is nonnegative precisely on the vectors declared to be in N. `SignComplete` says that each vector in the symmetric test set has at least one weak sign: $x \in N$ or $-x \in N$. `WeightedSequenceCancellation` says that if vectors in N, each multiplied by a strictly positive real coefficient, sum to zero, then every summand is neutral—its negative also belongs to N. The necessary direction is elementary: apply a realizing functional to such a positive dependence and compare signs.

3.5.2 Theorem 1.2 — rationalization

Theorem 1.2 replaces arbitrary positive real coefficients with repeated unit coefficients when vectors have rational coordinates. The nontrivial direction of the rational weighted-to-unweighted reduction (`UnweightedSequenceCancellation.rationalWeighted`) is factored through `Rationalization.lean`:

The pipeline is: replace a positive real dependence by a positive rational dependence; clear denominators to positive natural multiplicities; repeat each vector according to its multiplicity; apply unweighted cancel-

lation to the resulting finite sequence. This is the precise point where `[Fintype S]` and `IsRational Set X` enter. Theorem 1.1 then supplies the realizing functional:

Lean 4 source

```
theorem scott_theorem_1_2 {S : Type*} [Fintype S] {X N : Set (S → ℝ)}
    (hX : X.Finite) (hrat : IsRationalSet X) (hsym : Symmetric X) :
    Realizable X N ↔ SignComplete X N ∧ UnweightedSequenceCancellation X N
```

Here `UnweightedSequenceCancellation` has the same conclusion as the weighted condition, but its hypothesis is simply that a nonempty finite sequence of vectors in N sums to zero. Repetition supplies integer multiplicity. The rational-coordinate and finite-dimensional hypotheses are what permit the positive real coefficients of Theorem 1.1 to be replaced by rational coefficients, cleared denominators, and finally repetitions.

3.5.3 Theorems 1.3 and 1.4 — relations and additive closure

For a relation R on a finite set Y, Theorem 1.3 moves from points to differences $x - y$. Completeness gives sign completeness of the difference set. Scott's paired equal-sums condition gives unweighted cancellation:

$$\sum_i x_i = \sum_i y_i, \quad x_i R y_i \text{ for every } i \quad \Longrightarrow \quad y_i R x_i \text{ for every } i.$$

The proof constructs the symmetric rational difference set and applies Theorem 1.2. A short two-element cancellation argument ensures that the functional's weak inequality implies the original relation rather than only some relation with the same positive cone:

Lean 4 source

```
theorem scott_theorem_1_3 {S : Type*} [Fintype S]
    {Y : Set (S → ℝ)} {R : (S → ℝ) → (S → ℝ) → Prop}
    (hY : Y.Finite) (hYrat : IsRationalSet Y) :
    RelationRealizable Y R ↔
      RelationComplete Y R ∧ RelationSequenceCancellation Y R
```

Theorem 1.4 characterizes the same realizability by extension to the additive closure Y^+. A realizing functional immediately defines the extension. Conversely, strict monotonicity gives paired cancellation by summing all comparisons except the distinguished one and cancelling equal totals:

Lean 4 source

```
theorem scott_theorem_1_4 {S : Type*} [Fintype S]
    {Y : Set (S → ℝ)} {R : (S → ℝ) → (S → ℝ) → Prop}
    (hY : Y.Finite) (hYrat : IsRationalSet Y) :
    RelationRealizable Y R ↔
      ∃ Rplus,
        ExtendsOn Y (additiveClosure Y) R Rplus ∧
        StrictlyMonotonic (additiveClosure Y) Rplus
```

On the right side of Theorem 1.4, `ExtendsOn` requires agreement with R on the original set. `StrictlyMonotonic` requires the extension to be complete, to preserve two comparisons when their respective sides are added, and to cancel an equal-total equation: if $x_0 + x_1 = y_0 + y_1$ and x_1 is weakly above

y_1, then y_0 is weakly above x_0. Thus the theorem characterizes the same linear-functional representation by an explicitly additive relational extension.

3.5.4 Theorem 2.1 — intransitive indifference

Scott's relation permits indifference to be intransitive while preserving a real representation with a fixed discrimination threshold:

$$xPy \quad \Longleftrightarrow \quad f(x) \geq f(y) + 1.$$

The forward implications to irreflexivity and the two quadruple axioms are linear arithmetic. The completed converse uses a direct finite Scott–Suppes construction [2]. `ScottWeakOrder` compares alternatives by their strict-preference profiles. The quadruple axioms make it a total preorder; `Antisymmetrization` quotients alternatives with identical profiles. On the finite quotient, lower sections are nested, and `finite_staircase_representation` recursively assigns real values with the required unit gap. The representation is then pulled back to alternatives.

Lean 4 source

```
theorem theorem_2_1 {A : Type u} [Fintype A] [Nonempty A]
    (P : A → A → Prop) :
    RealizablePreference P ↔
      PrefIrrefl P ∧ PrefQuadA P ∧ PrefQuadB P
```

The library also verifies Scott's remarks around the theorem: every positive threshold is equivalent up to rescaling, every realization can be widened to a positive margin, and on a finite set the boundary can be avoided with a strict inequality. `Preference/Cycle.lean` separately proves transitivity and the three local cycle-shortening moves used in Scott's printed route. There is not yet a second completed proof assembling those reductions through the section-1 linear-inequality theorem.

3.5.5 Theorems 3.1 and 3.2 — pairs and differences

For Theorem 3.1, a pair (x, x') is represented by an incidence vector having one unit coordinate in each side of `Sum A A'`. Equality of sums of such vectors means that the first coordinates and second coordinates occur with the same multiplicities. `pairVector_sum_eq_permutations` upgrades those fiberwise cardinality equalities to two permutations.

The pair relation is transported to the finite rational range of `pairVector`. Pair totality becomes relation completeness and Scott's permutation axiom becomes paired sequence cancellation. Theorem 1.3 then returns a functional φ. Restricting φ to the two kinds of singleton coordinates yields f and f':

Lean 4 source

```
theorem theorem_3_1 {A : Type u} {A' : Type v}
    [Fintype A] [Nonempty A] [Fintype A'] [Nonempty A']
    (V : A → A' → A → A' → Prop) :
    RealizableUtilityPair V ↔ PairTotal V ∧ PairPermutation V
```

`PairTotal` says that every two mixed pairs are comparable. `PairPermutation` takes a finite list (x_i, x'_i) and independently permutes its first coordinates by π and second coordinates by σ. If, at every index except the distinguished index 0, the original pair is weakly above $(x_{\pi(i)}, x'_{\sigma(i)})$, then the distinguished

comparison must hold in reverse. This is the cancellation condition equivalent to exact representation by the additive score $f(x) + f'(x')$.

Theorem 3.2 treats a difference comparison $D(x, y, z, w)$ first as a pair comparison represented by $g(x) + q(y)$. Reversal supplies the second inequality needed to combine the scales. The function $f(x) = g(x) - q(x)$ then represents the ordered differences. Thus the sufficiency proof is a short application of Theorem 3.1 plus `difference_of_pair_and_reversal`.

The right side of Theorem 3.2 consists of three requirements. `DiffTotal` makes every two ordered differences comparable. `DiffPermutation` is the analogous distinguished-index cancellation rule after independently permuting the left and right entries of a finite list of ordered pairs. `DiffReversal` says $D(x, y, z, w) \Rightarrow D(w, z, y, x)$, as required when both differences are multiplied by -1. The permutation condition remains an infinite scheme: Lean quantifies over every `n : ℕ` and both permutations of `Fin (n + 1)`. No finite truncation is claimed.

3.5.6 Theorem 4.1 — finite subjective probability

For a finite Boolean algebra B, `atomVector x` is the real-valued characteristic function of the atoms below x. It is injective, and finite additivity is coordinate addition on disjoint suprema.

Scott writes `(4_B)` as equality of algebraic sums of characteristic functions. The primary `Prob Cancellation` uses the equivalent atom-counting reading stated immediately after the theorem: each atom occurs below equally many events on the two sides. `ProbVectorCancellation` is the literal vector-sum form, and the equivalence is proved explicitly:

Lean 4 source

```
theorem probVectorCancellation_iff_probCancellation :
    ProbVectorCancellation R ↔ ProbCancellation R
```

Totality and cancellation are transported through the range of `atomVector` and discharged by Theorem 1.3, yielding `AtomLinearRepresentation R`. Evaluation of the resulting functional on atom vectors is already a finitely additive signed charge. Nontriviality makes its value at $\top$ positive; nonnegativity makes all event values nonnegative. Division by the top value normalizes the charge without changing comparisons:

Lean 4 source

```
theorem theorem_4_1 {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧
      ProbTotal R ∧ ProbCancellation R
```

The left side requires one normalized, nonnegative, finitely additive probability μ such that $R(x, y)$ holds exactly when $\mu(x) \geq \mu(y)$. The four conditions on the right are:

1. **Nontriviality:** $\top$ is weakly above $\bot$, but $\bot$ is not weakly above $\top$.
2. **Nonnegativity:** every event is weakly above $\bot$.
3. **Totality:** every two events are comparable.
4. **Cancellation:** for two nonempty finite event lists in which every atom occurs equally often on both sides, comparisons at all non-distinguished positions force the distinguished comparison in reverse.

The structure `IsProbability` includes zero at $\bot$, normalization at $\top$, finite additivity on disjoint events, and nonnegativity. `IsSignedCharge` deliberately omits normalization and nonnegativity. This separation makes Scott's p. 15 statement exact:

Lean 4 source

```
theorem scott_p15_signed_charge (R : B → B → Prop) :
    RealizableSignedCharge R ↔ ProbTotal R ∧ ProbCancellation R
```

Cancellation also derives reflexivity, transitivity, and both directions of de Finetti's disjoint-union invariance. These lemmas connect Scott's stronger cancellation criterion with the older qualitative-probability axioms.

3.5.7 Ordered groups and the lexicographic boundary

Scott observes after Theorem 1.4 that finite pieces of an ordered abelian group can be represented in the reals while preserving the addition equations visible in that piece. `finite_local_real_embedding` encodes order tests and local addition tests as rational coordinate vectors and applies Theorem 1.4. It produces an order-reflecting $f : S \to \mathbb{R}$ preserving every equation $x + y = z$ whose three terms lie in the finite set.

This local statement cannot be extended globally without an Archimedean hypothesis. The lexicographic group $\mathbb{Z} \times \mathbb{Z}$ supplies the counterexample. Its positive comparison is strictly monotonic on the additive closure of two generators, but any additive real representation would force one positive generator to dominate arbitrarily many multiples of the other. `no_global_real_additive_lex_realization` formalizes the contradiction.

3.5.8 The Kraft–Pratt–Seidenberg counterexample

`Probability/KPSCounterexample.lean` encodes the exact 32-event order on a five-atom powerset from Kraft, Pratt, and Seidenberg [3]. Finite decidability is used to verify comparability, transitivity, bottom minimality, and de Finetti's disjoint-union invariance. These are bundled as `DeFinettiAxioms`.

Nonrepresentability is not left to exhaustive computation. Finite additivity expresses every event weight as a sum of singleton weights, and selected strict comparisons in the KPS order force an impossible cycle of linear inequalities. Consequently:

Lean 4 source

```
theorem deFinetti_axioms_insufficient :
    DeFinettiAxioms KPSGe ∧ ¬RealizableProbability KPSGe
```

`DeFinettiAxioms` uses de Finetti's literal weak nontriviality condition `¬ R ⊥ ⊤`. Scott's `Prob Nontrivial` is stronger: `R ⊤ ⊥ ∧ ¬ R ⊥ ⊤`.

3.5.9 Modern reconstruction of an infinite analogue

Scott's final paragraph announces an extension to infinite Boolean algebras but gives neither a theorem statement nor hypotheses. The repository therefore does not attempt to recover or attribute that unpublished result. Instead it states and proves a modern if-and-only-if characterization. It is stronger and more explicit

than the bare existence announcement recoverable from Scott's sentence, and its formulation uses modern normed-space infrastructure not present in the 1964 article.

`EventSpace.lean` embeds each event into the normed span of its evaluation function over all finitely additive probabilities. Stone-point indicators prove that this encoding is faithful. Weak comparisons generate a closed cone, described by its continuous-dual polar: a vector belongs to the cone when every continuous linear functional that is nonnegative on all declared weak comparisons is also nonnegative on that vector. A strict-comparison vector is $e(x) - e(y)$ when the reverse weak comparison $R(y, x)$ fails. `Generalized KelleyCondition` requires all such strict vectors to lie in the weak cone and to be covered by countably many layers. Each layer must be closed, convex, exclude zero, and remain closed upward when any weak-cone vector is added.

Adapting Kelley's measure-existence method [7], `Kelley.lean` separates each layer from zero and combines the separators with summable positive coefficients. The resulting continuous functional is nonnegative on every weak comparison and strictly positive on every strict comparison. Its value at $\top$ is positive, so normalization yields a finitely additive probability representing the relation:

Lean 4 source

```
theorem reconstructed_infinite_theorem_4_1 (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧ ProbTotal R ∧
        GeneralizedKelleyCondition R
```

The left side is exact representation on an arbitrary Boolean algebra by a normalized, nonnegative, finitely additive probability. The right side requires nontriviality, nonnegativity, totality, and the Kelley-cover condition just described. The theorem is a supplementary result of this project, adapting Kelley's proof method; it is not a ninth theorem of Scott's published paper and is not identified with his unstated future result.

3.5.10 Literature and priority

Scott's closing paragraph on p. 246 states the announcement and withholds the theorem:

> The author has obtained an extension of Theorem 4.1 to infinite Boolean algebras by using the Hahn-Banach Theorem in the same way that it was applied in Kelley (1959), where the problem of the existence of strictly positive measures was solved. If the result proves to be of interest, it will be published in a future paper.

No later paper by Scott publishes that proof. The works examined below are the candidates that cite Scott [5] in connection with infinite extensions or with Theorem 4.1. None of them carries out the functional-analytic construction announced in that paragraph: separation by the Hahn–Banach theorem, using Kelley's intersection-number technique, to represent a qualitative probability on an infinite Boolean algebra.

Cohen [8] studies weak qualitative probability structures. The argument extends an ordering of events to extended indicator functions and applies axioms from extensive measurement. It treats Scott's finite theorem as a known representation lemma. It does not separate a cone of weak comparisons by a Kelley cover.

Hardy [9] cites Scott's finite cancellation theorem while developing scaled Boolean algebras and comparative probability orderings. The infinite theory there is about scales, divisibility, and Archimedean conditions, not about producing one finitely additive probability from a countable Kelley cover of strict comparisons.

Ding, Harrison-Trainor, and Holliday [10] use Scott-style cancellation in the logic of comparative cardinality. Later work in the same line studies imprecise comparative probability and generalized cancellation. Those results stay with finite cancellation, weak representation by sets of measures, or logical axiomatization. They do not rebuild Theorem 4.1 for infinite algebras by Hahn–Banach separation.

Fu and Zhao [11] likewise invoke Scott's finite criterion inside a modal treatment of qualitative uncertainty. The paper does not supply the Kelley-cover construction.

The standard expositions keep the two ingredients apart. Krantz, Luce, Suppes, and Tversky [6, Ch. 5] review Scott's finite cancellation theorem and Kelley's existence theorem for strictly positive measures as separate chapters of the theory. Fishburn [12] surveys the axioms of subjective probability through the mid-1980s, including comparative probability on infinite algebras, without combining those two results into one representation theorem.

Based on the literature audit in Section 3.5.10, the formal characterization `reconstructed_infinite_theorem_4_1` provides the first explicit, self-contained realization of Scott's announced infinite extension in the published literature. The formal statement remains the modern equivalence under `GeneralizedKelleyCondition`. It is not a transcription of a theorem Scott did not write down.

3.6 Source fidelity

The working source is `sources/ScottMeasurement1964.pdf`. A searchable triple-pass vision transcription is generated at `sources/ScottMeasurement1964_vision.md` by `scripts/ocr_pdf_pipeline.sh`. The PDF and transcription are source material, not part of the Apache-2.0 grant. Section 4 gives an exhaustive, paper-order concordance from that transcription to the Lean development; `scripts/check_concordance.py` checks source-span coverage, verbatim quoted passages, card structure, and referenced declaration names.

3.6.1 Formal conventions

Scott writes finite sequences with an exceptional zeroth term. Lean uses `Fin (n + 1)` and the named index `finHead n`; this avoids empty-index edge cases and keeps cancellation statements uniform across modules.

Relations are oriented as "at least as preferred/probable." Thus `RelationRealizable Y R` states $R\,x\,y \leftrightarrow \varphi\,y \leq \varphi\,x$, and `RealizableProbability R` states $R\,x\,y \leftrightarrow \mu(x) \geq \mu(y)$. Scott's strict comparison is represented by `StrictlyPreferred R x y := ¬ R y x`.

The finite probability theorem uses abstract finite Boolean algebras rather than a concrete powerset. Characteristic vectors are indexed by Boolean atoms, which is equivalent to choosing the atoms of a finite field of events.

The development is classical. The disclosed axiom footprint is `propext`, `Quot.sound`, and `Classical.choice`; no project axiom is declared.

3.6.2 Recorded divergences and limitations

- Theorem 2.1 is completed by the direct finite Scott–Suppes staircase construction. Scott's local cycle reductions are formalized in `Preference/Cycle.lean`, but they are not assembled into an alternate section-1 proof.
- The primary `theorem_4_1` uses Scott's atom-count explanation of `(4_B)`. `theorem_4_1_vector` separately proves the equivalent literal equality of characteristic-vector sums.

- Scott's closing paragraph does not state his infinite theorem. `reconstructed_infinite_theorem_4_1` is a modern if-and-only-if characterization under a generalized Kelley condition. The literature basis for treating it as the first explicit realization of Scott's announcement is the audit in Section 3.5.10. It adapts Kelley's 1959 countable-cover and separation method but is not attributed to Kelley or identified with Scott's unstated result.
- The KPS development formalizes an external 1959 counterexample used to delimit the strength of de Finetti's axioms; it is not a numbered theorem of Scott's article.
- No external mathematical review has been performed. Every proof is checked by the Lean kernel, but the project remains self-assessed.

4 Source–Lean Concordance and Mathematical Reconstruction

This section is a line-by-line concordance of Scott's 1964 paper with the Lean 4 development. It is a numbered report section, not an appendix: the aim is a readable bridge from the printed mathematics to the checked proofs, in the expository spirit of the Liquid Tensor Experiment blueprint.

Each card has three panels.

1. **Scott 1964 (verbatim).** A contiguous passage of `sources/ScottMeasurement1964_vision.md`, quoted as source text. Page markers and the editorial OCR heading are retained for provenance and stripped only by the mechanical checker when comparing quotations.
2. **Lean 4 correspondence (exact source).** The actual declaration or proof fragment, or an explicit statement that no direct counterpart exists. Nearest formal objects are identified only when they are genuine.
3. **Mathematical reconstruction from Lean.** Human-readable mathematics driven by the formal argument: the objects, the reduction, and the reason the conclusion follows. This panel is not a tactic-by-tactic decompilation, and it is not a paraphrase of Scott when the Lean proof takes a different route.

Divergences are marked in place. The most important are: Theorem 2.1 is proved by the Scott–Suppes staircase rather than by the printed cycle argument through Theorem 1.2; Scott's E^2 non-Archimedean example is illustrated by the lexicographic group $\mathbb{Z} \times \mathbb{Z}$; and the infinite probability theorem is a modern reconstruction, not an attribution of Scott's unpublished announcement.

Quoted passages remain under the source-material copyright carve-out in `NOTICE` and `sources/README.md`. They are not released under Apache-2.0. Republication of this report requires clearance for the verbatim 1964 text.

Source-span coverage, quotation fidelity, card structure, and referenced declaration names are checked by `scripts/check_concordance.py`. The working method used to produce the cards is recorded in the appendix Concordance methodology.

4.1 Front matter, abstract, and the three problems

C-F01 (source lines 7-10)

Editorial transcription header.
Scott 1964 (verbatim).

> # Transcription (LLM vision OCR)

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
The vision transcription begins with an editorial heading and a page marker. These are navigation aids for the formalization, not part of Scott's article, and they do not determine any Lean object.

C-F02 (source lines 11-19)

Journal heading, title, and author.
Scott 1964 (verbatim).

> JOURNAL OF MATHEMATICAL PSYCHOLOGY: 1, 233-247 (1964)
>
> # Measurement Structures and Linear Inequalities
>
> **DANA SCOTT**
>
> *Stanford University, Stanford, California*

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
The published title identifies the paper formalized by the `Scott1964.MeasurementStructures` library. Lean records theorems and constructions, not the journal masthead.

C-F03 (source lines 20-21)

Scott's opening abstract.
Scott 1964 (verbatim).

> The general mathematical criterion for the solvability of finite systems of linear inequalities is applied to some specific situations from measurement theory. Three examples are treated in detail, and in each case the necessary and sufficient conditions for existence of a suitable real-valued (utility) function on a finite structure are obtained.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `Realizable`, `Realizable Preference`, `RealizableDifference`, and `RealizableProbability`.
Mathematical reconstruction from Lean.
This passage is historical, motivational, or bibliographic. There is no Lean definition or proof to reconstruct. The formal development begins only where a mathematical object, axiom, or theorem is isolated

below. The three examples announced here become Theorems 2.1, 3.1–3.2, and 4.1; the common solvability criterion becomes Theorems 1.1–1.4.

C-F04 (source lines 22-23)

Measurement as numerical assignment.
Scott 1964 (verbatim).

> In establishing a system of measurement one notices first that a class of objects (or events) has a certain inherent structure. Then the next step is to find a method of assigning (real) numbers to the objects in such a way that the observed structure corresponds exactly to reasonably simple arithmetical relationships involving the assigned numbers. In this paper some theoretical aspects of this subject will be discussed particularly in connection with the following specific problems:

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
This passage is historical, motivational, or bibliographic. There is no Lean definition or proof to reconstruct. The formal development begins only where a mathematical object, axiom, or theorem is isolated below.

C-F05 (source lines 24-31)

Problem I: intransitive indifference.
Scott 1964 (verbatim).

> **PROBLEM I. (Intransitive Indifference)**
>
> Let A be a finite set and let P be a binary relation on A. Under what conditions on P will there exist a real function f on A such that
>
> $$xPy \quad \text{if and only if} \quad f(x) \geqslant f(y) + 1,$$
>
> for all $x, y \in A$?

Lean 4 correspondence (exact source).
The numerical representation is exactly `RealizablePreference`.

Lean 4 source

```
def RealizablePreference {A : Type u} (P : A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∀ x y, P x y ↔ f x ≥ f y + 1
```

Mathematical reconstruction from Lean.
Scott asks when a finite strict relation P is the unit-threshold cut of a real assignment. Lean packages that existence statement and nothing more: no axioms yet, and no construction of f. The constant 1 is conventional; a later lemma rescales any positive threshold to this unit. The solution is `theorem_2_1`.

C-F06 (source lines 32-39)

Problem II: ordered differences.
Scott 1964 (verbatim).

> **PROBLEM II. (Ordered Differences)**
>
> Let A be a finite set and let D be a quaternary relation on A. Under what conditions on D will there exist a real function f on A such that
>
> $$xy\ D\ zw \quad \text{if and only if} \quad f(x) - f(y) \geqslant f(z) - f(w),$$
>
> for all $x, y, z, w \in A$?

Lean 4 correspondence (exact source).

Lean 4 source

```
def RealizableDifference {A : Type u} (D : A → A → A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∀ x y z w, D x y z w ↔ f x - f y ≥ f z - f w
```

Mathematical reconstruction from Lean.
The quaternary comparison is a comparison of increments of one utility. Lean keeps Scott's four-argument order and the same difference inequality. The reduction through a pair of utilities appears only when Theorem 3.2 is proved.

C-F07 (source lines 40-47)

Problem III: subjective probability.
Scott 1964 (verbatim).

> **PROBLEM III. (Subjective Probability)**
>
> Let B be a finite Boolean algebra and let $\succcurlyeq$ be a binary relation on B. Under what conditions on $\succcurlyeq$ will there exist a probability measure μ on B such that
>
> $$x \succcurlyeq y \quad \text{if and only if} \quad \mu(x) \geqslant \mu(y),$$
>
> for all $x, y \in B$?

Lean 4 correspondence (exact source).

Lean 4 source

```
def RealizableProbability {B : Type u} [BooleanAlgebra B] (R : B → B → Prop) :
    Prop :=
  ∃ μ : B → ℝ, IsProbability μ ∧ ∀ x y, R x y ↔ μ x ≥ μ y
```

Mathematical reconstruction from Lean.
A qualitative comparison of events is realized when some finitely additive, nonnegative, normalized set function reproduces it. `IsProbability` forces $\mu(\bot) = 0$, $\mu(\top) = 1$, additivity on disjoint events,

and nonnegativity. Scott's Boolean algebra is an abstract finite `BooleanAlgebra`, not a hardcoded powerset.

C-F08 (source lines 48-53)

History and motivation for Problem I.
Scott 1964 (verbatim).

> Problem I was first considered in Luce (1956), where a partial solution was given. The complete solution was given in Scott and Suppes (1958) and the proof, as well as a
>
> comprehensive survey of related problems, is also presented in Suppes and Zinnes (1963). For motivation imagine that the relation xPy means that the object x is *definitely preferred* to the object y. Hence if neither xPy nor yPx hold, then x and y are *indifferent*. Unfortunately human powers of discrimination often lead to cases where indifference is intransitive. The question is then to find numerical assignments in which the boundary between preference and indifference is made explicit: the arithmetic relation $\alpha \geqslant \beta + 1$ is an obvious candidate for performing this service (note that the constant 1 could be replaced by any other convenient positive constant by a change of units.) The solution in Scott and Suppes (1958), though direct, is a quite tedious and clumsy proof exploiting special properties of the particular situation. The method to be presented in Section I and applied in Section II uses very well-known theorems on the existence of solutions of linear inequalities. As a special virtue of the approach we find that analysis of the conditions from the general result specialized to the particular instance leads us to *discover* the required properties of the relation P in an almost mechanical way. This virtue will be illustrated in the discussion of the other problems as well.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `theorem_2_1` and the Scott–Suppes staircase in `Preference/Direct.lean`.
Mathematical reconstruction from Lean.
Lean does not formalize Luce's 1956 partial solution or the prose claim that indifference is often intransitive. What it does record is the completed representation theorem and the observation that a realizing f forces irreflexivity. The printed 1964 cycle argument is only partly formalized; the checked converse uses the 1958 staircase construction rather than Section I cancellation.

C-F09 (source lines 54-55)

Problem III and the Kraft–Pratt–Seidenberg paper.
Scott 1964 (verbatim).

> Problem III was solved in Kraft, Pratt, and Seidenberg (1959). Those authors suggest the kind of method used here, but carry out a different, more direct approach. Their conditions are a little hard to digest because they write certain formulas *multiplicatively* when an *additive* notation is more suggestive and more natural in application to Boolean algebras. For this reason, and for purely expository reasons, the author decided to include the details in Section IV; the proof, however, is quite short given the material of Section I.

Lean 4 correspondence (exact source).
Scott's re-derivation is `theorem_4_1`. The 1959 insufficiency result is the integrated counterexample

Lean 4 source

```
theorem deFinetti_axioms_insufficient :
    DeFinettiAxioms KPSGe ∧ ¬RealizableProbability KPSGe
```

Mathematical reconstruction from Lean.
The formal library separates two claims that Scott mentions together. Theorem 4.1 is the correct finite criterion, obtained from Theorem 1.3 on atom vectors. The KPS order shows that de Finetti's five older axioms do not imply that criterion. There is no separate Lean theorem named "KPS Theorem 2"; Scott's strengthening is the object that was formalized.

C-F10 (source lines 56-57)

Priority remark on Problem II.
Scott 1964 (verbatim).

> A solution to Problem II has, to the author's best knowledge, not been previously published. (After this paper was prepared for publication, the referee informed the author that E. W. Adams had independently obtained similar results by pursuing the method of Adams and Fagot (1956). More specifically, Adams obtained results including those given below in Theorem 3.1 and 4.1. The details are contained in a dittoed report "Remarks on inexact additive measurement." The author is happy to acknowledge priority to Professor Adams. The purpose of the present paper is, in any case, to show how to apply a general method to many problems of this type.) For motivation the reader is referred to Luce and Tukey (1964) and Suppes and Zinnes (1963).

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `theorem_3_1` and `theorem_4_1`.
Mathematical reconstruction from Lean.
Adams's unpublished report is not formalized. Lean proves the theorems Scott states, not the historical priority claim. The methodological promise—that the general inequality criterion discovers the axioms—is realized for Theorems 3.1 and 4.1, which do route through Theorem 1.3, and is not yet realized for Theorem 2.1.

4.2 I. The general method

C-I01 (source lines 58-63)

Symmetric sets and realizable sign patterns.
Scott 1964 (verbatim).

> ## I. THE GENERAL METHOD
>
> Let L be a finite-dimensional real linear vector space. A subset $X \subseteq L$ is *symmetric* if $X = -X = \{-x : x \in X\}$. A subset $N \subseteq X$ is called *realizable* in X if there is a linear

functional φ on L such that for all $x \in X$

$$x \in N \quad \text{if and only if} \quad \varphi(x) \geqslant 0.$$

Lean 4 correspondence (exact source).

Lean 4 source

```
def Symmetric (X : Set L) : Prop := ∀ {{x}}, x ∈ X → -x ∈ X

def Realizable (X N : Set L) : Prop :=
  ∃ φ : Module.Dual ℝ L, ∀ x ∈ X, x ∈ N ↔ 0 ≤ φ x
```

Mathematical reconstruction from Lean.
Scott works in a finite-dimensional real vector space; Lean uses a module L over $\mathbb{R}$, later specialized to a finite-dimensional normed space when separation is invoked. Symmetry is closure under negation. Realizability is the existence of one linear functional whose nonnegative half-space cuts X exactly along N. The zero functional realizes the case $N = X$, which Lean does not isolate as a named lemma.

C-I02 (source lines 64-67)

Sign notation and the half-space remark.
Scott 1964 (verbatim).

> (Recall that a linear functional is a real-valued, homogeneous, additive function defined on L. Thus in case $N \neq X$ the condition means that there is a half-space H of L separating the sets N and $X \sim N$ so that $X \cap H = N \cap H$. If $N = X$, the trivial 0 functional shows that N is realizable.) We shall employ a more suggestive notation writing $x \geqslant 0$ to mean $x \in N$. Further we write $x \leqslant 0$ to mean $-x \geqslant 0$, and $x \succ 0$ for not $x \leqslant 0$. The whole approach can be summed up by a simple, general theorem.

Lean 4 correspondence (exact source).

Lean 4 source

```
def strictPositive (X N : Set L) : Set L :=
  {x | x ∈ X ∧ x ∈ N ∧ -x ∉ N}

def indifferent (X N : Set L) : Set L :=
  {x | x ∈ X ∧ x ∈ N ∧ -x ∈ N}

def SignComplete (X N : Set L) : Prop :=
  ∀ {{x}}, x ∈ X → x ∈ N ∨ -x ∈ N
```

Mathematical reconstruction from Lean.
Scott's suggestive notation $x \geq 0$ is membership in N. The strict vectors are those with only one weak sign; the indifferent vectors carry both. Sign completeness is condition (1). The half-space paragraph is exposition: `Realizable` already permits a trivial functional, and the later separation lemma produces a functional that is strictly positive on the strict set and zero on the indifferent span.

C-I03 (source lines 68-75)

Theorem 1.1.
Scott 1964 (verbatim).

> **THEOREM 1.1.** Let X be a finite, symmetric subset of L. For a subset $\{x \in X : x \geqslant 0\}$ to be realizable in X it is necessary and sufficient that the conditions
>
> $$x \geqslant 0 \quad \text{or} \quad x \leqslant 0, \tag{1}$$
>
> $$\sum_{i<n} \lambda_i x_i = 0 \quad \text{implies} \quad x_0 \leqslant 0, \tag{2}$$
>
> hold for all $x \in X$ and all sequences $x_0, \cdots, x_{n-1} \in X$, and all scalars $\lambda_0, \cdots, \lambda_{n-1}$, where $\lambda_i > 0$ and $x_i \geqslant 0$, for $i < n$, and $n > 0$.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem scott_theorem_1_1 {X N : Set L}
    (hX : X.Finite) (hsym : Symmetric X) :
    Realizable X N ↔
      SignComplete X N ∧ WeightedSequenceCancellation X N
```

Mathematical reconstruction from Lean.
Condition (2) is a positive-weight dependence among members of N that sums to zero. Lean concludes that every summand is indifferent, which is stronger than Scott's displayed $x_0 \leq 0$ but is the form used in the convex-hull contradiction. Sequences are indexed by `Fin (n+1)` so the distinguished term always exists. The geometric shadow of (2) is `WeightedCancellation`: the convex hull of strict vectors misses the span of indifferent vectors.

C-I04 (source lines 76-77)

Necessity in Theorem 1.1.
Scott 1964 (verbatim).

> **PROOF.** The necessity of (1) is clear. The necessity of (2) becomes at once clear when it is considered that a realizing functional makes $\sum_{i<n} \lambda_i x_i$ nonnegative because $\lambda_i > 0$ and $x_i \geqslant 0$, for $i < n$; and since the vector sum is actually 0, the functional cannot make x_0 (or any other x_i, for that matter) strictly positive.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem Realizable.weightedSequenceCancellation {X N : Set L}
    (h : Realizable X N) (hsym : Symmetric X) :
    WeightedSequenceCancellation X N
```

Mathematical reconstruction from Lean.
If φ realizes N, a positive combination of members of N is sent to a nonnegative number. When that combination is the zero vector, each summand must have φ-value zero and therefore, by realization, must also lie in $-N$. Sign completeness is the same dichotomy applied to a single vector: $\varphi(x)$ is either nonnegative or its negative is.

C-I05 (source lines 78-85)

Sufficiency: cones, polyhedra, and separation.
Scott 1964 (verbatim).

> To prove the sufficiency, assume that the two conditions hold. Let Q be the convex polyhedral cone generated by the set $\{x \in X : x \leqslant 0\}$ and let P be the convex polyhedron generated by (the convex closure of) the set $\{x \in X : x \succ 0\}$. We can assume P is nonempty, since otherwise $x \leqslant 0$ holds for all $x \in X$; and therefore $x \geqslant 0$ would hold for all $x \in X$, because X is symmetric. If we can show that P and Q are *disjoint*, it follows at once (see Theorem 2, p. 50 of the book Kuhn and Tucker 1956, for example) that there is a linear functional φ on L such that for all $x \in L$,
>
> $$x \in P \quad \text{implies} \quad \varphi(x) > 0$$
>
> $$x \in Q \quad \text{implies} \quad \varphi(x) \leqslant 0.$$
>
> Thus if $x \in X$ and $x \geqslant 0$, then $-x \leqslant 0$, $-x \in Q$, $\varphi(-x) \leqslant 0$, and $\varphi(x) \geqslant 0$. If $\varphi(x) \geqslant 0$, then $\varphi(-x) \not> 0$, so $-x \notin P$, and so $-x \leqslant 0$ and $x \geqslant 0$. Hence, φ is the required functional.

Lean 4 correspondence (exact source).
Scott's P and Q become the convex hull of the strict set and the span of the indifferent set. The checked separation lemma is Mathlib's compact-closed Hahn–Banach theorem, specialized to a finite hull:

Lean 4 source

```
theorem finite_strict_separation {A B : Set L} (hA : A.Finite) :
    Disjoint (convexHull ℝ A) (Submodule.span ℝ B : Set L) ↔
      ∃ φ : Module.Dual ℝ L,
        (∀ x ∈ A, 0 < φ x) ∧ ∀ x ∈ B, φ x = 0
```

Mathematical reconstruction from Lean.
The printed argument cites Kuhn–Tucker. Lean does not cite that page; it invokes `geometric_hahn_banach_compact_closed` on the compact convex hull of a finite strict set and the closed finite-dimensional span of the indifferent set. The resulting continuous functional is flipped in sign so that it is strictly positive on every strict vector and identically zero on the indifferent subspace. Sign completeness then upgrades this geometric separator to a realizing functional for N.

C-I06 (source lines 86-97)

The intersection contradiction in Theorem 1.1.
Scott 1964 (verbatim).

> Let us then suppose that there is a vector $z \in P \cap Q$. Now
>
> $$z = \sum_{i<l} \lambda_i x_i = \sum_{i<m} \lambda'_i x'_i,$$
>
> where $x_i \succ 0$, $\lambda_i \geqslant 0$, for $i < l$, and $\sum_{i<l} \lambda_i = 1$, and where $x'_i \leqslant 0$, $\lambda'_i \geqslant 0$, for $i < m$. By condition (1), $x_i \geqslant 0$ holds for $i < l$. Thus
>
> $$\sum_{i<l} \lambda_i x_i + \sum_{i<m} \lambda'_i(-x'_i) = 0.$$
>
> We can assume that all the scalars are strictly positive, and because $\sum_{i<l} \lambda_i = 1$, we know $l > 0$. Since $-x'_i \geqslant 0$ for $i < m$, we can apply (2) to conclude that $x_0 \leqslant 0$. But this contradicts the assumption that $x_0 \succ 0$, and the proof is complete.

Lean 4 correspondence (exact source).
The rewrite of a hypothetical point of $P \cap Q$ as one positive dependence is `weighted Cancellation_of_sequence` in `LinearInequalities/ScottTheorems.lean`.
Mathematical reconstruction from Lean.
Suppose a vector lies in both the convex hull of strict vectors and the linear span of indifferent vectors. Expanding both expressions and using symmetry to flip the sign of any negative indifferent coefficient produces one strictly positive finite combination equal to zero. Sequence cancellation would force a strict vector to be indifferent, which it is not. Therefore the two sets are disjoint, and the separation lemma applies. Lean does not keep Scott's barycentric normalization $\sum \lambda_i = 1$ as a separate hypothesis; convex-hull membership already supplies it, and vanishing weights are discarded before the sequence is formed.

C-I07 (source lines 98-107)

Rational coordinates and Theorem 1.2.
Scott 1964 (verbatim).

> There is a case where condition (2) of Theorem 1.1 can be simplified. Let S be a finite set and let $L = L(S)$ be the vector space of all real-valued functions defined on S (the ordinary S-dimensional vector space). A *vector* (function) in L is called *rational* if all its coordinates (values) are rational numbers. A *set of rational vectors* is also called rational. With these conventions Theorem 1.1 becomes more combinatorial.
>
> **THEOREM 1.2.** Let X be a finite, rational, symmetric subset of L. For a subset $\{x \in X : x \geqslant 0\}$ to be realizable it is necessary and sufficient that the conditions
>
> $$x \geqslant 0 \quad \text{or} \quad x \leqslant 0, \tag{3}$$

$$\sum_{i<n} x_i = 0 \quad \text{implies} \quad x_0 \leqslant 0, \tag{4}$$

hold for all $x \in X$ and all sequences $x_0, \cdots, x_{n-1} \in X$, where $x_i \geqslant 0$ for $i < n$, and $n > 0$.

Lean 4 correspondence (exact source).

Lean 4 source

```
def IsRationalVector {S : Type*} (x : S → ℝ) : Prop :=
  ∀ s, ∃ q : ℚ, x s = q

theorem scott_theorem_1_2 {S : Type*} [Fintype S] {X N : Set (S → ℝ)}
    (hX : X.Finite) (hrat : IsRationalSet X) (hsym : Symmetric X) :
    Realizable X N ↔ SignComplete X N ∧ UnweightedSequenceCancellation X N
```

Mathematical reconstruction from Lean.
Scott's $L(S)$ is the function space $S \to \mathbb{R}$ with S finite. Rationality is coordinatewise. Condition (4) is the same cancellation with every coefficient equal to 1; repetitions stand in for integer multiplicities. The theorem is the combinatorial special case of Theorem 1.1 once positive real weights can be replaced by unweighted repetitions.

C-I08 (source lines 108-109)

Proof of Theorem 1.2.
Scott 1964 (verbatim).

> **PROOF.** Suppose that $\sum_{i<n} \lambda_i x_i = 0$, where $n > 0$, and $\lambda_i > 0$, $x_i \geqslant 0$ for $i < n$. In other words, the λ_i are positive solutions to a system of homogeneous linear equations with *rational* coefficients (the coordinates of the x_i). Hence there must also be a *rational* set of positive λ_i satisfying the equation. By clearing fractions and replacing integral multiples by repetitions, we can apply (4) to conclude that $x_0 \leqslant 0$. Thus condition (2) is verified and the result follows by Theorem 1.1.

Lean 4 correspondence (exact source).
The rationalization pipeline is isolated in `LinearInequalities/Rationalization.lean`: a positive real dependence among rational vectors is replaced by a positive rational dependence, denominators are cleared, and each vector is repeated according to its natural multiplicity.
Mathematical reconstruction from Lean.
A homogeneous linear relation with rational coordinates and positive real coefficients admits a positive rational solution. Clearing a common denominator yields a positive natural-weight relation. Repeating each vector that many times produces an unweighted sequence summing to zero, to which (4) applies. Thus unweighted cancellation implies weighted cancellation on rational sets, and Theorem 1.1 supplies the functional. The converse is immediate: drop the weights.

C-I09 (source lines 110-113)

Realizable relations.
Scott 1964 (verbatim).

> Suppose next that Y is any subset of L. Suppose further that $\succsim$ is a *binary relation* on Y. We shall write $x \preccurlyeq y$ for $y \succsim x$. We shall say that $\succsim$ is *realizable* if there exists a linear functional φ on L such that for all $x, y \in Y$ we have
>
> $$x \succsim y \quad \text{if and only if} \quad \varphi(x) \geqslant \varphi(y).$$

Lean 4 correspondence (exact source).

Lean 4 source

```
def RelationRealizable (Y : Set L) (R : L → L → Prop) : Prop :=
  ∃ φ : Module.Dual ℝ L, ∀ x ∈ Y, ∀ y ∈ Y, R x y ↔ φ y ≤ φ x
```

Mathematical reconstruction from Lean.
A binary comparison on a set of vectors is realized by comparing the values of one linear functional. Lean orients $R\,x\,y$ as $\varphi(y) \leq \varphi(x)$, matching Scott's $x \succsim y$ iff $\varphi(x) \geq \varphi(y)$. Completeness is condition (5).

C-I10 (source lines 114-125)

Theorem 1.3.
Scott 1964 (verbatim).

> Conditions for realizability are easily deduced from Theorem 1.2.
>
> **THEOREM 1.3.** Let Y be a finite rational subset of L. For a binary relation $\succsim$ on Y to be realizable it is necessary and sufficient that the conditions
>
> $$x \succsim y \quad \text{or} \quad x \preccurlyeq y, \tag{5}$$
>
> $$\sum_{i<n} x_i = \sum_{i<n} y_i \quad \text{implies} \quad x_0 \preccurlyeq y_0, \tag{6}$$
>
> hold for all $x, y \in Y$ and all sequences $x_0, \cdots, x_{n-1}, y_0, \cdots, y_{n-1} \in Y$, where $x_i \succsim y_i$ for $i < n$ and $n > 0$.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem scott_theorem_1_3 {S : Type*} [Fintype S]
    {Y : Set (S → ℝ)} {R : (S → ℝ) → (S → ℝ) → Prop}
    (hY : Y.Finite) (hYrat : IsRationalSet Y) :
    RelationRealizable Y R ↔
      RelationComplete Y R ∧ RelationSequenceCancellation Y R
```

Mathematical reconstruction from Lean.
The difference set $X = Y - Y$ is finite, rational, and symmetric. Declaring $x - y \geq 0$ when $R\,x\,y$ is well-defined because paired equal-sums cancellation identifies differences that represent the same vector. Completeness of R becomes sign completeness of that sign pattern, and (6) becomes unweighted cancellation. Theorem 1.2 returns a functional on differences, hence a realization of R. Lean also records a geometric variant `scott_theorem_1_3_geometric` that never appears in the paper: disjointness of the convex hull of strict differences from the span of indifferent differences.

C-I11 (source lines 126-127)

Proof of Theorem 1.3.
Scott 1964 (verbatim).

> **PROOF.** Assume (5) and (6). Let $X = Y - Y = \{x - y : x, y \in Y\}$. Clearly X is finite, rational, and symmetric. Define $x - y \geqslant 0$ to mean that $x \succsim y$. This is justified because if $x - y = x' - y'$, $x, y, x', y' \in Y$, then by (6), $x \succsim y$ if and only if $x' \succsim y'$. The desired result is now a direct consequence of Theorem 1.2.

Lean 4 correspondence (exact source).
The body of `scott_theorem_1_3` constructs the difference set, transports (5)–(6) to Theorem 1.2, and uses a two-element cancellation to recover the original relation from the functional.
Mathematical reconstruction from Lean.
Necessity of (6) is evaluation of a realizing functional on equal sums. Sufficiency must also check that the sign assigned to a difference does not depend on the pair that represents it. If $x - y = x' - y'$ then the two-term sequence (x, y') versus (y, x') has equal sums, so (6) identifies $R\,x\,y$ with $R\,x'\,y'$. After Theorem 1.2 produces φ, a second two-point cancellation ensures that $\varphi(x) \geq \varphi(y)$ implies the original R, not merely some relation with the same nonnegative cone of differences.

C-I12 (source lines 128-139)

Strictly monotonic relations and additive closure.
Scott 1964 (verbatim).

> Let Z be a subset of L which is closed under addition ($Z + Z \subseteq Z$). A binary relation $\succsim$ on Z will be called *strictly monotonic* if the following three conditions are satisfied:
>
> (i) $x \succsim y$ or $x \preccurlyeq y$,
>
> (ii) $x_0 \succsim y_0$ and $x_1 \succsim y_1$ imply $x_0 + x_1 \succsim y_0 + y_1$,
>
> (iii) $x_0 + x_1 = y_0 + y_1$ and $x_1 \succsim y_1$ imply $x_0 \preccurlyeq y_0$,
>
> for all $x, y, x_0, y_0, x_1, y_1 \in Z$. For a subset $Y \subseteq L$, we let Y^+, the additive closure of Y, denote the least set $Z \supseteq Y$ closed under addition. Using this terminology, we can restate Theorem 1.3.
>
> **THEOREM 1.4.** Let Y be a finite rational subset of L. For a binary relation $\succsim$ on Y to be realizable it is necessary and sufficient that it be extendable to a *strictly monotonic* relation on Y^+.

Lean 4 correspondence (exact source).

Lean 4 source

```
structure StrictlyMonotonic (Z : Set L) (R : L → L → Prop) : Prop where
  complete : ∀ {{x}}, x ∈ Z → ∀ {{y}}, y ∈ Z → R x y ∨ R y x
  add : ∀ {{x₀ y₀ x₁ y₁}}, x₀ ∈ Z → y₀ ∈ Z → x₁ ∈ Z → y₁ ∈ Z →
    R x₀ y₀ → R x₁ y₁ → R (x₀ + x₁) (y₀ + y₁)
  cancel : ∀ {{x₀ y₀ x₁ y₁}}, x₀ ∈ Z → y₀ ∈ Z → x₁ ∈ Z → y₁ ∈ Z →
    x₀ + x₁ = y₀ + y₁ → R x₁ y₁ → R y₀ x₀

def additiveClosure (Y : Set L) : Set L :=
  AddSubmonoid.closure Y
```

Mathematical reconstruction from Lean.
Scott's Y^+ is the additive monoid generated by Y, including the empty sum. The three axioms are totality, preservation of addition, and cancellation of a summand. `ExtendsOn` says the enlarged relation agrees with the original on Y. These are the ingredients of Theorem 1.4, not an independent representation theorem on the whole closure.

C-I13 (source lines 140-149)

Theorem 1.4.
Scott 1964 (verbatim).

> **PROOF.** Suppose $\succsim$ on Y is realizable. Let φ be a functional that realizes $\succsim$. Define $\succsim$ on Y^+ by the condition
>
> $$x \succsim y \quad \text{if and only if} \quad \varphi(x) \geqslant \varphi(y)$$
>
> for all $x, y \in Y^+$. Then the new $\succsim$ is obviously an extension of the old $\succsim$ on Y, and the linearity of φ makes it trivial to verify (i), (ii), and (iii). Conversely, if we have the extension $\succsim$ to Y^+, we need only verify condition (6). Thus suppose that $x_i, y_i \in Y$, $x_i \succsim y_i$ for $i < n$, $n > 0$ and that $\sum_{i<n} x_i = \sum_{i<n} y_i$. If $n = 1$, then $x_0 = y_0$ and so $x_0 \preccurlyeq y_0$. If $n > 1$, then $\sum_{0<i<n} x_i \succsim \sum_{0<i<n} y_i$ in view of (ii). But
>
> $$x_0 + \sum_{0<i<n} x_i = y_0 + \sum_{0<i<n} y_i,$$
>
> so $x_0 \preccurlyeq y_0$ by virtue of (iii).

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem scott_theorem_1_4 {S : Type*} [Fintype S]
    {Y : Set (S → ℝ)} {R : (S → ℝ) → (S → ℝ) → Prop}
    (hY : Y.Finite) (hYrat : IsRationalSet Y) :
    RelationRealizable Y R ↔
      ∃ Rplus,
```

```
        ExtendsOn Y (additiveClosure Y) R Rplus ∧
        StrictlyMonotonic (additiveClosure Y) Rplus
```

Mathematical reconstruction from Lean.
A realizing functional extends immediately by the same comparison of values, and linearity gives the three monotonicity axioms. Conversely, an extension that is strictly monotonic on Y^+ implies (6): add the comparisons other than the distinguished pair, then cancel the resulting equal totals. Theorem 1.3 finishes the argument. Scott's warning is exact in Lean as well: the theorem realizes the restriction to finite Y, not the relation on the whole closure.

C-I14 (source lines 150-154)

The non-Archimedean warning.
Scott 1964 (verbatim).

> One should not suppose that a strictly monotonic relation on Y^+ is realizable on Y^+. We have only shown that the *restriction* of $\succsim$ is realizable on the *finite* set Y. (Example: Let $L = E^2$, the two dimensional space, and let $Y = \{(1,0),(0,1)\}$. Let $\succsim$ be defined on Y^+ by the stipulation
>
> $$(n_0, m_0) \succsim (n_1, m_1) \quad \text{if and only if} \quad n_0 > n_1 \quad \text{or} \quad n_0 = n_1 \quad \text{and} \quad m_0 \geqslant m_1.$$
>
> This is a non-Archimedean ordering of Y^+.)

Lean 4 correspondence (exact source).
Lean illustrates the same obstruction with the lexicographic group $\mathbb{Z} \times \mathbb{Z}$ rather than Scott's displayed subset of E^2:

Lean 4 source

```
theorem no_global_real_additive_lex_realization :
    ¬ ∃ φ : LexIntPair → ℝ,
        (∀ x y, lexPreference x y ↔ φ y ≤ φ x) ∧
          ∀ x y, φ (x + y) = φ x + φ y
```

Mathematical reconstruction from Lean.
Scott's example is a lexicographic order on the additive monoid generated by two basis vectors. Lean uses the lexicographic product of two copies of $\mathbb{Z}$. The order is strictly monotonic on the monoid generated by $(1,0)$ and $(0,1)$, yet any additive real representation would force one positive generator to dominate arbitrarily many copies of the other. The printed coordinatewise recipe and the Lean lexicographic group are the same phenomenon, not a line-for-line transcription of the E^2 display.

C16 (source lines 155-165)

Editorial OCR reconciliation notes.
Scott 1964 (verbatim).

Reconciliation notes (only where the passes diverged):

Discrepancy	Resolution
Pass 1 omits `\in Y` after `$x, y, x', y'$`	Image confirms `\in Y`; Passes 2 & 3
Summation subscripts `i<n` vs `i < n`	Image shows spaced subscripts; Passes 2 & 3
Italic *on* in "realizable on Y^+" (Pass 2 only)	Image shows *on* not italicized; Passes 1 & 3
Trailing comma/period spacing in display math	Image shows comma after second sum equation and period before closing parenthesis in the example

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
This table is an editorial record of discrepancies among OCR passes. It is not part of Scott's paper and contributes no Lean object. The checker treats the card as coverage only and does not compare its quotation to the source lines.

C-I15 (source lines 166-177)

Finite subsets of ordered abelian groups.
Scott 1964 (verbatim).

> The content of Theorem 1.4 amounts to this well-known fact from algebra: *Every finite subset of an ordered abelian group can be isomorphically embedded in the additive group of reals.* Proof: let G be the abelian group with $\oplus$ as the group operation and $\gtrdot$ as the ordering. Let $S = \{g_0, \cdots, g_{m-1}\}$ be the finite subset of G. For each $i < m$ let U_i be the characteristic function of the subset $\{g_i\}$ of S. So $U_i \in L = L(S)$, and $Y = \{U_0, \cdots, U_{m-1}\}$ is rational. Further Y^+ has only *integer-valued* functions as elements. Define $\succcurlyeq$ on Y^+ by the condition that for $x, y \in Y^+$
>
> $$x \succcurlyeq y \quad \text{if and only if}$$
> $$x(g_0) \cdot g_0 \oplus \cdots \oplus x(g_{m-1}) \cdot g_{m-1} \gtrdot y(g_0) \cdot g_0 \oplus \cdots \oplus y(g_{m-1}) \cdot g_{m-1}.$$
>
> It is obvious from the axioms for ordered abelian groups that $\succcurlyeq$ is a strictly monotonic relation on Y^+. Let φ be some linear functional which realizes $\succcurlyeq$ on Y. The function f on S where
>
> $$f(g_i) = \varphi(U_i)$$
>
> is well-defined and, by construction, preserves the addition and order relations *within* the finite set S.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem finite_local_real_embedding (S : Set G) (hS : S.Finite) :
    ∃ f : S → ℝ,
      (∀ x y : S, x ≤ y ↔ f x ≤ f y) ∧
      (∀ x y z : S, (x : G) + y = z → f x + f y = f z)
```

Mathematical reconstruction from Lean.
Characteristic functions of singleton elements of a finite subset S become rational test vectors. Finite addition equations visible inside S become further test vectors. The coordinatewise comparison of those vectors is strictly monotonic on their additive closure, so Theorem 1.4 supplies a functional. Evaluating that functional on basis vectors yields a real map that reflects the ambient order on S and preserves every equation $x + y = z$ whose three terms lie in S. Lean therefore proves a slightly stronger local embedding than the prose sentence "preserves the addition and order relations within the finite set S," but it does not state a separately named theorem that every ordered abelian group embeds globally in $\mathbb{R}$.

C-I16 (source lines 178-179)

Bridge to the three measurement problems.
Scott 1964 (verbatim).

> This trick of passing from an abstract structure to the "more concrete" vectors in a finite-dimensional linear space will now be consistently exploited for the solution of the three problems from measurement theory.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `theorem_3_1` and `theorem_4_1`, which apply Theorem 1.3.
Mathematical reconstruction from Lean.
The sentence is a methodological promise. In the formal library it is true of the pair and probability developments and not yet true of Theorem 2.1, whose completed converse uses the Scott–Suppes staircase instead of an incidence-vector instance of Theorem 1.2.

4.3 II. Intransitive indifference

C-II01 (source lines 180-186)

Section II setup.
Scott 1964 (verbatim).

> **II. SOLUTION TO THE PROBLEM OF INTRANSITIVE INDIFFERENCE**
>
> Let A be a finite nonempty set and let P be a binary relation on A. For the purposes of this section we shall call P *realizable* if there is a real-valued function f on A such that
>
> $$xPy \quad \text{if and only if} \quad f(x) \geqslant f(y) + 1$$
>
> for all $x, y \in A$. Realizable relations are obviously irreflexive, and so we shall assume P is irreflexive.

Lean 4 correspondence (exact source).

Lean 4 source

```
def PrefIrrefl {A : Type u} (P : A → A → Prop) : Prop :=
  ∀ x, ¬P x x
```

Mathematical reconstruction from Lean.
Realizability is again the unit-threshold representation. A realizing f is automatically irreflexive, so Scott assumes `PrefIrrefl` as axiom (1_P). Lean keeps that axiom on the right-hand side of `theorem_2_1` rather than as a standing typeclass on P.

C-II02 (source lines 187-201)

The auxiliary vector e and the set X.
Scott 1964 (verbatim).

> Let e be an element not in the finite set A and set $S = A \cup \{e\}$. Each element $x \in S$ determines a vector in $L = L(S)$: namely, the characteristic function of $\{x\}$. To avoid clumsy notation, we shall identify each $x \in S$ with its corresponding vector and pretend that $S \subseteq L$. Thus S becomes an *independent basis* for the $|S|$-dimensional space L.
>
> We let X be the subset of L containing just the vectors of the forms $x - y - e$ or $y + e - x$, where $x, y \in A$. We define $\succcurlyeq 0$ by these stipulations:
>
> (i) $x - y - e \succcurlyeq 0$ if and only if xPy;
>
> (ii) $y + e - x \succcurlyeq 0$ if and only if not xPy;
>
> MEASUREMENT STRUCTURES AND LINEAR INEQUALITIES
>
> for all $x, y \in A$. This is justified because P is irreflexive. Obviously X is a finite, rational, symmetric set. Further $u \succcurlyeq 0$ or $-u \succcurlyeq 0$ holds for all $u \in X$. We are now going to find necessary conditions on P that will verify the conditions of Theorem 1.2 to make $\succcurlyeq 0$ realizable on X. Using such a realization we will then be able to show that the conditions are sufficient to make P realizable in the sense of the present section.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `RealizablePreference` and the unused local moves in `Preference/Cycle.lean`. There is no incidence-vector encoding of Section II.
Mathematical reconstruction from Lean.
Scott embeds $A \cup \{e\}$ as a basis and realizes the sign pattern of the vectors $x - y - e$ and $y + e - x$ by Theorem 1.2. That construction is not in the library. The checked proof of Theorem 2.1 never builds X, never mentions e, and never calls `scott_theorem_1_2`. This is the principal missing formal counterpart in Section II.

C-II03 (source lines 202-206)

Normalization $\varphi(e) = 1$ and necessity of the axioms.
Scott 1964 (verbatim).

> Since P is irreflexive, we see that this means that $e \succcurlyeq 0$ holds while $-e \succcurlyeq 0$ does not hold, in other words $e \succ 0$. Hence, assuming that $\succcurlyeq 0$ is realizable on X, we would have a linear functional φ realizing $\succcurlyeq 0$ such that $\varphi(e) > 0$. Multiplying by a positive scalar, we can assume that $\varphi(e) = 1$. It follows at once that
>
> $$xPy \quad \text{if and only if} \quad \varphi(x) \geqslant \varphi(y) + 1,$$
>
> for $x, y \in A$. Hence φ restricted to A gives the realization of P. To complete the picture, we need now only find further necessary conditions on P that will imply condition (2) of 1.2 for $\succcurlyeq 0$ as defined on X.

Lean 4 correspondence (exact source).
Necessity of (1_P)–(3_P) is proved directly from f, not from a functional on X:

Lean 4 source

```
theorem RealizablePreference.necessary {A : Type u} {P : A → A → Prop}
    (hP : RealizablePreference P) :
    PrefIrrefl P ∧ PrefQuadA P ∧ PrefQuadB P
```

Mathematical reconstruction from Lean.
If a unit-threshold f exists, irreflexivity and the two quadruple implications are linear arithmetic. Scott obtains the same necessities after normalizing $\varphi(e) = 1$. Lean skips that normalization because it never constructs φ. The conclusions coincide; the routes do not.

C-II04 (source lines 207-221)

Cancellation sequences and independence of e.
Scott 1964 (verbatim).

> Suppose, therefore, that we have sequences $x_0, \cdots, x_{k-1}, y_0, \cdots, y_{k-1}, x'_0, \cdots, x'_{m-1}, y'_0, \cdots, y'_{m-1} \in A$ where $x_i P y_i$ holds for $i < k$, but not $x'_i P y'_i$ holds for $i < m$. We assume $k + m = n > 0$. If we can establish that in this situation the equation
>
> $$\sum_{i<k}(x_i - y_i - e) + \sum_{i<m}(y'_i + e - x'_i) = 0$$
>
> is *never valid*, then we will have indeed verified what amounts to condition (4) of Theorem 1.2.
>
> Thus, by way of contradiction, let us assume that this last equation does hold for the indicated system of elements of A. We can rewrite the equation as

$$\sum_{i<k}(x_i - y_i) + \sum_{i<m}(y'_i - x'_i) + (m-k)\cdot e = 0.$$

Now the vector $e \in L$ is known to be *independent* of vectors in A, thus $m - k = 0$ and $m = k > 0$. In the equation

$$\sum_{i<k}(x_i - y_i) + \sum_{i<m}(y'_i - x'_i) = 0$$

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
The printed argument shows that a putative zero-sum of the incidence vectors forces equally many primed and unprimed terms because e is independent of A. There is no Lean lemma for this count, because the incidence vectors themselves are absent.

C-II05 (source lines 222-234)

Permutation of the two arrays.
Scott 1964 (verbatim).

only vectors from A appear, and A is an independent set of vectors. Unfortunately in the equation vectors can occur several times with both positive and negative coefficients. The best we can say in view of the independence of A is that the two arrays of vectors

$$(x_0, \cdots, x_{k-1}, y'_0, \cdots, y'_{m-1})$$

and

$$(y_0, \cdots, y_{k-1}, x'_0, \cdots, x'_{m-1})$$

are alike in that one is a *permutation* of the other. Let us make a tour through these arrays following out the permutation.

Start with y_0 in the lower array. Since P is irreflexive, $x_0 \neq y_0$, so y_0 coincides with some other element in the upper array. Take the element in the lower array *directly under* the element in the upper array just considered. It must coincide with *some other* element in the upper array. Proceeding in this manner, passing from an element in the lower array to an equal element in the upper and then dropping to the corresponding element of the lower directly below this upper element, we shall eventually come back to our starting element y_0. We shall have traced out a *cycle*. There are two cases.

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
Independence of the basis A yields only that the upper and lower arrays are permutations of one another. Lean has this style of argument for pairs (`pairVector_sum_eq_permutations`) but not for the

Section II encoding. The subsequent "tour through the permutation" is therefore also unformalized as a global cycle decomposition.

C-II06 (source lines 235-244)

Case 1: an unprimed P-cycle.
Scott 1964 (verbatim).

> CASE 1. *The cycle never leaves the unprimed elements.* By a simple permutation of subscripts we can assume that we have an $l < k, l > 0$, such that
>
> $$y_0 = x_1, y_1 = x_2, \cdots, y_{l-1} = x_l, y_l = x_0,$$
>
> and that $x_i P y_i$ holds for all $i < k$, thus we have a cycle in P-relationships:
>
> $$x_0 P x_1 P x_2 \cdots x_l P x_0.$$
>
> This can easily be ruled out by assuming that P is *transitive*, so that $x_0 P x_0$ would follow contradicting the irreflexivity of P. If P is realizable, then P is necessarily transitive, because $\alpha \geqslant \beta + 1$ is obviously a transitive relation between real numbers.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem preference_transitive_of_quadA {A : Type u} {P : A → A → Prop}
    (hirr : ∀ x, ¬ P x x)
    (hquad : ∀ x y z w, P x y → P z w → P x w ∨ P z y) :
    IsTrans A P
```

Mathematical reconstruction from Lean.
Scott rules out a pure P-cycle by transitivity plus irreflexivity. Lean proves transitivity from the quadruple axiom (2_P) by substituting y for z, which is exactly Scott's later parenthetical. It does not construct the cycle from a permutation of arrays, and the lemma is not imported by `theorem_2_1`.

C-II07 (source lines 245-270)

Case 2: adjacent P's and condition (3_P).
Scott 1964 (verbatim).

> CASE 2. *The cycle passes through primed and unprimed elements.* If the cycle starting with y_0 has *more* primed than unprimed elements, forget it. It cannot contain all the unprimed elements, because $k = m$. Start with an unused element y_i. Its cycle we may assume has both primed and unprimed elements, otherwise we are back in Case 1. Again, if there are more primed elements, start a new cycle. By this argument we may be sure that the cycle we obtain finally has *at least as many* unprimed elements as primed. Let us use the notation yQx as shorthand for *not* xPy. This time we will get a cycle of relationships which, after a change of subscripts, we can assume to be of the form

$$x_0 P x_1 \cdots x_l P y'_0 Q y'_1 \cdots y'_j Q x_{l+1} \cdots x_0.$$

The number of P's is the same as the number of x's and the Q's as the y's; hence, there are at least as many P's as Q's. A degenerate case is $x_0 P y'_0 Q x_0$, which is the only case where there is only one P. This case is impossible because $y'_0 Q x_0$ means that *not* $x_0 P y'_0$. Thus we may assume there are at least two P's.

Suppose there are two P's adjacent in the cycle. The situation looks like

$$zPxPyQw$$

or like

$$wQzPxPy,$$

(where the letters x, y, z, w now no longer refer to the arrays from which the cycle was obtained.) Assuming that P is realizable by a function f we would have in the first situation

$$f(z) \geqslant f(x) + 1, \quad f(x) \geqslant f(y) + 1, \quad \text{and} \quad f(y) + 1 > f(w).$$

Therefore $f(z) \geqslant f(w) + 1$, and zPw would follow. In the second situation by a similar argument wPy would follow. These *two* transitivity conditions are equivalent and are equivalent to the following implication

$$zPxPy \quad \text{implies} \quad wPy \quad \text{or} \quad zPw,$$

for $x, y, z, w \in A$. If we assume that P has this necessary property, then our cycle of P, Q-relationships can be reduced to a *shorter cycle* where there are still as many P's as Q's.

Lean 4 correspondence (exact source).

Lean 4 source

```
def PrefQuadB {A : Type u} (P : A → A → Prop) : Prop :=
  ∀ x y z w, P x y → P z x → P w y ∨ P z w

theorem preference_shorten_ppq {A : Type u} {P : A → A → Prop}
    (hquad : ∀ x y z w, P x y → P z x → P w y ∨ P z w)
    {z x y w : A} (hzx : P z x) (hxy : P x y) (hyw : ¬ P w y) :
    P z w
```

Mathematical reconstruction from Lean.
The local configurations $zPxPyQw$ and $wQzPxPy$ are the two adjacent-P shortenings. Given (3_P), each configuration collapses to a shorter cycle with equally many P's and Q's. Lean records both local inferences and stops there. It does not assemble them into a descent on a global mixed cycle, which is why they cannot yet replace the staircase proof.

C-II08 (source lines 271-287)

Alternating cycles and condition (2_P).
Scott 1964 (verbatim).

> Applying the above reasoning, we shall reduce the cycle either to one P and one Q (which is impossible) or to a cycle where there are no adjacent P's. Since there are as many P's as Q's, there can be no pair of adjacent Q's either. Thus a part of the cycle must look like
>
> $$xPyQzPw.$$
>
> Assuming that f realizes P on A we have
>
> $$f(x) \geqslant f(y) + 1 > f(z) \geqslant f(w) + 1,$$
>
> whence $f(x) \geqslant f(w)+1$ and so xPw. This means that if we assume about P the necessary implication that
>
> $$xPy \quad \text{and} \quad zPw \quad \text{implies} \quad xPw \quad \text{or} \quad zPy,$$
>
> for $x, y, z, w \in A$, then our cycle can be reduced step by step to xPx which is already assumed impossible. This completes the impossibility argument.
>
> Notice in this last implication that if we substitute y for z and eliminate the false yPy, then P is found to satisfy the transitive law. We can thus summarize this lengthy discussion as a theorem.

Lean 4 correspondence (exact source).

Lean 4 source

```
def PrefQuadA {A : Type u} (P : A → A → Prop) : Prop :=
  ∀ x y z w, P x y → P z w → P x w ∨ P z y

theorem preference_shorten_pqp {A : Type u} {P : A → A → Prop}
    (hquad : ∀ x y z w, P x y → P z w → P x w ∨ P z y)
    {x y z w : A} (hxy : P x y) (hyz : ¬ P z y) (hzw : P z w) :
    P x w
```

Mathematical reconstruction from Lean.
Once adjacent P's are removed, a balanced cycle is alternating and contains the pattern $xPyQzPw$. Condition (2_P) shortens that pattern. Repeated shortening would reach xPx, contradicting irreflexivity. Lean has the local move and the observation that (2_P) implies transitivity. The global descent remains a gap.

C-II09 (source lines 288-297)

Theorem 2.1.
Scott 1964 (verbatim).

> **THEOREM 2.1.** *Let A be a finite nonempty set and let P be a binary relation on A. For P to be realizable it is necessary and sufficient that the conditions*
>
> (1_p) *not* xPx,
>
> (2_p) xPy *and* zPw *imply* xPw *or* zPy,
>
> (3_p) xPy *and* zPx *imply* wPy *or* zPw,
>
> *hold for all* $x, y, z, w \in A$.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem theorem_2_1 {A : Type u} [Fintype A] [Nonempty A]
    (P : A → A → Prop) :
    RealizablePreference P ↔
      PrefIrrefl P ∧ PrefQuadA P ∧ PrefQuadB P
```

The converse is not Scott's cycle argument. It uses `ScottWeakOrder`, antisymmetrization, and `finite_staircase_representation`.

Mathematical reconstruction from Lean.

The statement is faithful. Necessity is the linear-arithmetic lemmas already cited. Sufficiency compares alternatives by their strict-preference profiles: x is weakly below y when every predecessor of x is a predecessor of y and every successor of y is a successor of x. The two quadruple axioms make this relation a total preorder. Antisymmetrization quotients by identical profiles. On the finite quotient the lower sections are nested, and a recursively constructed staircase assigns real values with a unit gap, which pulls back to A. This is the Scott–Suppes construction, not a formalization of pages 7–9.

C-II10 (source lines 298-307)

Strict margins and Scott's methodological remark.
Scott 1964 (verbatim).

> It will be noted that in our argument we did not make full use of some *strict inequalities* that turned up. The reason is that we did not require the more definite information; indeed, the method gives a realization f of P where for all $x, y \in A$,
>
> $$f(x) \geqslant f(y) + 1 \quad \text{if and only if} \quad f(x) > f(y) + 1$$
>
> holds for all $x, y \in A$.
>
> The author does not claim that the proof of Theorem 2.1 given here is particularly attractive. But he does feel that the relentless application of Theorem 1.2 gave the required solution without having to know the answer in advance—that by elementary combinatorial analysis the result was systematically uncovered.

Lean 4 correspondence (exact source).

Lean 4 source

```
def StrictlyRealizablePreference {A : Type u} (P : A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∀ x y, P x y ↔ f x > f y + 1

theorem RealizablePreference.strict_of_finite
    {A : Type u} [Fintype A] [Nonempty A] {P : A → A → Prop}
    (hP : RealizablePreference P) : StrictlyRealizablePreference P
```

Mathematical reconstruction from Lean.
On a finite set a realizing assignment can be perturbed so that the unit threshold is never attained. Lean proves that the weak and strict unit-threshold representations are equivalent for finite A, and that any positive threshold is equivalent to 1 after rescaling. Scott's closing remark that the proof is unattractive but mechanical is commentary; the formal converse is attractive for a different reason, and it is not the mechanical Section I analysis he describes.

4.4 III. Ordered differences

C-III01 (source lines 308-317)

Additive utilities for pairs.
Scott 1964 (verbatim).

> ## III. SOLUTION TO THE PROBLEM OF ORDERED DIFFERENCES
>
> Before presenting the solution to Problem II, we shall treat a related, but simpler, question which also has independent interest.
>
> Let A and A^* be two finite nonempty sets, and let V be a quaternary relation which we treat as a binary relation on the cartesian product $A \times A^*$. Think of A and A^* as sets of two different kinds of commodities, and interpret xx^*Vyy^*, where $x, y \in A$, $x^*, y^* \in A^*$, to mean that the combination of x and x^* is *more valuable than* the combination of y and y^*. The question about these structures is whether there exist real-valued functions f and f^* on A and A^*, respectively, that realize V in the sense that
>
> $$xx^*Vyy^* \quad \text{if and only if} \quad f(x) + f^*(x^*) \geqslant f(y) + f^*(y^*),$$
>
> for all $x, y \in A$, $x^*, y^* \in A^*$. Such a pair f, f^* of functions is called a pair of *utility functions* for V. Thinking of f and f^* as assigning values to the commodities, the interpretation of V as an ordering of the combinations is clear. Of course, we are treating the situation with the philosophy that "the whole is equal to the sum of its parts"; hence, a straightforward adding of utilities in finding the utility of a combination is assumed to be subtle enough for the present theory.

Lean 4 correspondence (exact source).

Lean 4 source

```
def RealizableUtilityPair {A : Type u} {A' : Type v}
    (V : A → A' → A → A' → Prop) : Prop :=
  ∃ f : A → ℝ, ∃ f' : A' → ℝ,
    ∀ x x' y y', V x x' y y' ↔ f x + f' x' ≥ f y + f' y'
```

Mathematical reconstruction from Lean.
A comparison of commodity bundles is additive across the two factors. Lean uses a four-argument predicate rather than a relation on a cartesian product, which is the same data. The philosophy that "the whole is the sum of its parts" is the meaning of the displayed inequality, not an extra axiom.

C-III02 (source lines 318-325)

Incidence vectors of pairs.
Scott 1964 (verbatim).

> We shall now apply our general Theorem 1.3 to characterize the relations V realizable in the above sense. To this end let $S = A \cup A^*$ be the finite set determining the linear space $L = L(S)$. As in Section II we identify S with the set of basis vectors for L in the obvious way. We let
>
> $$Y = \{x + x^* : x \in A, x^* \in A^*\}.$$
>
> Without loss of generality we can assume that the sets A and A^* are *disjoint*, so that Y is in a one-one correspondence with the set $A \times A^*$. We define $\geqslant$ on Y in the inescapable way for $x, y \in A$, $x^*, y^* \in A^*$:
>
> $$x + x^* \geqslant y + y^* \quad \text{if and only if} \quad xx^* V yy^*.$$

Lean 4 correspondence (exact source).

Lean 4 source

```
noncomputable def pairVector {A : Type u} {A' : Type v} (x : A) (x' : A') :
    Sum A A' → ℝ :=
  Pi.single (Sum.inl x) 1 + Pi.single (Sum.inr x') 1
```

Mathematical reconstruction from Lean.
Scott's $S = A \cup A^*$ and $Y = \{x + x^*\}$ become functions on `Sum A A'`. Disjointness of the two summands is built into the coproduct, so the later remark that disjointness is only a technical device is already reflected in the types: Theorem 3.1 never assumes `Disjoint` of the original sets. The relation on Y is V transported along `pairVector`.

C-III03 (source lines 326-348)

Totality, equal sums, and the two permutations.
Scott 1964 (verbatim).

> MEASUREMENT STRUCTURES AND LINEAR INEQUALITIES

It is immediate that the binary relation $\geqslant$ is realizable on the finite rational set Y (in the sense of Section I) if and only if V is realizable on A by a pair of utility functions.

Considering Theorem 1.3, we need now only transcribe conditions (5) and (6) into equivalent conditions about V. There is no problem about (5): we must assume the relation V has the property that xx^*Vyy^* or yy^*Vxx^* holds for all $x, y \in A, x^*, y^* \in A^*$.

For property (6), assume we have elements satisfying the equation

$$\sum_{i<n}(x_i + x_i^*) = \sum_{i<n}(y_i + y_i^*)$$

where $x_i, y_i \in A, x_i^*, y_i^* \in A^*$ and $x_i x_i^* V y_i y_i^*$ holds for $i < n$ and $n > 0$. We require conditions on V so that $y_0 y_0^* V x_0 x_0^*$ follows from these assumptions.

Now in the linear space L, remember that the vectors in $S = A \cup A^*$ are independent of each other. Hence, for the above equation to be valid, we must have

$$\sum_{i<n} x_i = \sum_{i<n} y_i \quad \text{and} \quad \sum_{i<n} x_i^* = \sum_{i<n} y_i^*,$$

because the sets A and A^* are assumed disjoint. In other words, there must exist a pair π, σ of permutations of the indices $\{0, 1, \cdots, n-1\}$ such that the equations

$$y_i = x_{\pi(i)} \quad \text{and} \quad y_i^* = x_{\sigma(i)}^*$$

hold for $i < n$. Therefore, the required assumption about V states that $x_i x_i^* V x_{\pi(i)} x_{\sigma(i)}^*$ for $i < n$ implies $x_{\pi(0)} x_{\sigma(0)}^* V x_0 x_0^*$, where $n > 0$, π and σ are permutations of $\{0, 1, \ldots, n-1\}$, and $x_i \in A, x_i^* \in A^*$ for $i < n$. (Actually the hypothesis of this implication is too strong. In view of the other assumption on V, we need only take $i > 0$ in the hypothesis of the second assumption.) It should be clear to everyone that these two conditions are necessary for V to be realizable, and that 1.3 shows conversely the sufficiency. We can therefore state the desired theorem.

Lean 4 correspondence (exact source).

Lean 4 source

```
def PairPermutation {A : Type u} {A' : Type v}
    (V : A → A' → A → A' → Prop) : Prop :=
  ∀ (n : ℕ) (x : Fin (n + 1) → A) (x' : Fin (n + 1) → A')
    (π σ : Equiv.Perm (Fin (n + 1))),
    (∀ i, i ≠ finHead n → V (x i) (x' i) (x (π i)) (x' (σ i))) →
      V (x (π (finHead n))) (x' (σ (finHead n)))
        (x (finHead n)) (x' (finHead n))
```

Mathematical reconstruction from Lean.
Independence of the two families of coordinates splits an equality of pair-vector sums into two multiplicity identities. `pairVector_sum_eq_permutations` upgrades those identities to permutations π and σ. Condition (2_V) is the resulting cancellation, with Scott's distinguished index 0 rendered as `fin Head n` and the hypothesis taken only for the other indices, as he allows parenthetically.

C-III04 (source lines 349-359)

Theorem 3.1.
Scott 1964 (verbatim).

> **THEOREM 3.1.** *Let A and A^* be two finite nonempty sets and let V be a binary relation on $A \times A^*$. For V to be realizable by a pair of utility functions on A and A^* it is necessary and sufficient that the conditions*
>
> $$(1_V) \quad xx^*Vyy^* \quad \text{or} \quad yy^*Vxx^*,$$
>
> $$(2_V) \quad x_i x_i^* V x_{\pi(i)} x_{\sigma(i)}^* \quad \text{for} \quad i < n,\ i > 0, \quad \text{implies} \quad x_{\pi(0)} x_{\sigma(0)}^* V x_0 x_0^*,$$
>
> *hold for all $x, y \in A, x^*, y^* \in A^*$ and all sequences $x_0, \ldots, x_{n-1} \in A, x_0^*, \ldots, x_{n-1}^* \in A^*$ and all permutations π, σ of $\{0, \ldots, n-1\}$, where $n > 0$.*
>
> We do not need to assume that A and A^* are disjoint. That assumption was only a technical device in the proof to obtain *two* utility functions from *one* linear functional on $L = L(S) = L(A \cup A^*)$.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem theorem_3_1 {A : Type u} {A' : Type v}
    [Fintype A] [Nonempty A] [Fintype A'] [Nonempty A']
    (V : A → A' → A → A' → Prop) :
    RealizableUtilityPair V ↔ PairTotal V ∧ PairPermutation V
```

Mathematical reconstruction from Lean.
Necessity is evaluation of $f + f'$. Sufficiency transports V to the finite rational range of `pairVector`, turns totality into `RelationComplete` and the permutation axiom into `RelationSequence Cancellation`, and applies Theorem 1.3. The resulting functional, restricted to the two kinds of singleton spikes, is the pair of utilities. This is the printed route.

C-III05 (source lines 360-365)

(2_V) implies transitivity.
Scott 1964 (verbatim).

> SCOTT
>
> It is mildly amusing to note how condition (2_V) implies that V is transitive. Suppose xx^*Vyy^* and yy^*Vzz^* both hold. Let $x_0 = z$, $x_1 = x$, $x_2 = y$, and $x_0^* = z^*$, $x_1^* = x^*$, $x_2^* = y^*$. Let $\pi(i) = \sigma(i) = i + 1 \pmod 3$. Then by (2_V) we see that $x_1 x_1^* V x_0 x_0^*$ holds; that is, xx^*Vzz^* holds as required.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem PairPermutation.transitive {A : Type u} {A' : Type v}
    {V : A → A' → A → A' → Prop} (hV : PairPermutation V) :
    ∀ p q r : A × A',
      V p.1 p.2 q.1 q.2 → V q.1 q.2 r.1 r.2 → V p.1 p.2 r.1 r.2
```

Mathematical reconstruction from Lean.
Scott's three-cycle of indices with $\pi = \sigma$ the cycle $(0\,1\,2)$ is copied exactly, using the length-3 instance of `PairPermutation`. The calculation is a check that the infinite scheme already contains ordinary transitivity.

C-III06 (source lines 366-381)

Ordered differences and the pair of utilities g, h.
Scott 1964 (verbatim).

> Turning now to the ordered differences, we have in this case only *one* finite non-empty set A and a quaternary relation D on A (that is, a binary relation on $A \times A$). The problem is to characterize those D realizable by a single utility function f on A such that
>
> $$xyDzw \quad \text{if and only if} \quad f(x) - f(y) \geqslant f(z) - f(w)$$
>
> for all $x, y, z, w \in A$.
>
> Note that if we define a relation V in terms of D by the condition
>
> $$xwVzy \quad \text{if and only if} \quad xyDzw$$
>
> for all $x, y, z, w \in A$, then D is also definable in terms of V. Further, if D is realizable in the sense just defined, then V is realizable in the sense of Theorem 3.1 where $A = A^*$ and the two utility functions coincide. Suppose we assume (1_V) and (2_V) of Theorem 3.1, which can be equivalently rewritten in terms of D. (By the remark just made, these are necessary conditions for D to be realizable.) In view of Theorem 3.1 there must exist a *pair* of utility functions on A, which we shall call g and h, such that
>
> $$xyDzw \quad \text{if and only if} \quad g(x) - h(y) \geqslant g(z) - h(w)$$
>
> for all $x, y, z, w \in A$. Unfortunately, g and h need not be the same function, so the problem is not yet solved.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem RealizableDifference.utilityPair {A : Type u}
    {D : A → A → A → A → Prop} (hD : RealizableDifference D) :
    RealizableUtilityPair D
```

Mathematical reconstruction from Lean.
Scott defines an auxiliary V by swapping slots, $xw\,V\,zy$ iff $xy\,D\,zw$, then invokes Theorem 3.1 to obtain g and h with $g(x) - h(y)$ representing D. Lean never names that V. It treats D itself as a pair relation and represents it by $g(x) + q(y)$, i.e. $g(x) - h(y)$ with $q = -h$. The algebra is the same; the intermediate relabeling is not a definition in the library.

C-III07 (source lines 382-417)

Reversal and the sum of the two scales.
Scott 1964 (verbatim).

> To complete the solution, we note that not all necessary conditions for D to be realizable by a single utility function have yet been appreciated. In particular, there is an obvious *commutativity* condition that D must satisfy to be realizable:
>
> $$xyDzw \quad \text{implies} \quad wzDyx$$
>
> for all $x, y, z, w \in A$. Assuming this property we see that if $xyDzw$ holds, then
>
> $$g(x) - h(y) \geqslant g(z) - h(w)$$
>
> and
>
> $$g(w) - h(z) \geqslant g(y) - h(x).$$
>
> Therefore,
>
> $$h(x) - g(y) \geqslant h(z) - g(w),$$
>
> and so, by adding inequalities,
>
> $$[g(x) + h(x)] - [g(y) + h(y)] \geqslant [g(z) + h(z)] - [g(w) + h(w)].$$
>
> MEASUREMENT STRUCTURES AND LINEAR INEQUALITIES
>
> Similarly, if $xy\ D\ zw$ does not hold, then neither does $wz\ D\ yx$. Whence, we find that
>
> $$g(x) - h(y) < g(z) - h(w),$$
>
> and
>
> $$g(w) - h(z) < g(y) - h(x),$$
>
> and so
>
> $$[g(x) + h(x)] - [g(y) + h(y)] < [g(z) + h(z)] - [g(w) + h(w)].$$

Lean 4 correspondence (exact source).

Lean 4 source

```
def DiffReversal {A : Type u} (D : A → A → A → A → Prop) : Prop :=
  ∀ x y z w, D x y z w → D w z y x

theorem difference_of_pair_and_reversal {A : Type u}
    {D : A → A → A → A → Prop} (hp : RealizableUtilityPair D)
    (hr : DiffReversal D) : RealizableDifference D
```

Mathematical reconstruction from Lean.
Reversal is the commutativity $xy\,D\,zw \Rightarrow wz\,D\,yx$. From a pair representation g, q one obtains both $g(x) + q(y) \geq g(z) + q(w)$ and the reversed inequality. Adding them produces the single difference $f(x) - f(y)$ with $f = g - q$. The converse direction is the same addition on the strictly opposite inequalities. This is Scott's $f = g + h$ after the sign convention $q = -h$.

C-III08 (source lines 418-429)

Theorem 3.2.
Scott 1964 (verbatim).

> In other words, define the function f on A so that $f(x) = g(x) + h(x)$ for all $x \in A$, then f realizes D in the required sense. This discussion leads directly to our main result.
>
> **THEOREM 3.2.** *Let A be a finite nonempty set and let D be a quaternary relation on A. For D to be realizable by a single utility function on A it is necessary and sufficient that the conditions*
>
> $$(1_D) \quad xy\ D\ zw \quad \text{or} \quad zw\ D\ xy,$$
>
> $$(2_D) \quad x_i y_i\ D\ x_{\pi(i)} y_{\sigma(i)} \quad \text{for} \quad i < n,\ i > 0, \quad \text{implies} \quad x_{\pi(0)} y_{\sigma(0)}\ D\ x_0 y_0,$$
>
> $$(3_D) \quad xy\ D\ zw \quad \text{implies} \quad wz\ D\ yx,$$
>
> *hold for all $x, y, z, w \in A$ and all sequences $x_0, \ldots, x_{n-1}, y_0, \ldots, y_{n-1} \in A$ and all permutations π, σ of $\{0, \ldots, n-1\}$, where $n > 0$.*

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem theorem_3_2 {A : Type u} [Fintype A] [Nonempty A]
    (D : A → A → A → A → Prop) :
    RealizableDifference D ↔
      DiffTotal D ∧ DiffPermutation D ∧ DiffReversal D
```

Mathematical reconstruction from Lean.
The three axioms are totality, the same permutation scheme as (2_V), and reversal. Sufficiency applies Theorem 3.1 to D as a pair relation and then `difference_of_pair_and_reversal`. Lean therefore follows Scott's architecture: one-utility differences are two-utility pairs plus commutativity.

C-III09 (source lines 430-431)

The infinite bundle (2_D).
Scott 1964 (verbatim).

> It should be remarked that (2_D) is an infinite bundle of conditions (for each $n > 0$, each π, σ), and it was shown in Scott and Suppes (1958), that no finite number of them could be sufficient.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `DiffPermutation`, which quantifies over every `n` and both permutations.
Mathematical reconstruction from Lean.
Scott remarks that no finite truncation of the permutation scheme is sufficient, citing Scott–Suppes 1958. Lean does not prove that non-finite-axiomatizability result. It does keep (2_D) as a genuine infinite scheme: the universal quantifiers over `n`, π, and σ are essential to the definition.

4.5 IV. Subjective probability

C-IV01 (source lines 432-439)

Section IV: qualitative probability.
Scott 1964 (verbatim).

> ## IV. SOLUTION TO THE PROBLEM OF SUBJECTIVE PROBABILITIES
>
> Let B be a finite Boolean algebra that we can call the algebra of *events*. A binary relation $\succcurlyeq$ is given on B, where $x \succcurlyeq y$ means that the event x is *more probable than* the event y in some subjective sense. We wonder if these subjective decisions about probabilities are *rational* in the sense that $\succcurlyeq$ is realizable by some real-valued probability measure μ on B where
>
> $$x \succcurlyeq y \quad \text{if and only if} \quad \mu(x) \geqslant \mu(y)$$
>
> for all $x, y \in B$.

Lean 4 correspondence (exact source).
The existence question is again `RealizableProbability`.
Mathematical reconstruction from Lean.
A finite Boolean algebra of events carries a binary comparison intended as subjective probability. Realization means a genuine finitely additive probability, not a signed charge. The infinite case is postponed to the last paragraph and is not a numbered theorem of the paper.

C-IV02 (source lines 440-455)

de Finetti's five axioms.
Scott 1964 (verbatim).

> It seems that de Finetti first raised this question and proposed these obviously necessary axioms for $\succcurlyeq$ for all $x, y, z, w \in B$:
>
> (i) not $0 \succcurlyeq 1$,
>
> (ii) $x \succcurlyeq 0$,
>
> (iii) $x \succcurlyeq y$ or $y \succcurlyeq x$,
>
> (iv) $x \succcurlyeq y \succcurlyeq z$ implies $x \succcurlyeq z$,
>
> (v) $x \succcurlyeq y$ if and only if $x \cup z \succcurlyeq y \cup z$,
>
> in case z is disjoint from both x and y.

Lean 4 correspondence (exact source).

Lean 4 source

```
structure DeFinettiAxioms {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop) : Prop where
  nontrivial : ¬R ⊥ ⊤
  bottom : ∀ x, R x ⊥
  total : ProbTotal R
  transitive : ProbTransitive R
  disjointUnionInvariant : ProbDisjointUnionInvariant R
```

Mathematical reconstruction from Lean.
Lean bundles the five axioms Scott attributes to de Finetti. Condition (i) is the weak nontriviality $\neg\bot \succsim \top$, not Scott's later (1_B) which also demands $\top \succsim \bot$. Condition (v) is invariance of comparisons under disjoint union with a third event. These axioms are recorded in order to state the KPS insufficiency theorem; they are not the hypotheses of Theorem 4.1.

C-IV03 (source lines 456-459)

The KPS counterexample and atom vectors.
Scott 1964 (verbatim).

> It was shown in Kraft, Pratt, and Seidenberg (1959) by an ingenious counter-example in a 32-element Boolean algebra that the axioms (i)–(v) are *not* sufficient. The proper strengthening of the axioms was also given in Theorem 2 of that paper and we shall derive what amounts to their result as a consequence of Theorem 1.3.
>
> Let S be the (finite) set of atoms of B. In the well-known way, we can identify B with the set of $\{0,1\}$-valued functions on S (that is, the set of characteristic functions of subsets of S.) Thus we imagine that B is a subset of $L = L(S)$, and so the 0 element of B is the 0 vector of L, the 1 element of B is the vector with coordinates all equal to 1. Further if $x, y \in B$ are disjoint, then the vector sum $x + y = x \cup y$. Of course, if $x, y \in B$ overlap,

then $x + y \notin B$. We shall write $x \succ y$ for not $y \succcurlyeq x$ and $x \preccurlyeq y$ for $y \succcurlyeq x$. With these conventions the desired result can be formulated as follows:

Lean 4 correspondence (exact source).

Lean 4 source

```
def atomVector {B : Type u} [BooleanAlgebra B] (x : B) :
    {a : B // IsAtom a} → ℝ :=
  fun a ↦ if a.1 ≤ x then 1 else 0
```

Mathematical reconstruction from Lean.
The 32-element counterexample is the powerset of five atoms with the exact KPS rank order `KPSGe`. Lean verifies the bundled de Finetti axioms by finite inspection and proves non-representability from four strict inequalities among singleton weights, not by exhaustive search over measures. Atom vectors are the characteristic functions of the atoms below an event; disjoint suprema become vector sums. Scott's $x \succ y$ is `StrictlyPreferred R x y := ¬ R y x`.

C-IV04 (source lines 460-473)

Theorem 4.1.
Scott 1964 (verbatim).

THEOREM 4.1. *Let B be a finite Boolean algebra and let $\succcurlyeq$ be a binary relation on B. For $\succcurlyeq$ to be realizable by a probability measure on B it is necessary and sufficient that the conditions*

$$(1_B) \quad 1 \succ 0,$$

$$(2_B) \quad x \succcurlyeq 0,$$

$$(3_B) \quad x \succcurlyeq y \quad \text{or} \quad x \preccurlyeq y,$$

$$(4_B) \quad x_0 + x_1 + \cdots + x_{n-1} = y_0 + y_1 + \cdots + y_{n-1} \quad \text{implies} \quad x_0 \preccurlyeq y_0,$$

hold for all $x, y \in B$ and all sequences $x_0, \ldots, x_{n-1}, y_0, \ldots, y_{n-1} \in B$, where $x_i \succcurlyeq y_i$ for $i < n$, $i > 0$, and $n > 0$.

The unpleasant feature of (4_B) is that it is not a strictly Boolean condition: $x_0 + x_1 + \cdots + x_{n-1}$ means the *algebraic sum* of characteristic functions and *does not* stand for the *union* of the x_i. However, the equation in the hypothesis of (4_B) does have a reasonably simple interpretation in words: every point (atom) belongs to exactly the same number of the x_i as the y_i.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem theorem_4_1 {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧
      ProbTotal R ∧ ProbCancellation R
```

The displayed algebraic sum is `theorem_4_1_vector`. The atom-count reading Scott immediately supplies is the primary `ProbCancellation`.

Mathematical reconstruction from Lean.

(1_B) is a strict comparison of $\top$ with $\bot$; (2_B) is nonnegativity; (3_B) is totality; (4_B) is cancellation of sequences with equal atom multiplicities. Lean's primary statement uses that atom-count reading. The literal equality of characteristic-vector sums is an equivalent form, and `sum_atomVector_eq_iff` identifies the two languages. The distinguished index is again `finHead n`.

C-IV05 (source lines 474-483)

Proof of Theorem 4.1.
Scott 1964 (verbatim).

> **PROOF OF THEOREM 4.1.** There is no question that the conditions are necessary. To apply Theorem 1.3, we simply take $Y = B$. Thus there will be a linear functional φ on L that realizes $\succcurlyeq$ on B. In view of (1_B) we have $\varphi(1) > 0$; while $\varphi(x) \geqslant 0$ for
>
> $x \in B$ follows from (2_B). Then the measure μ we are looking for is defined by the equation:
>
> $$\mu(x) = \varphi(x)/\varphi(1)$$
>
> for all $x \in B$.

Lean 4 correspondence (exact source).

Totality and cancellation are transported to the finite rational range of `atomVector` and discharged by Theorem 1.3. The resulting `AtomLinearRepresentation` is normalized by $\varphi(\mathbf{1})$.

Mathematical reconstruction from Lean.

Necessity is evaluation of a probability. Sufficiency applies Theorem 1.3 to atom vectors. The functional on incidence vectors is already a finitely additive signed charge on events. Nontriviality makes its value at $\top$ positive; nonnegativity makes every event value nonnegative. Division by $\varphi(\top)$ is Scott's $\mu(x) = \varphi(x)/\varphi(1)$ and does not change comparisons. This is the printed short proof.

C-IV06 (source lines 484-487)

Signed charges and de Finetti's (v).
Scott 1964 (verbatim).

> This argument shows that Theorem 4.1 is almost immediately a special case of Theorem 1.3. If we had left out (1_B) and (2_B), the best we could have said is that $\succcurlyeq$ is realized by a *signed* measure on B.
>
> The derivation of (v) from (3_B) and (4_B) is quite easy. Suppose $x \succcurlyeq y$ but not $x+z \succcurlyeq y+z$.

Then $y+z \succcurlyeq x+z$ and $(y+z)+x = (x+z)+y$. So by (4_B) $y+z \preccurlyeq x+z$ must indeed hold. The converse is analogous.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem scott_p15_signed_charge (R : B → B → Prop) :
    RealizableSignedCharge R ↔ ProbTotal R ∧ ProbCancellation R

theorem ProbCancellation.disjointUnionInvariant
    {B : Type u} [BooleanAlgebra B] [Fintype B] {R : B → B → Prop}
    (h : ProbCancellation R) : ProbDisjointUnionInvariant R
```

Mathematical reconstruction from Lean.
Dropping (1_B) and (2_B) leaves a signed finitely additive charge, which is Scott's p. 15 remark. The derivation of disjoint-union invariance is the two-term instance of (4_B) that he sketches: if $x \succsim y$ but not $x \cup z \succsim y \cup z$, totality and the identity $(y \cup z) + x = (x \cup z) + y$ force the opposite comparison. Lean also derives reflexivity and transitivity from cancellation, connecting Scott's criterion to the older qualitative axioms.

C-IV07 (source lines 488-489)

The unpublished infinite extension.
Scott 1964 (verbatim).

> The author has obtained an extension of Theorem 4.1 to infinite Boolean algebras by using the Hahn-Banach Theorem in the same way that it was applied in Kelley (1959), where the problem of the existence of strictly positive measures was solved. If the result proves to be of interest, it will be published in a future paper.

Lean 4 correspondence (exact source).

Lean 4 source

```
theorem reconstructed_infinite_theorem_4_1 (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧ ProbTotal R ∧
        GeneralizedKelleyCondition R
```

This is a modern theorem. It is not Scott's unpublished result.
Mathematical reconstruction from Lean.
Scott announces an extension via Hahn–Banach and Kelley (1959) and withholds the statement. Lean therefore does not attribute an infinite theorem to the 1964 paper. The library proves a separately labelled reconstruction: events embed in a normed span of evaluation functions, weak comparisons generate a closed cone, and a countable Kelley cover of the strict comparisons produces one functional that is non-negative on weak comparisons and positive on strict ones. Normalization again yields a finitely additive probability. The extra hypothesis `GeneralizedKelleyCondition` is explicit and modern. The literature basis for the priority sentence is the audit in Section 3.5.10.

4.6 References in Scott 1964

C-R00 (source lines 490-491)

References heading.
Scott 1964 (verbatim).

> ## REFERENCES

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
Bibliographic only. The citation is recorded in the report bibliography; it is not a Lean declaration.

C-R01 (source lines 492-493)

Adams and Fagot 1956.
Scott 1964 (verbatim).

> ADAMS, E. W., AND FAGOT, R. F. A model of riskless choice. *Rep. No. 4, Applied Mathematics and Statistics Laboratory, Stanford University*, 1956.

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
Bibliographic only. The citation is recorded in the report bibliography; it is not a Lean declaration.

C-R02 (source lines 494-495)

Luce 1956.
Scott 1964 (verbatim).

> LUCE, R. D. Semi-orders and a theory of utility discrimination. *Econometrica*, 1956, **24**, 178-191.

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
Bibliographic only. Luce's semiorders motivate Problem I but are not imported as a Lean theory.

C-R03 (source lines 496-497)

Luce and Tukey 1964.
Scott 1964 (verbatim).

> LUCE, R. D., AND TUKEY, J. W. Simultaneous conjoint measurement: a new type of fundamental measurement. *J. math. Psychol.*, 1964, **1**, 1-27.

Lean 4 correspondence (exact source).

No direct Lean counterpart.
Mathematical reconstruction from Lean.
Bibliographic only. The citation is recorded in the report bibliography; it is not a Lean declaration.

C-R04 (source lines 498-499)

Kelley 1959.
Scott 1964 (verbatim).

> KELLEY, J. L. Measures on Boolean algebras. *Pacific J. Math.*, 1959, **18**, 1165-1172.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `reconstructed_infinite_theorem_4_1` and `KelleyCover`, which follow Kelley's method without claiming Scott's unpublished theorem.
Mathematical reconstruction from Lean.
The report cites the correct *Pacific Journal of Mathematics* volume for Kelley. The vision transcription's volume number is an OCR discrepancy, not a Lean fact.

C-R05 (source lines 500-501)

Kraft, Pratt, and Seidenberg 1959.
Scott 1964 (verbatim).

> KRAFT, C. H., PRATT, J. W., AND SEIDENBERG, A. Intuitive probability on finite sets. *Ann. Math. Statist.*, 1959, **30**, 408-419.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `deFinetti_axioms_insufficient` and the `KPSGe` order.
Mathematical reconstruction from Lean.
The citation is bibliographic. The mathematical content of that paper used here is the five-atom order already aligned above.

C-R06 (source lines 502-503)

Kuhn and Tucker 1956.
Scott 1964 (verbatim).

> KUHN, H. W., AND TUCKER, A. W. (Eds.) Linear inequalities and related systems. *Annals of mathematics studies*, No. 38, 1956.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `finite_strict_separation`.
Mathematical reconstruction from Lean.
Scott's separation citation is replaced, in the formal proof, by Mathlib's compact-closed Hahn–Banach theorem. The book is not a dependency of the Lean library.

C-R07 (source lines 504-505)

Scott and Suppes 1958.
Scott 1964 (verbatim).

> SCOTT, D., AND SUPPES, P. Foundational aspects of theories of measurement. *J. Symbolic Logic*, 1958, **23**, 113-128.

Lean 4 correspondence (exact source).
No direct Lean counterpart. The nearest genuine formal object is `finite_staircase_representation`.
Mathematical reconstruction from Lean.
The 1958 paper is the source of the completed converse to Theorem 2.1 and of the remark that (2_D) is not finitely axiomatizable. Only the representation construction is formalized here.

C-R08 (source lines 506-507)

Suppes and Zinnes 1963.
Scott 1964 (verbatim).

> SUPPES, P., AND ZINNES, J. L. Basic measurement theory. *Handbook of mathematical psychology*, Vol. 1. New York: Wiley, 1963. Pp. 1-76.

Lean 4 correspondence (exact source).
No direct Lean counterpart.
Mathematical reconstruction from Lean.
Bibliographic only. The citation is recorded in the report bibliography; it is not a Lean declaration.

5 Verification and Automated Pipeline

5.1 Kernel verification and reproducible build

The repository pins Lean and mathlib **v4.35.0-rc2**.

```
lake exe cache get
lake build
python3 scripts/check_concordance.py
bash scripts/generate_arxiv_with_code.sh              # → arxiv_with_code.md
```

`lake build` checks the `Scott1964` library (and optional `Solution` re-export).

`arxiv_with_code.md` is generated (narrative, Concordance methodology appendix, complete `MeasurementStructures` Lean source, and the Scott 1964 source PDF as Appendix C). Regenerate when `arxiv.md` or listed sources change; build the PDF and arXiv zip with `bash scripts/build_arxiv_pdf.sh`.

The source-fidelity workflow begins with `sources/ScottMeasurement1964.pdf`. The reproducible OCR command `scripts/ocr_pdf_pipeline.sh` produces the searchable triple-pass vision transcription used to compare formal statements with the paper. Generated proof text is accepted only after elaboration and kernel checking by the pinned Lean toolchain; the PDF pipeline is separate from the trusted proof kernel.

5.2 LLM-assisted drafting and interactive proof verification

Large language models assisted with source transcription, theorem inventory, Lean proof drafting, build repair, and preparation of this report. Their outputs were treated as provisional edits rather than evidence: a declaration entered the library only when Lean elaborated it and the kernel accepted its proof term. Repository-level checks build the complete import graph, scan the completed source tree for proof holes, and report the axiom footprint.

This division is methodological. Automation can propose definitions, proof strategies, and prose, but the durable certificate is the checked Lean environment under the pinned version of Mathlib. The human authors remain responsible for the selection and interpretation of mathematical claims; no large language model is listed as an author.

Verification and explanation therefore serve different, complementary roles. Lean checks that a proof term establishes the formal theorem from its stated assumptions. Human review checks the other direction of the translation: that the theorem is neither weaker nor stronger than the source claim, that its definitions carry the intended interpretation, and that divergences are reported rather than hidden. The concordance and mathematical reconstructions make this human-facing check inspectable instead of treating formal source code as self-explanatory.

6 Discussion and Future Work

The formalization shows that Scott's common linear-inequality engine can be made explicit enough for a modern proof assistant while retaining the distinct combinatorics of preference, utility-difference, and probability applications. The verified equivalence between atom-count and literal characteristic-vector cancellation is especially useful for separating the paper's mathematical content from representational choices made in Lean.

Formalization of pure mathematics already provides many examples of machine-checked proof. The additional opportunity in this case is presentational: Scott's examples are understandable outside the specialty, yet they exercise a nontrivial stack of order theory, combinatorics, linear algebra, convex separation, and probability. The layered account used here can therefore serve as a model for reports aimed at both formalization experts and mathematically interested readers: state the motivating question, expose the reusable library boundary, show the formal certificate, and translate the certificate back into a source-facing argument.

Two proof-development questions remain open. First, `Preference/Cycle.lean` records Scott's local cycle-shortening lemmas, but a second proof of Theorem 2.1 through the section-1 separation machinery has not yet been assembled. Second, Scott's closing announcement of an infinite extension does not supply a recoverable theorem statement. The present Hahn–Banach/Kelley result should therefore remain a modern, explicitly hypothesized analogue rather than an attribution to the 1964 paper. Based on the literature audit in Section 3.5.10, the formal characterization `reconstructed_infinite_theorem_4_1` provides the first explicit, self-contained realization of Scott's announced infinite extension in the published literature.

Scott's later work in domain theory concerns a different class of representation problems and is not a dependency of this development. Nevertheless, both programs study how qualitative order-theoretic structure admits concrete mathematical models. Future work may compare proof patterns across the independently maintained formalizations of Scott's 1972, 1976, 1980, and 1982 papers, while preserving their separate source claims and dependency graphs. External mathematical review and broader interoperability with formalized decision theory are also natural next steps.

7 Code Availability and Archival

7.1 GitHub repository

The complete Lean development, report source, build scripts, and source alignment documentation are available at https://github.com/catskillsresearch/scott1964 [13]. The principal machine-checked artifact is the sorry-free `Scott1964/MeasurementStructures/` library; `Scott1964/MeasurementStructures/Basic.lean` re-exports the complete development. `Challenge.lean`, `Solution.lean`, and `comparator.json` provide the formal statement/solution interface used for independent comparison.

The generated review copy appends the complete `MeasurementStructures` source, with one heading per file. The report build creates the PDF, including sanitized Lean listings and pre-rendered dependency figures.

7.2 Palomar registry

The compared statements are registered with Palomar as **PALOMAR-2026-09-25-000015**, version 1 [14]. That registry entry records a Mathlib-only statement of *Formalization of Scott's Measurement Structures and Linear Inequalities in Lean 4*, by Lars Warren Ericson, Dana S. Scott, Vijay D'Silva, and Brian Milnes.

7.3 License and source PDF

Original Lean code and author-written documentation are Apache-2.0. The source paper PDF, its transcription, and the verbatim Scott quotations in Section 4 are not Apache-2.0; see the repository `NOTICE` and source-material README for the copyright carve-out. Republication of the concordance requires clearance for the quoted 1964 text.

8 Acknowledgments

8.1 AI-assisted development

Lean in this repository was drafted with AI-agent assistance under Lars Warren Ericson's direction and review. The human authors retain responsibility for the mathematical content, the formalization route, and every formal claim. **No large language model is listed as a co-author.**

We gratefully acknowledge assistance from the following tools:

- **Cursor** [15] — agent-assisted editing in the Cursor IDE: formalizing Scott's 1964 *Measurement Structures and Linear Inequalities* in Lean 4 / mathlib, `lake build` repair, vision-OCR transcription, drafting this narrative (`arxiv.md`), and tracking the formalized inventory. Generated Lean was provisional until it compiled under the pinned toolchain.
- **Cursor Composer 2.5 Fast** [16] — routine multi-step work: module scaffolding, dependency-ordered wiring of `Scott1964/MeasurementStructures/`, documentation, and medium proof obligations where the strategy was already fixed.
- **Anthropic Claude Sonnet 5 (medium reasoning)** [17] — day-to-day formalization and proof-engineering in Cursor at the medium reasoning tier: inventory and narrative maintenance, module wiring, and medium-complexity Lean obligations where the proof strategy was already fixed.

- **Anthropic Claude Opus 4.8 (high reasoning)** [18] — selective use for the heaviest proof work. Every emitted proof term was checked by the Lean kernel.
- **OpenAI GPT 5.6** [19] — used to produce the source–Lean concordance in Section 4: partitioning Scott 1964, aligning each passage to Lean declarations or an explicit gap, and writing Lean-driven mathematical reconstructions rather than tactic decompilations. The working method is recorded in the Concordance methodology appendix.

9 References

References

[1] R. D. Luce. Semiorders and a theory of utility discrimination. *Econometrica*, 24:178–191, 1956.

[2] D. Scott and P. Suppes. Foundational aspects of theories of measurement. *Journal of Symbolic Logic*, 23:113–128, 1958.

[3] C. H. Kraft, J. W. Pratt, and A. Seidenberg. Intuitive probability on finite sets. *Annals of Mathematical Statistics*, 30:408–419, 1959.

[4] R. D. Luce and J. W. Tukey. Simultaneous conjoint measurement: A new type of fundamental measurement. *Journal of Mathematical Psychology*, 1:1–27, 1964.

[5] D. S. Scott. Measurement structures and linear inequalities. *Journal of Mathematical Psychology*, 1:233–247, 1964.

[6] D. H. Krantz, R. D. Luce, P. Suppes, and A. Tversky. *Foundations of Measurement, Volume I: Additive and Polynomial Representations*. Academic Press, 1971.

[7] J. L. Kelley. Measures on Boolean algebras. *Pacific Journal of Mathematics*, 9:1165–1177, 1959.

[8] M. A. Cohen. Necessary and sufficient conditions for existence and uniqueness of weak qualitative probability structures. *Journal of Mathematical Psychology*, 35:242–259, 1991.

[9] M. Hardy. Scaled Boolean algebras. *Advances in Applied Mathematics*, 29, 2002.

[10] Y. Ding, M. Harrison-Trainor, and W. H. Holliday. The logic of comparative cardinality. *Journal of Symbolic Logic*, 85:972–1005, 2020.

[11] X. Fu and Z. Zhao. Modal logic with "Most". *Studia Logica*, 2024.

[12] P. C. Fishburn. The axioms of subjective probability. *Statistical Science*, 1:335–345, 1986.

[13] Lars Warren Ericson, Dana S. Scott, Vijay D'Silva, and Brian Milnes. Formalization of Scott's measurement structures and linear inequalities in Lean 4, 2026. Repository. https://github.com/catskillsresearch/scott1964.

[14] Palomar Registry. Formalization of Scott's measurement structures and linear inequalities in Lean 4, 2026. PALOMAR-2026-09-25-000015, version 1. https://palomar-registry.org/entry.html?id=PALOMAR-2026-09-25-000015&version=1.

[15] Anysphere, Inc. Cursor: AI-native code editor and agent environment, 2026. Accessed 2026. https://cursor.com.

[16] Anysphere, Inc. Composer 2.5, 2026. Model announcement, https://cursor.com/blog/composer-2-5; model card as integrated in Cursor, https://cursor.com/docs/models. Accessed 2026.

[17] Anthropic. Claude sonnet 5 (medium reasoning variant), 2026. System card, https://www.anthropic.com/claude-sonnet-5-system-card; model documentation as integrated in Cursor, https://cursor.com/docs/models. Accessed 2026.

[18] Anthropic. Claude opus 4.8 (high thinking/reasoning variant), 2026. System card and announcement, https://www.anthropic.com/news/claude-opus-4-8; model documentation as integrated in Cursor, https://cursor.com/docs/models/claude-opus-4-8. Accessed 2026.

[19] OpenAI. GPT-5.6, 2026. Model documentation as integrated in Cursor, https://cursor.com/docs/models. Accessed 2026.

A Concordance methodology

This appendix records the method used to write Section 4. It is a public version of the working plan that guided the concordance: a paper-order alignment of Scott 1964 with the Lean development, together with a readable reconstruction of each formal definition or proof. GPT 5.6 was used for that alignment and drafting. The cards themselves remain in Section 4, not here.

A.1 Report form and controls

Section 4, *Source–Lean Concordance and Mathematical Reconstruction*, is a numbered main-matter section of this report, placed before the verification section. It is not an appendix.

Each card uses one repeatable three-part format: **Scott 1964 (verbatim)**, **Lean 4 correspondence (exact source)**, and **Mathematical reconstruction from Lean**. Every card carries a stable identifier, a source-line span in `sources/ScottMeasurement1964_vision.md`, and a Lean declaration field (or `none`).

The Markdown cards remain readable in `arxiv.md`. The PDF build turns the hidden card markers into breakable boxes via `scripts/tex_preamble_arxiv.tex` and `scripts/build_arxiv_tex.py`.

The expository model is the Liquid Tensor Experiment blueprint: motivation, precise informal mathematics, proof architecture, and unobtrusive Lean attribution, without tactic-by-tactic narration (https://leanprover-community.github.io/liquid/, https://leanprover-community.github.io/liquid/BD_orig.html).

A.2 Exhaustive alignment method

The vision transcription is partitioned in paper order at sentence, displayed formula, definition, proof-step, example, and bibliographic-entry boundaries. The partition covers the title, abstract, introductory problems, Sections I–IV, closing remarks, and references. OCR front matter and reconciliation metadata are identified separately from the paper text.

For each unit, the relevant declaration and its proof dependencies in `Scott1964/Measurement Structures` are inspected. The Lean panel quotes the actual definition, theorem, or proof fragment, including helper declarations from other modules when they carry the mathematical argument.

No correspondence is invented. For historical prose, motivation, citations, or unformalized steps, the Lean panel says **No direct Lean counterpart** and identifies only the nearest genuine formal object. Alternate routes and gaps are marked in place—especially Theorem 2.1's staircase proof versus Scott's unassembled linear/cycle route, the partial ordered-group example, and the modern infinite theorem.

The third panel is written from the formal proof's mathematical structure: the objects, invariants, reductions, imported Mathlib results, and why the conclusion follows. Tactic names are avoided unless a named lemma materially identifies the mathematics.

A.3 Paper-order drafting passes

- Introduction and Section I: definitions, Theorems 1.1–1.4, separation, rationalization, relation cancellation, additive closure, the non-Archimedean warning, and the finite ordered-group embedding.
- Sections II–III: intransitive indifference and the printed cycle prose; the Scott–Suppes staircase proof actually used by `theorem_2_1`; pair incidence vectors, permutation extraction, and the single-utility difference construction.
- Section IV and the references: de Finetti axioms, atom vectors, Theorem 4.1, signed charges, the KPS counterexample, the unpublished infinite announcement versus the labelled modern reconstruction, and bibliography-only cards.

A.4 Traceability and verification

Stable card identifiers and source-span metadata are embedded in `arxiv.md`. `scripts/check_concordance.py` reports omitted or duplicated source spans and stale or missing Lean declaration references. `scripts/emit_concordance.py` regenerates the card body from those spans so quotations cannot drift from the transcription. The OCR transcription is a navigation aid; doubtful quotations are checked against the source PDF.

The report's source-fidelity discussion and `formalization.yaml` point to the exhaustive concordance and keep the existing divergence disclosures. Compared Lean declarations and generated report artifacts are not edited by hand.

The concordance checker, `lake build`, mechanical Palomar preflight, and the PDF build are run after regeneration. Card page breaks, code wrapping, cross-references, and first-to-last-paper coverage are inspected. The source-material licensing carve-out is preserved; verbatim republication of the 1964 text requires publication-rights clearance before release.

B Complete Lean source

The complete source of `Scott1964/MeasurementStructures`, checked by `lake build`. Directory names are subsections. Each filename is a heading in this appendix. Repository: https://github.com/catskillsresearch/scott1964.

B.1 FinHead.lean

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Mathlib.Data.Fin.Basic

/-!
# Stable `Fin (n + 1)` head index

Compared Scott permutation/cancellation conditions mention the distinguished
index `0 : Fin (n + 1)`.  Elaborating that literal attaches a generated
`._proof_N` to whichever declaration first needs `OfNat`.  Palomar compares
those generated constants, so Challenge and Solution must share a named
proof of `0 < n + 1` instead.
-/

namespace Scott1964.MeasurementStructures

theorem fin_succ_pos (n : ℕ) : 0 < n + 1 :=
  Nat.succ_pos n

/-- The distinguished index in a nonempty Scott sequence of length `n + 1`. -/
def finHead (n : ℕ) : Fin (n + 1) :=
  ⟨0, fin_succ_pos n⟩

theorem finHead_eq_zero (n : ℕ) : finHead n = 0 :=
  Fin.ext rfl

end Scott1964.MeasurementStructures
```

B.2 LinearInequalities

B.2.1 Definitions.lean

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Mathlib.Analysis.Convex.Topology
import Mathlib.LinearAlgebra.FiniteDimensional.Basic

/-!
# Scott's finite systems of linear inequalities
```

```
Definitions corresponding to §1 of Scott (1964), pp. 233–237.
-/

open Set

namespace Scott1964.MeasurementStructures.LinearInequalities

variable {L : Type*} [AddCommGroup L] [Module ℝ L]

/-- A set is symmetric when it is closed under negation. -/
def Symmetric (X : Set L) : Prop := ∀ {{x}}, x ∈ X → -x ∈ X

/-- `N` is realized on `X` by a real linear functional. -/
def Realizable (X N : Set L) : Prop :=
  ∃ φ : Module.Dual ℝ L, ∀ x ∈ X, x ∈ N ↔ 0 ≤ φ x

/-- Scott's notation `x ≻ 0`: positive, but its negative is not positive. -/
def strictPositive (X N : Set L) : Set L :=
  {x | x ∈ X ∧ x ∈ N ∧ -x ∉ N}

/-- Vectors declared both nonnegative and nonpositive. -/
def indifferent (X N : Set L) : Set L :=
  {x | x ∈ X ∧ x ∈ N ∧ -x ∈ N}

/-- Condition (1): every vector has one of the two weak signs. -/
def SignComplete (X N : Set L) : Prop :=
  ∀ {{x}}, x ∈ X → x ∈ N ∨ -x ∈ N

/-- Geometric form of Scott's condition (2).

No convex combination of strictly positive vectors belongs to the linear
span of vectors carrying both weak signs. For finite symmetric systems this
is equivalent to Scott's positive-coefficient cancellation condition.
-/
def WeightedCancellation (X N : Set L) : Prop :=
  Disjoint (convexHull ℝ (strictPositive X N))
    (Submodule.span ℝ (indifferent X N) : Set L)

/-- Scott's literal positive-weight finite-sequence condition (2).
Sequences are indexed by `Fin (n + 1)`, making nonemptiness explicit. -/
def WeightedSequenceCancellation (X N : Set L) : Prop :=
  ∀ (n : ℕ) (x : Fin (n + 1) → L) (c : Fin (n + 1) → ℝ),
    (∀ i, x i ∈ X) → (∀ i, x i ∈ N) → (∀ i, 0 < c i) →
    ∑ i, c i • x i = 0 → ∀ i, -x i ∈ N

/-- Scott's literal unweighted finite-sequence condition (4). -/
def UnweightedSequenceCancellation (X N : Set L) : Prop :=
  ∀ (n : ℕ) (x : Fin (n + 1) → L),
    (∀ i, x i ∈ X) → (∀ i, x i ∈ N) →
    ∑ i, x i = 0 → ∀ i, -x i ∈ N

/-- Positive natural-multiplicity version, intermediate between rational
weights and literal repetition. -/
def NatWeightedSequenceCancellation (X N : Set L) : Prop :=
  ∀ (n : ℕ) (x : Fin (n + 1) → L) (k : Fin (n + 1) → ℕ),
    (∀ i, x i ∈ X) → (∀ i, x i ∈ N) → (∀ i, 0 < k i) →
```

```
    ∑ i, k i • x i = 0 → ∀ i, -x i ∈ N

/-- A vector in `S → ℝ` has rational coordinates. -/
def IsRationalVector {S : Type*} (x : S → ℝ) : Prop :=
  ∀ s, ∃ q : ℚ, x s = q

/-- A set consists of rational vectors. -/
def IsRationalSet {S : Type*} (X : Set (S → ℝ)) : Prop :=
  ∀ {{x}}, x ∈ X → IsRationalVector x

/-- Realizability of a relation by comparison of functional values. -/
def RelationRealizable (Y : Set L) (R : L → L → Prop) : Prop :=
  ∃ φ : Module.Dual ℝ L, ∀ x ∈ Y, ∀ y ∈ Y, R x y ↔ φ y ≤ φ x

/-- Condition (5): completeness on `Y`. -/
def RelationComplete (Y : Set L) (R : L → L → Prop) : Prop :=
  ∀ {{x}}, x ∈ Y → ∀ {{y}}, y ∈ Y → R x y ∨ R y x

/-- Strictly positive differences associated to a relation. -/
def strictDifferences (Y : Set L) (R : L → L → Prop) : Set L :=
  {z | ∃ x ∈ Y, ∃ y ∈ Y, R x y ∧ ¬R y x ∧ z = x - y}

/-- Indifferent differences associated to a relation. -/
def indifferentDifferences (Y : Set L) (R : L → L → Prop) : Set L :=
  {z | ∃ x ∈ Y, ∃ y ∈ Y, R x y ∧ R y x ∧ z = x - y}

/-- Geometric form of Scott's equal-sums cancellation condition (6). -/
def RelationCancellation (Y : Set L) (R : L → L → Prop) : Prop :=
  Disjoint (convexHull ℝ (strictDifferences Y R))
    (Submodule.span ℝ (indifferentDifferences Y R) : Set L)

/-- Scott's literal paired equal-sums condition (6). -/
def RelationSequenceCancellation (Y : Set L) (R : L → L → Prop) : Prop :=
  ∀ (n : ℕ) (x y : Fin (n + 1) → L),
    (∀ i, x i ∈ Y) → (∀ i, y i ∈ Y) → (∀ i, R (x i) (y i)) →
    ∑ i, x i = ∑ i, y i → ∀ i, R (y i) (x i)

/-- The additive closure `Y⁺` (including the empty sum). -/
def additiveClosure (Y : Set L) : Set L :=
  AddSubmonoid.closure Y

/-- Scott's three axioms (i)–(iii) for a strictly monotonic relation. -/
structure StrictlyMonotonic (Z : Set L) (R : L → L → Prop) : Prop where
  complete : ∀ {{x}}, x ∈ Z → ∀ {{y}}, y ∈ Z → R x y ∨ R y x
  add : ∀ {{x₀ y₀ x₁ y₁}}, x₀ ∈ Z → y₀ ∈ Z → x₁ ∈ Z → y₁ ∈ Z →
    R x₀ y₀ → R x₁ y₁ → R (x₀ + x₁) (y₀ + y₁)
  cancel : ∀ {{x₀ y₀ x₁ y₁}}, x₀ ∈ Z → y₀ ∈ Z → x₁ ∈ Z → y₁ ∈ Z →
    x₀ + x₁ = y₀ + y₁ → R x₁ y₁ → R y₀ x₀

/-- `Rplus` extends `R` from `Y` to `Z`. -/
def ExtendsOn (Y Z : Set L) (R Rplus : L → L → Prop) : Prop :=
  Y ⊆ Z ∧ ∀ {{x}}, x ∈ Y → ∀ {{y}}, y ∈ Y → (Rplus x y ↔ R x y)

end Scott1964.MeasurementStructures.LinearInequalities
```

B.2.2 Separation.lean

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.LinearInequalities.Definitions
import Mathlib.Analysis.LocallyConvex.Separation
import Mathlib.Analysis.LocallyConvex.WithSeminorms

/-!
# Finite convex separation

The separation lemma used in Scott's proof of Theorem 1.1.
-/

open Set

namespace Scott1964.MeasurementStructures.LinearInequalities

variable {L : Type*} [NormedAddCommGroup L] [NormedSpace ℝ L]
  [FiniteDimensional ℝ L]

/-- A finite set `A` can be made strictly positive while `B` is made zero
exactly when its convex hull misses the span of `B`. -/
theorem finite_strict_separation {A B : Set L} (hA : A.Finite) :
    Disjoint (convexHull ℝ A) (Submodule.span ℝ B : Set L) ↔
      ∃ φ : Module.Dual ℝ L, (∀ x ∈ A, 0 < φ x) ∧ ∀ x ∈ B, φ x = 0 := by
  constructor
  · intro hdisj
    let M : Submodule ℝ L := Submodule.span ℝ B
    have hMclosed : IsClosed (M : Set L) := M.closed_of_finiteDimensional
    obtain ⟨f, u, v, hfA, huv, hfM⟩ :=
      geometric_hahn_banach_compact_closed (convex_convexHull ℝ A)
        (hA.isCompact_convexHull ℝ) M.convex hMclosed hdisj
    have hf_zero : ∀ m ∈ M, f m = 0 := by
      intro m hm
      by_contra hne
      have hscaled := hfM (((v - 1) / f m) • m) (M.smul_mem _ hm)
      have hmap : f (((v - 1) / f m) • m) = v - 1 := by
        rw [map_smul, smul_eq_mul]
        field_simp
      rw [hmap] at hscaled
      linarith
    let φ : Module.Dual ℝ L := -(f : L →L[ℝ] ℝ).toLinearMap
    refine ⟨φ, ?_, ?_⟩
    · intro x hx
      have hxHull : x ∈ convexHull ℝ A := subset_convexHull ℝ A hx
      have hv : v < 0 := by simpa using hfM 0 M.zero_mem
      have hfx : f x < 0 := (hfA x hxHull).trans (huv.trans hv)
      simpa [φ] using neg_pos.mpr hfx
    · intro x hx
```

```
      have hxM : x ∈ M := Submodule.subset_span hx
      simp [φ, hf_zero x hxM]
  · rintro ⟨φ, hφA, hφB⟩
    rw [Set.disjoint_left]
    intro z hzA hzB
    have hzpos : 0 < φ z := by
      exact convexHull_min (fun x hx ↦ hφA x hx)
        (convex_halfSpace_gt φ.isLinear 0) hzA
    have hzM : φ z = 0 := by
      have hle : Submodule.span ℝ B ≤ LinearMap.ker φ :=
        Submodule.span_le.mpr fun x hx => LinearMap.mem_ker.mpr (hφB x hx)
      exact LinearMap.mem_ker.mp (hle hzB)
    linarith

end Scott1964.MeasurementStructures.LinearInequalities
```

B.2.3 Rationalization.lean

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.LinearInequalities.Definitions
import Mathlib.Algebra.Algebra.Rat
import Mathlib.LinearAlgebra.TensorProduct.Finiteness
import Mathlib.LinearAlgebra.TensorProduct.Pi
import Mathlib.RingTheory.Flat.Basic
import Mathlib.Topology.Algebra.Order.Archimedean
import Mathlib.Topology.NhdsWithin

/-!
# Rational vectors

Elementary closure facts used in Scott's reduction from relations to
inequalities on the rational difference set `Y - Y`.
-/

namespace Scott1964.MeasurementStructures.LinearInequalities

variable {S : Type*}

open scoped TensorProduct

theorem isRationalVector_zero : IsRationalVector (0 : S → ℝ) := by
  intro s
  exact ⟨0, by simp⟩

theorem IsRationalVector.neg {x : S → ℝ} (hx : IsRationalVector x) :
    IsRationalVector (-x) := by
  intro s
```

```
  obtain ⟨q, hq⟩ := hx s
  exact ⟨-q, by simp [hq]⟩

theorem IsRationalVector.sub {x y : S → ℝ}
    (hx : IsRationalVector x) (hy : IsRationalVector y) :
    IsRationalVector (x - y) := by
  intro s
  obtain ⟨q, hq⟩ := hx s
  obtain ⟨r, hr⟩ := hy s
  exact ⟨q - r, by simp [hq, hr]⟩

theorem rational_difference_set {Y : Set (S → ℝ)} (hY : IsRationalSet Y) :
    IsRationalSet {z | ∃ x ∈ Y, ∃ y ∈ Y, z = x - y} := by
  rintro z ⟨x, hx, y, hy, rfl⟩
  exact (hY hx).sub (hY hy)

section PositiveKernel

variable {I : Type*} [Fintype I] [DecidableEq I]
  [Fintype S] [DecidableEq S]

/-- A strictly positive real relation between finitely many rational vectors
can be replaced by a strictly positive rational relation.

This is the scalar-extension step in Scott's proof of Theorem 1.2.  Exactness
of tensoring the rational kernel with `ℝ` writes the given real relation as a
finite real linear combination of rational kernel vectors.  Density of a
finite product of copies of `ℚ` then preserves all strict coordinate
inequalities. -/
theorem exists_pos_rational_relation (x : I → S → ℝ)
    (hx : ∀ i, IsRationalVector (x i)) (c : I → ℝ)
    (hc : ∀ i, 0 < c i) (hsum : ∑ i, c i • x i = 0) :
    ∃ q : I → ℚ, (∀ i, 0 < q i) ∧ ∑ i, (q i : ℝ) • x i = 0 := by
  classical
  choose a ha using fun i s ↦ hx i s
  let f : (I → ℚ) →ₗ[ℚ] (S → ℚ) :=
    { toFun := fun d s ↦ ∑ i, d i * a i s
      map_add' := by
        intro d e
        ext s
        simp only [Pi.add_apply, add_mul, Finset.sum_add_distrib]
      map_smul' := by
        intro r d
        ext s
        simp only [Pi.smul_apply, smul_eq_mul, RingHom.id_apply, mul_assoc, Finset.
    mul_sum] }
  let eI := TensorProduct.piScalarRight ℚ ℝ ℝ I
  let eS := TensorProduct.piScalarRight ℚ ℝ ℝ S
  have hc_repr : eI.symm c = ∑ i, c i ⊗ₜ[ℚ] Pi.single i 1 := by
    apply eI.injective
    ext i
    simp [eI, Pi.single_apply]
  have hbase : f.baseChange ℝ (eI.symm c) = 0 := by
    rw [hc_repr]
    apply eS.injective
    ext s
```

```
  have hs := congrFun hsum s
  simpa [eS, f, Pi.single_apply, ha, Algebra.smul_def, smul_eq_mul, mul_comm] using
  hs
have hexact :=
  Module.Flat.lTensor_exact ℝ (LinearMap.exact_subtype_ker_map f)
rw [← LinearMap.baseChange_eq_ltensor] at hexact
obtain ⟨t, ht⟩ := (hexact (eI.symm c)).mp hbase
obtain ⟨k, r, v, rfl⟩ := TensorProduct.exists_sum_tmul_eq t
let U : Set (Fin k → ℝ) :=
  {d | ∀ i, 0 < ∑ j, d j * ((v j : I → ℚ) i : ℝ)}
have hrU : r ∈ U := by
  intro i
  have htc := congrArg eI ht
  have hi := congrFun htc i
  have hi' : ∑ j, r j * ((v j : I → ℚ) i : ℝ) = c i := by
    simpa [eI, Algebra.smul_def, smul_eq_mul, mul_comm] using hi
  rw [hi']
  exact hc i
have hUopen : IsOpen U := by
  rw [show U = ⋂ i, {d | (0 : ℝ) < ∑ j, d j * ((v j : I → ℚ) i : ℝ)} by
    ext d
    simp [U]]
  apply isOpen_iInter_of_finite
  intro i
  apply isOpen_lt continuous_const
  fun_prop
have hdense : DenseRange (Pi.map fun _ : Fin k ↦ ((↑) : ℚ → ℝ)) :=
  DenseRange.piMap fun _ ↦ Rat.denseRange_cast
obtain ⟨q, hqU⟩ := hdense.exists_mem_open hUopen ⟨r, hrU⟩
let w : I → ℚ := fun i ↦ ∑ j, q j * (v j : I → ℚ) i
refine ⟨w, ?_, ?_⟩
· intro i
  have hi : (0 : ℝ) < ∑ j, (q j : ℝ) * ((v j : I → ℚ) i : ℝ) := by
    simpa using hqU i
  have hcast : (w i : ℝ) = ∑ j, (q j : ℝ) * ((v j : I → ℚ) i : ℝ) := by
    simp [w]
  rw [← hcast] at hi
  exact_mod_cast hi
· have hv : ∀ j, f (v j : I → ℚ) = 0 :=
    fun j ↦ LinearMap.mem_ker.mp (v j).property
  ext s
  have hwzero : ∑ i, w i * a i s = 0 := by
    simp_rw [w, Finset.sum_mul]
    rw [Finset.sum_comm, Finset.sum_eq_zero]
    intro j hj
    have hfj := congrFun (hv j) s
    change (∑ i, (v j : I → ℚ) i * a i s) = 0 at hfj
    calc
      ∑ i, q j * (v j : I → ℚ) i * a i s =
          q j * ∑ i, (v j : I → ℚ) i * a i s := by
            rw [Finset.mul_sum]
            apply Finset.sum_congr rfl
            intro i hi
            ring
      _ = 0 := by rw [hfj, mul_zero]
  have hwzeroR := congrArg ((↑) : ℚ → ℝ) hwzero
```

```
    simpa only [Finset.sum_apply, Pi.smul_apply, smul_eq_mul, Pi.zero_apply,
      Rat.cast_sum, Rat.cast_mul, Rat.cast_zero, ha] using hwzeroR

end PositiveKernel

section Repetition

variable {L : Type*} [AddCommGroup L] [Module ℝ L]

/-- Clearing to positive natural coefficients can be discharged by literal
repetition of each vector. -/
theorem UnweightedSequenceCancellation.natWeighted {X N : Set L}
    (h : UnweightedSequenceCancellation X N) :
    NatWeightedSequenceCancellation X N := by
  intro n x k hx hN hk hsum
  let I := Σ i : Fin (n + 1), Fin (k i)
  let i0 : I := ⟨0, ⟨0, hk 0⟩⟩
  have hI : 0 < Fintype.card I := Fintype.card_pos_iff.mpr ⟨i0⟩
  let m := Fintype.card I - 1
  have hcard : Fintype.card I = m + 1 := by
    dsimp [m]
    omega
  let e : Fin (m + 1) ≃ I :=
    Fintype.equivOfCardEq (by simpa using hcard.symm)
  let z : Fin (m + 1) → L := fun j ↦ x (e j).1
  have hzsum : ∑ j, z j = 0 := by
    rw [← hsum]
    change ∑ j, x (e j).1 = ∑ i, k i • x i
    calc
      ∑ j, x (e j).1 = ∑ a : I, x a.1 := e.sum_comp (fun a ↦ x a.1)
      _ = ∑ i, ∑ _j : Fin (k i), x i := Fintype.sum_sigma (fun a : I ↦ x a.1)
      _ = ∑ i, k i • x i := by simp
  have hz := h m z (fun j ↦ hx (e j).1) (fun j ↦ hN (e j).1) hzsum
  intro i
  let a : I := ⟨i, ⟨0, hk i⟩⟩
  simpa [z, a] using hz (e.symm a)

end Repetition

section RationalRepetition

variable {S : Type*} [Fintype S]

/-- On rational coordinate vectors, Scott's unit-weight cancellation
condition implies his arbitrary positive-real-weight condition. -/
theorem UnweightedSequenceCancellation.rationalWeighted
    {X N : Set (S → ℝ)} (h : UnweightedSequenceCancellation X N)
    (hX : IsRationalSet X) : WeightedSequenceCancellation X N := by
  intro n x c hx hN hc hsum
  classical
  obtain ⟨q, hq, hqsum⟩ :=
    exists_pos_rational_relation x (fun i ↦ hX (hx i)) c hc hsum
  let D : ℕ := ∏ i, (q i).den
  have hden : ∀ i, (q i).den ∣ D := by
    intro i
    exact Finset.dvd_prod_of_mem (fun j ↦ (q j).den) (Finset.mem_univ i)
```

```
  choose d hd using hden
  let k : Fin (n + 1) → ℕ := fun i ↦ (q i).num.natAbs * d i
  have hnum : ∀ i, 0 < (q i).num := fun i ↦ Rat.num_pos.mpr (hq i)
  have hk : ∀ i, 0 < k i := by
    intro i
    have hD : 0 < D := Finset.prod_pos fun i _ ↦ (q i).den_pos
    have hdpos : 0 < d i := by
      have := hd i
      nlinarith [(q i).den_pos]
    exact Nat.mul_pos (Int.natAbs_pos.mpr (hnum i).ne') hdpos
  have hkq : ∀ i, (k i : ℚ) = (D : ℚ) * q i := by
    intro i
    have hnumcast : ((q i).num.natAbs : ℤ) = (q i).num :=
      Int.natAbs_of_nonneg (hnum i).le
    rw [← (q i).num_div_den]
    have hnumq : (0 : ℚ) < (q i).num := by exact_mod_cast hnum i
    norm_num [k, hd i, hnumcast, abs_of_pos hnumq]
    field_simp
  have hksum : ∑ i, k i • x i = 0 := by
    calc
      ∑ i, k i • x i = ∑ i, (k i : ℝ) • x i := by
        apply Finset.sum_congr rfl
        intro i hi
        exact (Nat.cast_smul_eq_nsmul ℝ (k i) (x i)).symm
      _ = ∑ i, ((D : ℚ) * q i : ℚ) • x i := by
        apply Finset.sum_congr rfl
        intro i hi
        rw [← hkq i]
        rfl
      _ = (D : ℝ) • ∑ i, (q i : ℝ) • x i := by
        ext s
        simp only [Finset.sum_apply, Pi.smul_apply, smul_eq_mul]
        rw [Finset.mul_sum]
        apply Finset.sum_congr rfl
        intro i hi
        simp [Algebra.smul_def]
        ring
      _ = 0 := by rw [hqsum, smul_zero]
  exact h.natWeighted n x k hx hN hk hksum

end RationalRepetition

end Scott1964.MeasurementStructures.LinearInequalities
```

B.2.4 Sequences.lean

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/
```

```
import Scott1964.MeasurementStructures.LinearInequalities.Definitions

/-!
# Scott's explicit finite sequence conditions
-/

open scoped BigOperators

namespace Scott1964.MeasurementStructures.LinearInequalities

variable {L : Type*} [AddCommGroup L] [Module ℝ L]

/-- A realizing functional satisfies Scott's positive-weight condition (2). -/
theorem Realizable.weightedSequenceCancellation {X N : Set L}
    (hreal : Realizable X N) (hsym : Symmetric X) :
    WeightedSequenceCancellation X N := by
  obtain ⟨φ, hφ⟩ := hreal
  intro n x c hx hN hc hsum
  have hnonneg : ∀ i, 0 ≤ c i * φ (x i) :=
    fun i ↦ mul_nonneg (hc i).le ((hφ (x i) (hx i)).mp (hN i))
  have hzero : ∑ i, c i * φ (x i) = 0 := by
    have := congrArg φ hsum
    simpa only [map_sum, map_smul, smul_eq_mul, map_zero] using this
  intro j
  have hxzero : φ (x j) = 0 := by
    have hterm : c j * φ (x j) = 0 :=
      (Finset.sum_eq_zero_iff_of_nonneg fun i _ ↦ hnonneg i).mp hzero j (Finset.
    mem_univ j)
    exact (mul_eq_zero.mp hterm).resolve_left (hc j).ne'
  apply (hφ (-x j) (hsym (hx j))).mpr
  simp [hxzero]

/-- Condition (2) immediately implies the unit-weight condition (4). -/
theorem WeightedSequenceCancellation.unweighted {X N : Set L}
    (h : WeightedSequenceCancellation X N) :
    UnweightedSequenceCancellation X N := by
  intro n x hx hN hsum
  apply h n x (fun _ ↦ 1) hx hN (fun _ ↦ zero_lt_one)
  simpa using hsum

/-- A realizing relation satisfies Scott's paired equal-sums condition (6). -/
theorem RelationRealizable.sequenceCancellation {Y : Set L} {R : L → L → Prop}
    (hreal : RelationRealizable Y R) : RelationSequenceCancellation Y R := by
  obtain ⟨φ, hφ⟩ := hreal
  intro n x y hx hy hR hsum
  have hle : ∀ i, φ (y i) ≤ φ (x i) :=
    fun i ↦ (hφ (x i) (hx i) (y i) (hy i)).mp (hR i)
  have hsumφ : ∑ i, φ (x i) = ∑ i, φ (y i) := by
    simpa only [map_sum] using congrArg φ hsum
  intro j
  have hhead : φ (x j) ≤ φ (y j) := by
    have hall :=
      (Finset.sum_eq_sum_iff_of_le (s := Finset.univ)
        (fun i _ ↦ hle i)).mp hsumφ.symm
    exact (hall j (Finset.mem_univ j)).ge
  exact (hφ (y j) (hy j) (x j) (hx j)).mpr hhead
```

```
end Scott1964.MeasurementStructures.LinearInequalities
```

B.2.5 ScottTheorems.lean

Lean 4 source (lines 1–400)

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.LinearInequalities.Separation
import Scott1964.MeasurementStructures.LinearInequalities.Rationalization
import Scott1964.MeasurementStructures.LinearInequalities.Sequences

/-!
# Scott 1964, Theorems 1.1–1.4

Finite convex-separation forms of the general method in §1.
-/

open Set

namespace Scott1964.MeasurementStructures.LinearInequalities

variable {L : Type*} [NormedAddCommGroup L] [NormedSpace ℝ L]
  [FiniteDimensional ℝ L]

/-- The sequence formulation may be used with any nonempty finite index type. -/
theorem WeightedSequenceCancellation.fintype {X N : Set L}
    (h : WeightedSequenceCancellation X N) {I : Type*} [Fintype I] [Nonempty I]
    (x : I → L) (c : I → ℝ) (hx : ∀ i, x i ∈ X)
    (hN : ∀ i, x i ∈ N) (hc : ∀ i, 0 < c i)
    (hsum : ∑ i, c i • x i = 0) : ∀ i, -x i ∈ N := by
  let n := Fintype.card I - 1
  have hcard : Fintype.card I = n + 1 := by
    dsimp [n]
    have := Fintype.card_pos (α := I)
    omega
  let e : Fin (n + 1) ≃ I :=
    Fintype.equivOfCardEq (by simpa using hcard.symm)
  have he := h n (x ∘ e) (c ∘ e) (fun i ↦ hx (e i))
    (fun i ↦ hN (e i)) (fun i ↦ hc (e i))
    (by
      rw [← hsum]
      exact e.sum_comp (fun i ↦ c i • x i))
  intro i
  simpa using he (e.symm i)

/-- Under sign completeness, Scott's positive-weight sequence condition is
equivalent to the geometric cancellation condition. -/
theorem weightedCancellation_of_sequence {X N : Set L}
    (hsym : Symmetric X) (hcomplete : SignComplete X N)
```

```
  (hseq : WeightedSequenceCancellation X N) :
  WeightedCancellation X N := by
rw [WeightedCancellation, Set.disjoint_left]
intro z hzconv hzspan
obtain ⟨ι, instι, w, a, hw, hwone, ha, hwa⟩ :=
  mem_convexHull_iff_exists_fintype.mp hzconv
obtain ⟨d, t, ht, hdsupp, hdt⟩ :=
  Submodule.mem_span_iff_exists_finset_subset.mp hzspan
let I₁ := {i : ι // w i ≠ 0}
let I₂ := {y : ⇑t // d y ≠ 0}
let I := I₁ ⊕ I₂
let v : I → L
  | Sum.inl i => a i
  | Sum.inr y => if 0 < d y.1.1 then -y.1.1 else y.1.1
let c : I → ℝ
  | Sum.inl i => w i
  | Sum.inr y => |d y.1.1|
have hI : Nonempty I := by
  have hwne : ∃ i, w i ≠ 0 := by
    by_contra h
    push_neg at h
    simp [h] at hwone
  exact ⟨Sum.inl ⟨hwne.choose, hwne.choose_spec⟩⟩
letI : Nonempty I := hI
have hvX : ∀ i, v i ∈ X := by
  rintro (i | y)
  · exact (ha i).1
  · rcases ht y.1.2 with ⟨hyX, hyN, hnyN⟩
    dsimp [v]
    split_ifs
    · exact hsym hyX
    · exact hyX
have hvN : ∀ i, v i ∈ N := by
  rintro (i | y)
  · exact (ha i).2.1
  · rcases ht y.1.2 with ⟨hyX, hyN, hnyN⟩
    dsimp [v]
    split_ifs
    · exact hnyN
    · exact hyN
have hc : ∀ i, 0 < c i := by
  rintro (i | y)
  · exact lt_of_le_of_ne (hw i) i.property.symm
  · exact abs_pos.mpr y.property
have hsum : ∑ i, c i • v i = 0 := by
  rw [Fintype.sum_sum_type]
  have hleft : ∑ i : I₁, c (.inl i) • v (.inl i) = z := by
    rw [← hwa]
    rw [show (∑ i : I₁, c (.inl i) • v (.inl i)) =
        (∑ i ∈ Finset.univ.filter (fun i ↦ w i ≠ 0), w i • a i) by
      simpa [I₁, c, v] using
        (Finset.sum_subtype (Finset.univ.filter fun i ↦ w i ≠ 0)
          (by simp) (fun i ↦ w i • a i)).symm]
    exact Finset.sum_filter_of_ne fun i hi hterm hwi ↦
      hterm (by simp [hwi])
  have hright : ∑ y : I₂, c (.inr y) • v (.inr y) = -z := by
```

```
      rw [← hdt]
      have hsub :
          ∑ y : I₂, c (.inr y) • v (.inr y) =
            -∑ y ∈ t, d y • y := by
        rw [← Finset.sum_neg_distrib]
        rw [show (∑ y : I₂, c (.inr y) • v (.inr y)) =
            (∑ y : {y : ⇑t // d y.1 ≠ 0}, -(d y.1.1 • y.1.1)) by
          apply Finset.sum_congr rfl
          intro y hy
          dsimp [I₂, c, v]
          split_ifs with hdy
          · rw [abs_of_pos hdy, smul_neg]
          · have hdyneg : d y.1.1 < 0 := lt_of_le_of_ne (le_of_not_gt hdy) y.2
            rw [abs_of_neg hdyneg, neg_smul]]
        calc
          ∑ y : {y : ⇑t // d y.1 ≠ 0}, -(d y.1.1 • y.1.1) =
              ∑ y ∈ Finset.univ.filter (fun y : ⇑t ↦ d y.1 ≠ 0),
                -(d y.1 • y.1) := by
                  exact (Finset.sum_subtype
                    (Finset.univ.filter fun y : ⇑t ↦ d y.1 ≠ 0)
                    (by simp) (fun y : ⇑t ↦ -(d y.1 • y.1))).symm
          _ = ∑ y : ⇑t, -(d y.1 • y.1) := by
                exact Finset.sum_filter_of_ne fun y hy hterm hdy ↦
                  hterm (by simp [hdy])
          _ = ∑ y ∈ t, -(d y • y) := by
                change (t.attach.sum fun y ↦ -(d y.1 • y.1)) = _
                exact Finset.sum_attach t (fun y ↦ -(d y • y))
      exact hsub
    rw [hleft, hright]
    exact add_neg_cancel z
  have hall := hseq.fintype v c hvX hvN hc hsum
  obtain ⟨i, hi⟩ : ∃ i, w i ≠ 0 := by
    by_contra h
    push_neg at h
    simp [h] at hwone
  have hnot := (ha i).2.2
  exact hnot (by
    simpa [v] using hall (Sum.inl (show I₁ from ⟨i, hi⟩)))

/-- Scott (1964), Theorem 1.1. -/
theorem scott_theorem_1_1 {X N : Set L} (hX : X.Finite) (hsym : Symmetric X) :
    Realizable X N ↔ SignComplete X N ∧ WeightedSequenceCancellation X N := by
  constructor
  · rintro ⟨φ, hφ⟩
    refine ⟨?_, ?_⟩
    · intro x hx
      by_cases h : 0 ≤ φ x
      · exact Or.inl ((hφ x hx).mpr h)
      · right
        have hnx := hsym hx
        apply (hφ (-x) hnx).mpr
        simpa using le_of_not_ge h
    · exact Realizable.weightedSequenceCancellation ⟨φ, hφ⟩ hsym
  · rintro ⟨htotal, hseq⟩
    have hcancel := weightedCancellation_of_sequence hsym htotal hseq
    obtain ⟨φ, hstrict, hneutral⟩ :=
```

```
      (finite_strict_separation
        (hX.subset fun _ hx ↦ hx.1)).mp hcancel
    refine ⟨φ, fun x hx ↦ ?_⟩
    constructor
    · intro hN
      by_cases hnN : -x ∈ N
      · exact le_of_eq (hneutral x ⟨hx, hN, hnN⟩).symm
      · exact (hstrict x ⟨hx, hN, hnN⟩).le
    · intro hφ
      rcases htotal hx with hN | hnN
      · exact hN
      · by_contra hxN
        have hneg := hstrict (-x) ⟨hsym hx, hnN, by simpa using hxN⟩
        simp only [map_neg] at hneg
        linarith

/-- Scott (1964), Theorem 1.2, for rational coordinate vectors.

Rationality is used to replace a positive real dependence by a positive
rational dependence, clear denominators, and reduce to repeated unweighted
summands.
-/
theorem scott_theorem_1_2 {S : Type*} [Fintype S] {X N : Set (S → ℝ)}
    (hX : X.Finite) (hrat : IsRationalSet X) (hsym : Symmetric X) :
    Realizable X N ↔ SignComplete X N ∧ UnweightedSequenceCancellation X N := by
  constructor
  · intro hreal
    have h := (scott_theorem_1_1 hX hsym).mp hreal
    exact ⟨h.1, h.2.unweighted⟩
  · rintro ⟨hcomplete, hunweighted⟩
    apply (scott_theorem_1_1 hX hsym).mpr
    exact ⟨hcomplete, hunweighted.rationalWeighted hrat⟩

/-- Geometric intermediate form used in the proof of Theorem 1.3. -/
theorem scott_theorem_1_3_geometric {Y : Set L} {R : L → L → Prop} (hY : Y.Finite) :
    RelationRealizable Y R ↔ RelationComplete Y R ∧ RelationCancellation Y R := by
  constructor
  · rintro ⟨φ, hφ⟩
    refine ⟨?_, ?_⟩
    · intro x hx y hy
      exact le_total (φ y) (φ x) |>.imp (fun h ↦ (hφ x hx y hy).mpr h)
        (fun h ↦ (hφ y hy x hx).mpr h)
    · have hstrictFinite : (strictDifferences Y R).Finite := by
        apply ((hY.prod hY).image fun p : L × L ↦ p.1 - p.2).subset
        rintro z ⟨x, hx, y, hy, -, -, rfl⟩
        exact ⟨⟨x, y⟩, ⟨hx, hy⟩, rfl⟩
      apply (finite_strict_separation hstrictFinite).mpr
      refine ⟨φ, ?_, ?_⟩
      · rintro z ⟨x, hx, y, hy, hxy, hyx, rfl⟩
        have hle := (hφ x hx y hy).mp hxy
        have hnle : ¬φ x ≤ φ y := by
          intro h
          exact hyx ((hφ y hy x hx).mpr h)
        simp only [map_sub]
        linarith
      · rintro z ⟨x, hx, y, hy, hxy, hyx, rfl⟩
```

```
        have h₁ := (hφ x hx y hy).mp hxy
        have h₂ := (hφ y hy x hx).mp hyx
        simp only [map_sub]
        linarith
  · rintro ⟨hcomplete, hcancel⟩
    have hstrictFinite : (strictDifferences Y R).Finite := by
      apply ((hY.prod hY).image fun p : L × L ↦ p.1 - p.2).subset
      rintro z ⟨x, hx, y, hy, -, -, rfl⟩
      exact ⟨⟨x, y⟩, ⟨hx, hy⟩, rfl⟩
    obtain ⟨φ, hstrict, hneutral⟩ :=
      (finite_strict_separation hstrictFinite).mp hcancel
    refine ⟨φ, fun x hx y hy ↦ ?_⟩
    constructor
    · intro hxy
      by_cases hyx : R y x
      · have hz := hneutral (x - y) ⟨x, hx, y, hy, hxy, hyx, rfl⟩
        have heq : φ x = φ y := by
          simp only [map_sub] at hz
          linarith
        exact heq.ge
      · have hz := hstrict (x - y) ⟨x, hx, y, hy, hxy, hyx, rfl⟩
        simpa [map_sub, sub_pos] using hz.le
    · intro hle
      rcases hcomplete hx hy with hxy | hyx
      · exact hxy
      · by_contra hnxy
        have hz := hstrict (y - x) ⟨y, hy, x, hx, hyx, hnxy, rfl⟩
        simp only [map_sub, sub_pos] at hz
        linarith

/-- Scott (1964), Theorem 1.3, with the literal paired equal-sums
cancellation condition (6). -/
theorem scott_theorem_1_3 {S : Type*} [Fintype S]
    {Y : Set (S → ℝ)} {R : (S → ℝ) → (S → ℝ) → Prop}
    (hY : Y.Finite) (hYrat : IsRationalSet Y) :
    RelationRealizable Y R ↔
      RelationComplete Y R ∧ RelationSequenceCancellation Y R := by
  constructor
  · intro hreal
    obtain ⟨φ, hφ⟩ := hreal
    refine ⟨?_, RelationRealizable.sequenceCancellation ⟨φ, hφ⟩⟩
    intro x hx y hy
    exact le_total (φ y) (φ x) |>.imp
      (fun h ↦ (hφ x hx y hy).mpr h) (fun h ↦ (hφ y hy x hx).mpr h)
  · rintro ⟨hcomplete, hcancel⟩
    let X : Set (S → ℝ) := {z | ∃ x ∈ Y, ∃ y ∈ Y, z = x - y}
    let N : Set (S → ℝ) := {z | ∃ x ∈ Y, ∃ y ∈ Y, R x y ∧ z = x - y}
    have hXfinite : X.Finite := by
      apply ((hY.prod hY).image fun p : (S → ℝ) × (S → ℝ) ↦ p.1 - p.2).subset
      rintro z ⟨x, hx, y, hy, rfl⟩
      exact ⟨(x, y), ⟨hx, hy⟩, rfl⟩
    have hXrat : IsRationalSet X := rational_difference_set hYrat
    have hXsym : Symmetric X := by
      rintro z ⟨x, hx, y, hy, rfl⟩
      exact ⟨y, hy, x, hx, by simp [sub_eq_add_neg, add_comm]⟩
    have hsign : SignComplete X N := by
```

```
      rintro z ⟨x, hx, y, hy, rfl⟩
      rcases hcomplete hx hy with hxy | hyx
      · left
        exact ⟨x, hx, y, hy, hxy, rfl⟩
      · right
        exact ⟨y, hy, x, hx, hyx, by simp [sub_eq_add_neg, add_comm]⟩
    have hunweighted : UnweightedSequenceCancellation X N := by
      intro n z hzX hzN hsum
      choose x hx y hy hR hxy using hzN
      have hsumeq : ∑ i, x i = ∑ i, y i := by
        have hzero : ∑ i, x i - ∑ i, y i = 0 := by
          rw [← Finset.sum_sub_distrib]
          simpa only [hxy] using hsum
        exact sub_eq_zero.mp hzero
      have hrev := hcancel n x y hx hy hR hsumeq
      intro i
      exact ⟨y i, hy i, x i, hx i, hrev i, by
        rw [hxy i]
        simp [sub_eq_add_neg, add_comm]⟩
    obtain ⟨φ, hφ⟩ :=
      (scott_theorem_1_2 hXfinite hXrat hXsym).mpr ⟨hsign, hunweighted⟩
    refine ⟨φ, fun x hx y hy ↦ ?_⟩
    constructor
    · intro hxy
      have hN : x - y ∈ N := ⟨x, hx, y, hy, hxy, rfl⟩
      have hnonneg := (hφ (x - y) ⟨x, hx, y, hy, rfl⟩).mp hN
      simpa [map_sub, sub_nonneg] using hnonneg
    · intro hle
      have hnonneg : 0 ≤ φ (x - y) := by simpa [map_sub, sub_nonneg] using hle
      obtain ⟨x', hx', y', hy', hx'y', heq⟩ :=
        (hφ (x - y) ⟨x, hx, y, hy, rfl⟩).mpr hnonneg
      by_contra hnxy
      have hyx := (hcomplete hx hy).resolve_left hnxy
      let xs : Fin 2 → S → ℝ := ![x', y]
      let ys : Fin 2 → S → ℝ := ![y', x]
      have hsum2 : ∑ i, xs i = ∑ i, ys i := by
        rw [Fin.sum_univ_two, Fin.sum_univ_two]
        dsimp [xs, ys]
        ext s
        have hs := congrFun heq s
        dsimp at hs ⊢
        linarith
      have hback := hcancel 1 xs ys (by simp [xs, hx', hy])
        (by simp [ys, hy', hx]) (by
          intro i
          fin_cases i <;> simp [xs, ys, hx'y', hyx]) hsum2
      exact hnxy (by simpa [xs, ys] using hback (1 : Fin 2))

/-- A realizing functional supplies Scott's strictly monotonic extension to
the additive closure. This is the forward half of Theorem 1.4. -/
theorem scott_theorem_1_4_forward {Y : Set L} {R : L → L → Prop}
    (hreal : RelationRealizable Y R) :
    ∃ Rplus : L → L → Prop,
      ExtendsOn Y (additiveClosure Y) R Rplus ∧
        StrictlyMonotonic (additiveClosure Y) Rplus := by
  obtain ⟨φ, hφ⟩ := hreal
```

```
  let Rplus : L → L → Prop := fun x y ↦ φ y ≤ φ x
  refine ⟨Rplus, ⟨AddSubmonoid.subset_closure, ?_⟩, ?_⟩
  · intro x hx y hy
    exact (hφ x hx y hy).symm
  · refine ⟨?_, ?_, ?_⟩
    · intro x hx y hy
      exact le_total (φ y) (φ x)
    · intro x₀ y₀ x₁ y₁ hx₀ hy₀ hx₁ hy₁ h₀ h₁
      dsimp [Rplus] at h₀ h₁ ⊢
      simp only [map_add]
      linarith
    · intro x₀ y₀ x₁ y₁ hx₀ hy₀ hx₁ hy₁ heq h₁
      dsimp [Rplus] at h₁ ⊢
      have hsum : φ x₀ + φ x₁ = φ y₀ + φ y₁ := by
        simpa only [map_add] using congrArg φ heq
      linarith

/-- A strictly monotonic extension implies Scott's literal condition (6). -/
theorem StrictlyMonotonic.relationSequenceCancellation
    {Y : Set L} {R Rplus : L → L → Prop}
    (hext : ExtendsOn Y (additiveClosure Y) R Rplus)
    (hmono : StrictlyMonotonic (additiveClosure Y) Rplus) :
    RelationSequenceCancellation Y R := by
  intro n x y hx hy hR hsum
  have hsumRel :
      ∀ s : Finset (Fin (n + 1)),
        Rplus (∑ i ∈ s, x i) (∑ i ∈ s, y i) := by
    intro s
    classical
    induction s using Finset.induction_on with
    | empty =>
        simpa using (hmono.complete
          (show (0 : L) ∈ additiveClosure Y from AddSubmonoid.zero_mem _)
          (show (0 : L) ∈ additiveClosure Y from AddSubmonoid.zero_mem _)).elim id id
    | @insert i s hi ih =>
        rw [Finset.sum_insert hi, Finset.sum_insert hi]
        apply hmono.add (hext.1 (hx i)) (hext.1 (hy i))
          (AddSubmonoid.sum_mem _ fun j hj ↦ hext.1 (hx j))
          (AddSubmonoid.sum_mem _ fun j hj ↦ hext.1 (hy j))
          ((hext.2 (hx i) (hy i)).mpr (hR i)) ih
  intro j
  let t : Finset (Fin (n + 1)) := Finset.univ.erase j
  have htail := hsumRel t
  apply (hext.2 (hy j) (hx j)).mp
  apply hmono.cancel (hext.1 (hx j)) (hext.1 (hy j))
    (AddSubmonoid.sum_mem _ fun i hi ↦ hext.1 (hx i))
    (AddSubmonoid.sum_mem _ fun i hi ↦ hext.1 (hy i))
    ?_ htail
  have hxsplit : x j + ∑ i ∈ t, x i = ∑ i, x i := by
    rw [add_comm, ← Finset.sum_erase_add _ _ (Finset.mem_univ j)]
  have hysplit : y j + ∑ i ∈ t, y i = ∑ i, y i := by
    rw [add_comm, ← Finset.sum_erase_add _ _ (Finset.mem_univ j)]
  rw [hxsplit, hysplit, hsum]

/-- Scott (1964), Theorem 1.4: realizability is equivalent to extendability
to a strictly monotonic relation on the additive closure. -/
```

```
theorem scott_theorem_1_4 {S : Type*} [Fintype S]
    {Y : Set (S → ℝ)} {R : (S → ℝ) → (S → ℝ) → Prop}
    (hY : Y.Finite) (hYrat : IsRationalSet Y) :
    RelationRealizable Y R ↔
      ∃ Rplus : (S → ℝ) → (S → ℝ) → Prop,
        ExtendsOn Y (additiveClosure Y) R Rplus ∧
        StrictlyMonotonic (additiveClosure Y) Rplus := by
  constructor
  · intro h
    exact scott_theorem_1_4_forward h
  · rintro ⟨Rplus, hext, hmono⟩
    apply (scott_theorem_1_3 hY hYrat).mpr
    refine ⟨?_, hmono.relationSequenceCancellation hext⟩
    intro x hx y hy
    exact (hmono.complete (hext.1 hx) (hext.1 hy)).imp
      (fun h ↦ hext.2 hx hy |>.mp h) (fun h ↦ hext.2 hy hx |>.mp h)
```

Lean 4 source (lines 401–402)

```
end Scott1964.MeasurementStructures.LinearInequalities
```

B.2.6 OrderedGroup.lean

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.LinearInequalities.ScottTheorems
import Mathlib.Algebra.Order.Monoid.Lex

/-!
# Ordered groups after Scott's Theorem 1.4

The first result makes precise Scott's observation that every finite part of
a linearly ordered abelian group has a real-valued model.  Pair-sums are put
into the finite system, so the model preserves every addition equation visible
inside that finite part.

The second group of results records the standard lexicographic obstruction:
each finite subsystem is real-valued, but the whole ordered monoid is not.
-/

open Set

namespace Scott1964.MeasurementStructures.LinearInequalities

section Local

variable {G : Type*} [AddCommGroup G] [LinearOrder G] [IsOrderedAddMonoid G]
```

```
private def localBasis {S : Set G} (x : S) : S → ℝ :=
  Pi.single x 1

private def localPair {S : Set G} (p : S × S) : S → ℝ :=
  localBasis p.1 + localBasis p.2

private def localTests (S : Set G) : Set (S → ℝ) :=
  Set.range localBasis ∪ Set.range localPair

private def natCoordinates {S : Set G} (n : S → ℕ) : S → ℝ :=
  fun i ↦ n i

private def coordinateValue {S : Set G} [Fintype S] (n : S → ℕ) : G :=
  ∑ i, n i • (i : G)

private theorem natCoordinates_injective {S : Set G} :
    Function.Injective (natCoordinates (S := S)) := by
  intro m n h
  funext i
  exact Nat.cast_injective (congrFun h i)

private theorem localBasis_coordinates {S : Set G} [Fintype S] (x : S) :
    localBasis x = natCoordinates (Pi.single x 1) := by
  funext i
  classical
  by_cases h : i = x
  · subst i
    simp [localBasis, natCoordinates]
  · simp [localBasis, natCoordinates, Pi.single_apply, h]

private theorem localPair_coordinates {S : Set G} [Fintype S] (p : S × S) :
    localPair p =
      natCoordinates (Pi.single p.1 1 + Pi.single p.2 1) := by
  rw [localPair, localBasis_coordinates, localBasis_coordinates]
  funext i
  simp [natCoordinates]

private theorem coordinateValue_single {S : Set G} [Fintype S] (x : S) :
    coordinateValue (Pi.single x 1) = x := by
  classical
  rw [coordinateValue, Finset.sum_eq_single x]
  · simp
  · intro b _ hbx
    simp [Pi.single_apply, hbx]
  · simp

private theorem coordinateValue_add {S : Set G} [Fintype S] (m n : S → ℕ) :
    coordinateValue (m + n) = coordinateValue m + coordinateValue n := by
  classical
  simp only [coordinateValue, Pi.add_apply, add_nsmul, Finset.sum_add_distrib]

private theorem natCoordinates_add {S : Set G} (m n : S → ℕ) :
    natCoordinates m + natCoordinates n = natCoordinates (m + n) := by
  funext i
  simp [natCoordinates]
```

```
private def coordinateRelation {S : Set G} [Fintype S]
    (v w : S → ℝ) : Prop :=
  ∃ m n : S → ℕ,
    natCoordinates m = v ∧ natCoordinates n = w ∧
      coordinateValue n ≤ coordinateValue m

private theorem coordinateRelation_encode_iff {S : Set G} [Fintype S]
    (m n : S → ℕ) :
    coordinateRelation (natCoordinates m) (natCoordinates n) ↔
      coordinateValue n ≤ coordinateValue m := by
  constructor
  · rintro ⟨m', n', hm, hn, hle⟩
    have hmm : m' = m := natCoordinates_injective hm
    have hnn : n' = n := natCoordinates_injective hn
    simpa [hmm, hnn] using hle
  · intro hle
    exact ⟨m, n, rfl, rfl, hle⟩

private theorem localTests_finite {S : Set G} [Fintype S] :
    (localTests S).Finite :=
  (Set.finite_range localBasis).union (Set.finite_range localPair)

private theorem localTests_rational {S : Set G} [Fintype S] :
    IsRationalSet (localTests S) := by
  rintro v (⟨x, rfl⟩ | ⟨p, rfl⟩) i
  · classical
    by_cases h : i = x
    · subst i
      exact ⟨1, by simp [localBasis]⟩
    · exact ⟨0, by simp [localBasis, Pi.single_apply, h]⟩
  · classical
    by_cases h₁ : i = p.1 <;> by_cases h₂ : i = p.2
    · subst i
      have : p.1 = p.2 := h₂
      exact ⟨2, by simp [localPair, localBasis, this]; norm_num⟩
    · subst i
      exact ⟨1, by simp [localPair, localBasis, Pi.single_apply, h₂]⟩
    · subst i
      exact ⟨1, by simp [localPair, localBasis, Pi.single_apply, h₁]⟩
    · exact ⟨0, by simp [localPair, localBasis, Pi.single_apply, h₁, h₂]⟩

private theorem localTests_closure_coordinates {S : Set G} [Fintype S]
    {v : S → ℝ} (hv : v ∈ additiveClosure (localTests S)) :
    ∃ n : S → ℕ, natCoordinates n = v := by
  induction hv using AddSubmonoid.closure_induction with
  | mem v hv =>
      rcases hv with ⟨x, rfl⟩ | ⟨p, rfl⟩
      · exact ⟨Pi.single x 1, localBasis_coordinates x |>.symm⟩
      · exact ⟨Pi.single p.1 1 + Pi.single p.2 1,
          localPair_coordinates p |>.symm⟩
  | zero => exact ⟨fun _ ↦ 0, by funext i; norm_num [natCoordinates]⟩
  | add v w hv hw ihv ihw =>
      obtain ⟨m, rfl⟩ := ihv
      obtain ⟨n, rfl⟩ := ihw
      exact ⟨m + n, natCoordinates_add m n |>.symm⟩
```

```
private theorem coordinateRelation_strictlyMonotonic {S : Set G} [Fintype S] :
    StrictlyMonotonic (additiveClosure (localTests S))
      (coordinateRelation (S := S)) := by
  refine ⟨?_, ?_, ?_⟩
  · intro v hv w hw
    obtain ⟨m, rfl⟩ := localTests_closure_coordinates hv
    obtain ⟨n, rfl⟩ := localTests_closure_coordinates hw
    exact (le_total (coordinateValue n) (coordinateValue m)).imp
      (coordinateRelation_encode_iff m n |>.mpr)
      (coordinateRelation_encode_iff n m |>.mpr)
  · intro v₀ w₀ v₁ w₁ hv₀ hw₀ hv₁ hw₁ h₀ h₁
    obtain ⟨m₀, n₀, rfl, rfl, h₀⟩ := h₀
    obtain ⟨m₁, n₁, hm₁, hn₁, h₁⟩ := h₁
    subst v₁
    subst w₁
    rw [natCoordinates_add m₀ m₁, natCoordinates_add n₀ n₁,
      coordinateRelation_encode_iff, coordinateValue_add, coordinateValue_add]
    exact add_le_add h₀ h₁
  · intro v₀ w₀ v₁ w₁ hv₀ hw₀ hv₁ hw₁ heq h₁
    obtain ⟨m₀, rfl⟩ := localTests_closure_coordinates hv₀
    obtain ⟨n₀, rfl⟩ := localTests_closure_coordinates hw₀
    obtain ⟨m₁, rfl⟩ := localTests_closure_coordinates hv₁
    obtain ⟨n₁, rfl⟩ := localTests_closure_coordinates hw₁
    have hmn : m₀ + m₁ = n₀ + n₁ := by
      apply natCoordinates_injective
      rw [← natCoordinates_add, ← natCoordinates_add]
      exact heq
    have hvalue :
        coordinateValue m₀ + coordinateValue m₁ =
          coordinateValue n₀ + coordinateValue n₁ := by
      rw [← coordinateValue_add, ← coordinateValue_add, hmn]
    have htail : coordinateValue n₁ ≤ coordinateValue m₁ :=
      (coordinateRelation_encode_iff m₁ n₁).mp h₁
    apply (coordinateRelation_encode_iff n₀ m₀).mpr
    apply (add_le_add_iff_right (coordinateValue m₁)).mp
    rw [hvalue]
    simpa [add_comm] using add_le_add_left htail (coordinateValue n₀)

/-- Every finite subset of a linearly ordered abelian group admits a local
real embedding. It reflects the ambient order and preserves every equation
`x + y = z` all three of whose terms belong to the finite set. -/
theorem finite_local_real_embedding (S : Set G) (hS : S.Finite) :
    ∃ f : S → ℝ,
      (∀ x y : S, x ≤ y ↔ f x ≤ f y) ∧
      (∀ x y z : S, (x : G) + y = z → f x + f y = f z) := by
  letI : Fintype S := hS.fintype
  have hext :
      ∃ Rplus : (S → ℝ) → (S → ℝ) → Prop,
        ExtendsOn (localTests S) (additiveClosure (localTests S))
          (coordinateRelation (S := S)) Rplus ∧
        StrictlyMonotonic (additiveClosure (localTests S)) Rplus :=
    ⟨coordinateRelation, ⟨AddSubmonoid.subset_closure, fun _ _ _ _ ↦ Iff.rfl⟩,
      coordinateRelation_strictlyMonotonic⟩
  obtain ⟨φ, hφ⟩ :=
    (scott_theorem_1_4 (localTests_finite (S := S))
      (localTests_rational (S := S))).mpr hext
```

```
  refine ⟨fun x ↦ φ (localBasis x), ?_, ?_⟩
  · intro x y
    have hx : localBasis x ∈ localTests S := Or.inl ⟨x, rfl⟩
    have hy : localBasis y ∈ localTests S := Or.inl ⟨y, rfl⟩
    rw [← hφ (localBasis y) hy (localBasis x) hx]
    rw [localBasis_coordinates, localBasis_coordinates,
      coordinateRelation_encode_iff, coordinateValue_single,
      coordinateValue_single]
    rfl
  · intro x y z hxy
    have hp : localPair (x, y) ∈ localTests S := Or.inr ⟨(x, y), rfl⟩
    have hz : localBasis z ∈ localTests S := Or.inl ⟨z, rfl⟩
    have hpz : coordinateRelation (localPair (x, y)) (localBasis z) := by
      rw [localPair_coordinates, localBasis_coordinates,
        coordinateRelation_encode_iff, coordinateValue_add,
        coordinateValue_single, coordinateValue_single, coordinateValue_single,
        hxy]
    have hzp : coordinateRelation (localBasis z) (localPair (x, y)) := by
      rw [localPair_coordinates, localBasis_coordinates,
        coordinateRelation_encode_iff, coordinateValue_add,
        coordinateValue_single, coordinateValue_single, coordinateValue_single,
        hxy]
    have hle := (hφ (localPair (x, y)) hp (localBasis z) hz).mp hpz
    have hge := (hφ (localBasis z) hz (localPair (x, y)) hp).mp hzp
    simpa [localPair, map_add] using le_antisymm hge hle

end Local

section Lexicographic

abbrev LexIntPair := ℤ ×₁ ℤ

def lexGenerators : Set LexIntPair :=
  {toLex (1, 0), toLex (0, 1)}

def lexPreference (x y : LexIntPair) : Prop :=
  (ofLex y).1 < (ofLex x).1 ∨
    (ofLex y).1 = (ofLex x).1 ∧ (ofLex y).2 ≤ (ofLex x).2

theorem lexPreference_strictlyMonotonic :
    StrictlyMonotonic (additiveClosure lexGenerators) lexPreference := by
  refine ⟨?_, ?_, ?_⟩
  · intro x hx y hy
    rcases x with ⟨x₁, x₂⟩
    rcases y with ⟨y₁, y₂⟩
    simp only [lexPreference, ofLex]
    omega
  · intro x₀ y₀ x₁ y₁ _ _ _ _ h₀ h₁
    rcases x₀ with ⟨x₀₁, x₀₂⟩
    rcases y₀ with ⟨y₀₁, y₀₂⟩
    rcases x₁ with ⟨x₁₁, x₁₂⟩
    rcases y₁ with ⟨y₁₁, y₁₂⟩
    change y₀₁ < x₀₁ ∨ y₀₁ = x₀₁ ∧ y₀₂ ≤ x₀₂ at h₀
    change y₁₁ < x₁₁ ∨ y₁₁ = x₁₁ ∧ y₁₂ ≤ x₁₂ at h₁
    change y₀₁ + y₁₁ < x₀₁ + x₁₁ ∨
      y₀₁ + y₁₁ = x₀₁ + x₁₁ ∧ y₀₂ + y₁₂ ≤ x₀₂ + x₁₂
```

```
    omega
  · intro x₀ y₀ x₁ y₁ _ _ _ _ heq h₁
    rcases x₀ with ⟨x₀₁, x₀₂⟩
    rcases y₀ with ⟨y₀₁, y₀₂⟩
    rcases x₁ with ⟨x₁₁, x₁₂⟩
    rcases y₁ with ⟨y₁₁, y₁₂⟩
    change y₁₁ < x₁₁ ∨ y₁₁ = x₁₁ ∧ y₁₂ ≤ x₁₂ at h₁
    change x₀₁ < y₀₁ ∨ x₀₁ = y₀₁ ∧ x₀₂ ≤ y₀₂
    change (x₀₁ + x₁₁, x₀₂ + x₁₂) =
      (y₀₁ + y₁₁, y₀₂ + y₁₂) at heq
    have hfirst := congrArg Prod.fst heq
    have hsecond := congrArg Prod.snd heq
    simp only at hfirst hsecond
    omega

/-- No additive real-valued functional realizes lexicographic comparison on
the whole lexicographically ordered group `ℤ × ℤ`. The obstruction already
uses the positive additive closure of the two nonzero generators: `(0,1) > 0`,
while every natural multiple `(0,n)` remains below `(1,0)`. -/
theorem no_global_real_additive_lex_realization :
    ¬ ∃ f : LexIntPair →+ ℝ,
      ∀ x y, lexPreference x y ↔ f y ≤ f x := by
  rintro ⟨f, hf⟩
  let e₁ : LexIntPair := toLex (1, 0)
  let e₂ : LexIntPair := toLex (0, 1)
  have he₂pos : 0 < f e₂ := by
    have hstrict : lexPreference e₂ 0 ∧ ¬lexPreference 0 e₂ := by
      constructor
      · right
        norm_num [lexPreference, e₂]
      · intro h
        rcases h with h | h <;> norm_num [lexPreference, e₂] at h
    have hnonneg := (hf e₂ 0).mp hstrict.1
    have hne : f e₂ ≠ 0 := by
      intro hzero
      apply hstrict.2
      apply (hf 0 e₂).mpr
      simp [hzero]
    have hnonneg' : 0 ≤ f e₂ := by simpa using hnonneg
    exact lt_of_le_of_ne hnonneg' (Ne.symm hne)
  obtain ⟨n, hn⟩ := exists_nat_gt (f e₁ / f e₂)
  have hlex : lexPreference e₁ (n • e₂) := by
    left
    change (0 : ℤ) < 1
    norm_num
  have hbound := (hf e₁ (n • e₂)).mp hlex
  simp only [map_nsmul] at hbound
  have hlarge : f e₁ < n * f e₂ := by
    have := (mul_lt_mul_of_pos_right hn he₂pos)
    field_simp at this
    simpa [mul_comm] using this
  exact (not_lt_of_ge hbound) (by simpa [nsmul_eq_mul] using hlarge)

end Lexicographic

end Scott1964.MeasurementStructures.LinearInequalities
```

B.3 Preference

B.3.1 Direct.lean

Lean 4 source

```
import Mathlib
import Mathlib.Order.Antisymmetrization

/-!
# Direct finite semiorder representation

The order used here is the one introduced by Scott and Suppes (1958):
`x ≼ y` says that every predecessor of `x` is a predecessor of `y`, and
every successor of `y` is a successor of `x`.  Its antisymmetrization is
the paper's quotient by substitutability.
-/

namespace Scott1964.MeasurementStructures

section WeakOrder

variable {A : Type u} (P : A → A → Prop)

/-- The Scott--Suppes weak order associated with a strict preference. -/
def ScottWeakOrder (x y : A) : Prop :=
  (∀ z, P z x → P z y) ∧ (∀ z, P y z → P x z)

/-- Substitute alternatives have identical strict-preference profiles. -/
def PreferenceSubstitute (x y : A) : Prop :=
  (∀ z, P z x ↔ P z y) ∧ (∀ z, P x z ↔ P y z)

theorem scottWeakOrder_refl (x : A) : ScottWeakOrder P x x :=
  ⟨fun _ h ↦ h, fun _ h ↦ h⟩

theorem scottWeakOrder_trans :
    IsTrans A (ScottWeakOrder P) := by
  constructor
  intro x y z hxy hyz
  exact ⟨fun w hwx ↦ hyz.1 w (hxy.1 w hwx),
    fun w hzw ↦ hxy.2 w (hyz.2 w hzw)⟩

theorem scottWeakOrder_total
    (hirr : ∀ x, ¬P x x)
    (hA : ∀ x y z w, P x y → P z w → P x w ∨ P z y)
    (hB : ∀ x y z w, P x y → P z x → P w y ∨ P z w) :
    ∀ x y, ScottWeakOrder P x y ∨ ScottWeakOrder P y x := by
  intro x y
  classical
  by_cases hxy : ScottWeakOrder P x y
  · exact Or.inl hxy
  by_cases hyx : ScottWeakOrder P y x
  · exact Or.inr hyx
  exfalso
  unfold ScottWeakOrder at hxy hyx
  rw [not_and_or] at hxy hyx
```

```
  rcases hxy with hxy | hxy
  · push_neg at hxy
    rcases hxy with ⟨z, hzx, hzyny⟩
    rcases hyx with hyx | hyx
    · push_neg at hyx
      rcases hyx with ⟨w, hwy, hwnx⟩
      rcases hA z x w y hzx hwy with hzy | hwx
      · exact hzyny hzy
      · exact hwnx hwx
    · push_neg at hyx
      rcases hyx with ⟨w, hxw, hnyw⟩
      rcases hB x w z y hxw hzx with hyw | hzy
      · exact hnyw hyw
      · exact hzyny hzy
  · push_neg at hxy
    rcases hxy with ⟨z, hyz, hnxz⟩
    rcases hyx with hyx | hyx
    · push_neg at hyx
      rcases hyx with ⟨w, hwy, hwnx⟩
      rcases hB y z w x hyz hwy with hxz | hwx
      · exact hnxz hxz
      · exact hwnx hwx
    · push_neg at hyx
      rcases hyx with ⟨w, hxw, hnyw⟩
      rcases hA y z x w hyz hxw with hyw | hxz
      · exact hnyw hyw
      · exact hnxz hxz

theorem preferenceSubstitute_iff :
    PreferenceSubstitute P x y ↔
      ScottWeakOrder P x y ∧ ScottWeakOrder P y x := by
  constructor
  · rintro ⟨hin, hout⟩
    exact ⟨⟨fun z ↦ (hin z).mp, fun z ↦ (hout z).mpr⟩,
      ⟨fun z ↦ (hin z).mpr, fun z ↦ (hout z).mp⟩⟩
  · rintro ⟨hxy, hyx⟩
    exact ⟨fun z ↦ ⟨hxy.1 z, hyx.1 z⟩,
      fun z ↦ ⟨hyx.2 z, hxy.2 z⟩⟩

theorem preference_implies_scottWeakOrder
    (hirr : ∀ x, ¬P x x)
    (hA : ∀ x y z w, P x y → P z w → P x w ∨ P z y)
    {x y : A} (hxy : P x y) :
    ScottWeakOrder P x y := by
  have htrans : IsTrans A P := by
    constructor
    intro a b c hab hbc
    rcases hA a b b c hab hbc with hac | hbb
    · exact hac
    · exact (hirr b hbb).elim
  exact ⟨fun z hzx ↦ htrans.trans z x y hzx hxy, fun z hyz ↦ htrans.trans x y z hxy
    hyz⟩

theorem scottWeakOrder_preference_scottWeakOrder
    {x x₁ y₁ y : A} (hxx₁ : ScottWeakOrder P x x₁)
    (hx₁y₁ : P x₁ y₁) (hy₁y : ScottWeakOrder P y₁ y) : P x y :=
```

```
  hy₁y.1 x (hxx₁.2 y₁ hx₁y₁)

end WeakOrder

section FiniteStaircase

/-- A finite relation whose lower sections are nested along a linear order
has a unit-threshold representation.  This is the finite recursive core of
the Scott--Suppes construction. -/
theorem finite_staircase_representation
    (A : Type u) [Fintype A] [LinearOrder A] (P : A → A → Prop)
    (hforward : ∀ {x y}, P x y → y < x)
    (hmono : ∀ {x x' y y'}, x ≤ x' → y' ≤ y → P x y → P x' y') :
    ∃ f : A → ℝ,
      (∀ x y, P x y ↔ f x ≥ f y + 1) ∧
      (∀ {x y}, x < y → f x < f y) := by
  classical
  induction hcard : Fintype.card A using Nat.strong_induction_on generalizing A with
  | h n ih =>
    by_cases hA : IsEmpty A
    · letI := hA
      exact ⟨fun x ↦ isEmptyElim x, by simp, by simp⟩
    · letI : Nonempty A := not_isEmpty_iff.mp hA
      let m : A := Finset.univ.max' Finset.univ_nonempty
      let B := {x : A // x ≠ m}
      by_cases hB : IsEmpty B
      · refine ⟨fun _ ↦ 0, ?_, ?_⟩
        · intro x y
          have hx : x = m := by
            by_contra hx
            exact hB.false ⟨x, hx⟩
          have hy : y = m := by
            by_contra hy
            exact hB.false ⟨y, hy⟩
          subst x
          subst y
          simp only [ge_iff_le]
          constructor
          · intro hmm
            exact (hforward hmm).false.elim
          · norm_num
        · intro x y hxy
          exact (hxy.ne (by
            have hx : x = m := by
              by_contra hx
              exact hB.false ⟨x, hx⟩
            have hy : y = m := by
              by_contra hy
              exact hB.false ⟨y, hy⟩
            simpa [hx, hy])).elim
      · letI : Nonempty B := not_isEmpty_iff.mp hB
        let PB : B → B → Prop := fun x y ↦ P x.1 y.1
        have hcardB : Fintype.card B < n := by
          have hc : Fintype.card B = Fintype.card A - 1 := by
            simp [B]
          have hpos : 0 < Fintype.card A := Fintype.card_pos
```

```
  rw [← hcard]
  omega
obtain ⟨g, hg, gmono⟩ :=
  ih (Fintype.card B) hcardB B PB
    (fun h ↦ hforward h)
    (fun hxx' hyy' h ↦ hmono hxx' hyy' h) rfl
let nonpref : Finset B := Finset.univ.filter fun x ↦ ¬P m x.1
let score : B → ℝ := fun x ↦ if P m x.1 then g x + 1 else g x
obtain ⟨b, -, hb⟩ :=
  Finset.exists_max_image (Finset.univ : Finset B) score Finset.univ_nonempty
let top : ℝ :=
  if hn : nonpref.Nonempty then
    let j := nonpref.min' hn
    (score b + (g j + 1)) / 2
  else score b + 1
have hscore_lt {j : B} (hj : ¬P m j.1) :
    score b < g j + 1 := by
  apply lt_of_le_of_lt (hb b (Finset.mem_univ b))
  dsimp [score]
  split_ifs with hpb
  · have hbj : b < j := by
      apply lt_of_not_ge
      intro hjb
      exact hj (hmono le_rfl hjb hpb)
    linarith [gmono hbj]
  · have hnot : ¬PB b j := by
      intro hbj
      have hbm : b.1 ≤ m :=
        Finset.le_max' Finset.univ b.1 (Finset.mem_univ b.1)
      exact hj (hmono hbm le_rfl hbj)
    have hlt := not_le.mp (mt (hg b j).mpr hnot)
    linarith
have htop_score : score b < top := by
  dsimp [top]
  split_ifs with hn
  · let j := nonpref.min' hn
    have hj : ¬P m j.1 := by
      have hjmem := nonpref.min'_mem hn
      simpa [nonpref, j] using hjmem
    have hs := hscore_lt hj
    dsimp [j] at hs ⊢
    linarith
  · linarith
have htop_nonpref {j : B} (hj : ¬P m j.1) :
    top < g j + 1 := by
  dsimp [top]
  split_ifs with hn
  · let k := nonpref.min' hn
    have hk_mem : k ∈ nonpref := nonpref.min'_mem hn
    have hkj : k ≤ j := nonpref.min'_le _ (by simp [nonpref, hj])
    have hgkj : g k ≤ g j := by
      rcases hkj.eq_or_lt with rfl | hlt
      · exact le_rfl
      · exact (gmono hlt).le
    have hk : ¬P m k.1 := by simpa [nonpref] using hk_mem
    have hs := hscore_lt hk
```

```
      dsimp [k] at hs hgkj ⊢
      linarith
  · exact (hn ⟨j, by simp [nonpref, hj]⟩).elim
let f : A → ℝ := fun x ↦ if hx : x = m then top else g ⟨x, hx⟩
refine ⟨f, ?_, ?_⟩
· intro x y
  by_cases hx : x = m
  · subst x
    by_cases hy : y = m
    · subst y
      rw [show f m = top by simp [f]]
      exact ⟨fun hmm ↦ (hforward hmm).false.elim, by simp⟩
    · simp only [f, dif_pos rfl, dif_neg hy, ge_iff_le]
      constructor
      · intro hmy
        have hsc : g ⟨y, hy⟩ + 1 = score ⟨y, hy⟩ := by
          simp [score, hmy]
        rw [hsc]
        exact (hb _ (Finset.mem_univ _)).trans htop_score.le
      · intro hle
        by_contra hmy
        have hlt := htop_nonpref (j := ⟨y, hy⟩) hmy
        linarith
  · by_cases hy : y = m
    · subst y
      simp only [f, dif_neg hx, dif_pos rfl]
      exact ⟨fun h ↦ ((not_lt_of_ge (Finset.le_max' Finset.univ x
        (Finset.mem_univ x))) (hforward h)).elim, by
        intro hbad
        have hgs : g ⟨x, hx⟩ ≤ score ⟨x, hx⟩ := by
          dsimp [score]
          split_ifs <;> linarith
        have hlt :=
          (hgs.trans (hb _ (Finset.mem_univ _))).trans_lt htop_score
        linarith⟩
    · simpa [f, hx, hy, PB] using hg ⟨x, hx⟩ ⟨y, hy⟩
· intro x y hxy
  by_cases hy : y = m
  · subst y
    have hx : x ≠ m := hxy.ne
    simp only [f, dif_neg hx, dif_pos rfl]
    have hgs : g ⟨x, hx⟩ ≤ score ⟨x, hx⟩ := by
      dsimp [score]
      split_ifs <;> linarith
    exact (hgs.trans (hb _ (Finset.mem_univ _))).trans_lt htop_score
  · have hx : x ≠ m := by
      intro hx
      subst x
      exact (not_lt_of_ge (Finset.le_max' Finset.univ y
        (Finset.mem_univ y))) hxy
    simpa [f, hx, hy] using gmono (show (⟨x, hx⟩ : B) < ⟨y, hy⟩ from hxy)

end FiniteStaircase

end Scott1964.MeasurementStructures
```

B.3.2 Cycle.lean

Lean 4 source

```
import Mathlib

/-!
# Local reductions in Scott's cycle argument

These lemmas are the three local moves used on pp. 8--9 of Scott (1964).
They are kept separate from the linear-inequality interface so that the
finite-array cancellation proof does not depend on names in §1.
-/

namespace Scott1964.MeasurementStructures

theorem preference_transitive_of_quadA {A : Type u} {P : A → A → Prop}
    (hirr : ∀ x, ¬ P x x)
    (hquad : ∀ x y z w, P x y → P z w → P x w ∨ P z y) :
    IsTrans A P := by
  constructor
  intro x y z hxy hyz
  rcases hquad x y y z hxy hyz with hxz | hyy
  · exact hxz
  · exact (hirr y hyy).elim

/-- Scott's first shortening move, for `z P x P y Q w`.
Here `y Q w` abbreviates `¬ w P y`. -/
theorem preference_shorten_ppq {A : Type u} {P : A → A → Prop}
    (hquad : ∀ x y z w, P x y → P z x → P w y ∨ P z w) {z x y w : A}
    (hzx : P z x) (hxy : P x y) (hyw : ¬ P w y) :
    P z w := by
  rcases hquad x y z w hxy hzx with hwy | hzw
  · exact (hyw hwy).elim
  · exact hzw

/-- The rotated form of Scott's first shortening move, for `w Q z P x P y`.
Here `w Q z` abbreviates `¬ z P w`. -/
theorem preference_shorten_qpp {A : Type u} {P : A → A → Prop}
    (hquad : ∀ x y z w, P x y → P z x → P w y ∨ P z w) {w z x y : A}
    (hwz : ¬ P z w) (hzx : P z x) (hxy : P x y) :
    P w y := by
  rcases hquad x y z w hxy hzx with hwy | hzw
  · exact hwy
  · exact (hwz hzw).elim

/-- Scott's alternating shortening move, for `x P y Q z P w`.
Here `y Q z` abbreviates `¬ z P y`. -/
theorem preference_shorten_pqp {A : Type u} {P : A → A → Prop}
    (hquad : ∀ x y z w, P x y → P z w → P x w ∨ P z y) {x y z w : A}
    (hxy : P x y) (hyz : ¬ P z y) (hzw : P z w) :
    P x w := by
  rcases hquad x y z w hxy hzw with hxw | hzy
  · exact hxw
  · exact (hyz hzy).elim
```

```
end Scott1964.MeasurementStructures
```

B.3.3 Intransitive.lean

Lean 4 source

```
import Scott1964.MeasurementStructures.Preference.Direct

/-!
# Scott's intransitive-indifference problem

The definitions reproduce conditions `(1_P)`--`(3_P)` in §2.  The elementary
direction of Theorem 2.1 is recorded here; the converse is supplied by the
finite linear-inequality engine.
-/

namespace Scott1964.MeasurementStructures

/-- A strict preference relation is represented with discrimination threshold `1`. -/
def RealizablePreference {A : Type u} (P : A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∀ x y, P x y ↔ f x ≥ f y + 1

/-- Scott 1964, condition `(1_P)`. -/
def PrefIrrefl {A : Type u} (P : A → A → Prop) : Prop :=
  ∀ x, ¬P x x

/-- Scott 1964, condition `(2_P)`. -/
def PrefQuadA {A : Type u} (P : A → A → Prop) : Prop :=
  ∀ x y z w, P x y → P z w → P x w ∨ P z y

/-- Scott 1964, condition `(3_P)`. -/
def PrefQuadB {A : Type u} (P : A → A → Prop) : Prop :=
  ∀ x y z w, P x y → P z x → P w y ∨ P z w

/-- Realization with a strict margin above the conventional unit threshold. -/
def RealizablePreferenceWithGap {A : Type u} (P : A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∃ ε > 0, ∀ x y, P x y ↔ f x ≥ f y + 1 + ε

/-- Scott's stronger conclusion following Theorem 2.1: the same unit
threshold may be taken strict on every preferred pair. -/
def StrictlyRealizablePreference {A : Type u} (P : A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∀ x y, P x y ↔ f x > f y + 1

theorem RealizablePreference.irrefl {A : Type u} {P : A → A → Prop}
    (hP : RealizablePreference P) : PrefIrrefl P := by
  rcases hP with ⟨f, hf⟩
  intro x hxx
  have := (hf x x).1 hxx
  linarith

theorem RealizablePreference.quadA {A : Type u} {P : A → A → Prop}
    (hP : RealizablePreference P) : PrefQuadA P := by
  rcases hP with ⟨f, hf⟩
  intro x y z w hxy hzw
```

```
  rw [hf] at hxy hzw
  by_cases hxw : f w + 1 ≤ f x
  · exact Or.inl ((hf x w).2 hxw)
  · apply Or.inr
    apply (hf z y).2
    have hlt : f x < f w + 1 := lt_of_not_ge hxw
    linarith

theorem RealizablePreference.quadB {A : Type u} {P : A → A → Prop}
    (hP : RealizablePreference P) : PrefQuadB P := by
  rcases hP with ⟨f, hf⟩
  intro x y z w hxy hzx
  rw [hf] at hxy hzx
  by_cases hwy : f y + 1 ≤ f w
  · exact Or.inl ((hf w y).2 hwy)
  · apply Or.inr
    apply (hf z w).2
    have hlt : f w < f y + 1 := lt_of_not_ge hwy
    linarith

theorem RealizablePreference.necessary {A : Type u} {P : A → A → Prop}
    (hP : RealizablePreference P) :
    PrefIrrefl P ∧ PrefQuadA P ∧ PrefQuadB P :=
  ⟨hP.irrefl, hP.quadA, hP.quadB⟩

/-- **Scott 1964, Theorem 2.1 (intransitive indifference).**

The converse uses the direct finite Scott--Suppes construction: quotient by
substitutability, order the quotient by the weak order `ScottWeakOrder`, and
represent its nested lower sections by a recursively constructed sequence. -/
theorem theorem_2_1 {A : Type u} [Fintype A] [Nonempty A] (P : A → A → Prop) :
    RealizablePreference P ↔ PrefIrrefl P ∧ PrefQuadA P ∧ PrefQuadB P := by
  constructor
  · exact RealizablePreference.necessary
  · rintro ⟨hirr, hA, hB⟩
    classical
    letI : Preorder A :=
      {
      le x y := ScottWeakOrder P y x
      le_refl x := scottWeakOrder_refl P x
      le_trans x y z hxy hyz := (scottWeakOrder_trans P).trans z y x hyz hxy
      lt x y := ScottWeakOrder P y x ∧ ¬ScottWeakOrder P x y
      lt_iff_le_not_ge _ _ := Iff.rfl
      }
    letI : Std.Total (α := A) (· ≤ ·) :=
      ⟨fun x y ↦ scottWeakOrder_total P hirr hA hB y x⟩
    let Q := Antisymmetrization A (· ≤ ·)
    letI : Finite Q :=
      Finite.of_surjective (toAntisymmetrization (α := A) (· ≤ ·))
        (fun q ↦ ⟨ofAntisymmetrization (· ≤ ·) q,
          toAntisymmetrization_ofAntisymmetrization (· ≤ ·) q⟩)
    letI : Fintype Q := Fintype.ofFinite Q
    let PQ : Q → Q → Prop :=
      fun x y ↦ P (ofAntisymmetrization (· ≤ ·) x)
        (ofAntisymmetrization (· ≤ ·) y)
    have hPQ_forward {x y : Q} (hxy : PQ x y) : y < x := by
```

```
  let ox := ofAntisymmetrization (· ≤ ·) x
  let oy := ofAntisymmetrization (· ≤ ·) y
  have hle : y ≤ x := by
    rw [← toAntisymmetrization_ofAntisymmetrization (· ≤ ·) y,
      ← toAntisymmetrization_ofAntisymmetrization (· ≤ ·) x,
      toAntisymmetrization_le_toAntisymmetrization_iff]
    exact preference_implies_scottWeakOrder P hirr hA hxy
  refine lt_of_le_of_ne hle ?_
  intro heq
  have hback : ScottWeakOrder P oy ox := by
    have : x ≤ y := heq.ge
    rw [← toAntisymmetrization_ofAntisymmetrization (· ≤ ·) x,
      ← toAntisymmetrization_ofAntisymmetrization (· ≤ ·) y,
      toAntisymmetrization_le_toAntisymmetrization_iff] at this
    exact this
  exact hirr ox (hback.1 ox hxy)
have hPQ_mono {x x' y y' : Q} (hxx' : x ≤ x') (hyy' : y' ≤ y)
    (hxy : PQ x y) : PQ x' y' := by
  have hx :
      ScottWeakOrder P (ofAntisymmetrization (· ≤ ·) x')
        (ofAntisymmetrization (· ≤ ·) x) := by
    have hq :
        toAntisymmetrization (· ≤ ·)
            (ofAntisymmetrization (· ≤ ·) x) ≤
          toAntisymmetrization (· ≤ ·)
            (ofAntisymmetrization (· ≤ ·) x') := by
      simpa only [toAntisymmetrization_ofAntisymmetrization] using hxx'
    change ScottWeakOrder P
      (ofAntisymmetrization (· ≤ ·) x')
      (ofAntisymmetrization (· ≤ ·) x) at hq
    exact hq
  have hy :
      ScottWeakOrder P (ofAntisymmetrization (· ≤ ·) y)
        (ofAntisymmetrization (· ≤ ·) y') := by
    have hq :
        toAntisymmetrization (· ≤ ·)
            (ofAntisymmetrization (· ≤ ·) y') ≤
          toAntisymmetrization (· ≤ ·)
            (ofAntisymmetrization (· ≤ ·) y) := by
      simpa only [toAntisymmetrization_ofAntisymmetrization] using hyy'
    change ScottWeakOrder P
      (ofAntisymmetrization (· ≤ ·) y)
      (ofAntisymmetrization (· ≤ ·) y') at hq
    exact hq
  exact scottWeakOrder_preference_scottWeakOrder P hx hxy hy
obtain ⟨g, hg, -⟩ :=
  finite_staircase_representation Q PQ hPQ_forward hPQ_mono
refine ⟨fun x ↦ g (toAntisymmetrization (· ≤ ·) x), fun x y ↦ ?_⟩
let ox := ofAntisymmetrization (· ≤ ·)
  (toAntisymmetrization (· ≤ ·) x)
let oy := ofAntisymmetrization (· ≤ ·)
  (toAntisymmetrization (· ≤ ·) y)
have hx :
    PreferenceSubstitute P x ox := by
  apply (preferenceSubstitute_iff P).2
  constructor
```

```
      · have h :
            toAntisymmetrization (· ≤ ·) ox ≤
              toAntisymmetrization (· ≤ ·) x := by
          simp [ox]
        exact h
      · have h :
            toAntisymmetrization (· ≤ ·) x ≤
              toAntisymmetrization (· ≤ ·) ox := by
          simp [ox]
        exact h
    have hy :
        PreferenceSubstitute P y oy := by
      apply (preferenceSubstitute_iff P).2
      constructor
      · have h :
            toAntisymmetrization (· ≤ ·) oy ≤
              toAntisymmetrization (· ≤ ·) y := by
          simp [oy]
        exact h
      · have h :
            toAntisymmetrization (· ≤ ·) y ≤
              toAntisymmetrization (· ≤ ·) oy := by
          simp [oy]
        exact h
    rw [show P x y ↔ P ox oy from (hy.1 x).trans (hx.2 oy)]
    exact hg _ _

/-- Any positive discrimination threshold can be changed to `1` by a change
of units, as noted immediately before §1. -/
theorem realizablePreference_threshold_iff {A : Type u} (P : A → A → Prop)
    {c : ℝ} (hc : 0 < c) :
    (∃ f : A → ℝ, ∀ x y, P x y ↔ f x ≥ f y + c) ↔
      RealizablePreference P := by
  constructor
  · rintro ⟨f, hf⟩
    refine ⟨fun x ↦ f x / c, ?_⟩
    intro x y
    rw [hf]
    constructor <;> intro h
    · rw [show f y / c + 1 = (f y + c) / c by field_simp]
      exact (div_le_div_iff_of_pos_right hc).2 h
    · rw [show f y / c + 1 = (f y + c) / c by field_simp] at h
      exact (div_le_div_iff_of_pos_right hc).1 h
  · rintro ⟨f, hf⟩
    refine ⟨fun x ↦ c * f x, ?_⟩
    intro x y
    rw [hf]
    constructor <;> intro h
    · nlinarith
    · nlinarith

/-- Rescaling any unit-threshold realization produces an arbitrary positive
margin above the unit threshold. -/
theorem RealizablePreference.withGap {A : Type u} {P : A → A → Prop}
    (hP : RealizablePreference P) : RealizablePreferenceWithGap P := by
  rcases hP with ⟨f, hf⟩
```

```
  refine ⟨fun x ↦ 2 * f x, 1, by norm_num, ?_⟩
  intro x y
  rw [hf]
  constructor <;> intro h <;> nlinarith

theorem realizablePreferenceWithGap_iff {A : Type u} (P : A → A → Prop) :
    RealizablePreferenceWithGap P ↔ RealizablePreference P := by
  constructor
  · rintro ⟨f, ε, hε, hf⟩
    exact (realizablePreference_threshold_iff P (by linarith : 0 < 1 + ε)).mp
      ⟨f, by simpa [add_assoc] using hf⟩
  · exact RealizablePreference.withGap

/-- On a finite set the boundary can be avoided altogether, as Scott notes
after Theorem 2.1: preferred pairs lie strictly beyond the same unit
threshold. -/
theorem RealizablePreference.strict_of_finite {A : Type u} [Fintype A] [Nonempty A]
    {P : A → A → Prop} (hP : RealizablePreference P) :
    StrictlyRealizablePreference P := by
  classical
  rcases hP with ⟨f, hf⟩
  let values : Finset ℝ :=
    (Finset.univ ×ˢ Finset.univ).image fun p : A × A ↦
      if P p.1 p.2 then 0 else max 0 (f p.1 - f p.2)
  have hvalues : values.Nonempty := by
    let a : A := Classical.choice ⟨Nonempty A⟩
    exact ⟨if P a a then 0 else max 0 (f a - f a),
      Finset.mem_image.mpr ⟨(a, a), by simp, rfl⟩⟩
  let M : ℝ := values.max' hvalues
  have hM_nonneg : 0 ≤ M := by
    obtain ⟨v, hv⟩ := hvalues
    have hv_nonneg : 0 ≤ v := by
      rcases Finset.mem_image.mp hv with ⟨p, -, rfl⟩
      split <;> simp
    exact hv_nonneg.trans (Finset.le_max' values v hv)
  have hM_lt_one : M < 1 := by
    apply (Finset.max'_lt_iff values hvalues).mpr
    intro v hv
    rcases Finset.mem_image.mp hv with ⟨p, -, rfl⟩
    split_ifs with hp
    · norm_num
    · have hlt : f p.1 - f p.2 < 1 := by
        have := not_le.mp (mt (hf p.1 p.2).mpr hp)
        linarith
      exact max_lt (by norm_num) hlt
  have hbound {x y : A} (hxy : ¬P x y) : f x - f y ≤ M := by
    have hmem : max 0 (f x - f y) ∈ values := by
      exact Finset.mem_image.mpr ⟨(x, y), by simp, by simp [hxy]⟩
    exact (le_max_right 0 (f x - f y)).trans (Finset.le_max' values _ hmem)
  let s : ℝ := 1 + (1 - M) / 2
  have hs : 1 < s := by
    dsimp [s]
    linarith
  refine ⟨fun x ↦ s * f x, fun x y ↦ ?_⟩
  constructor
  · intro hxy
```

```
    have hdiff : 1 ≤ f x - f y := by
      have := (hf x y).mp hxy
      linarith
    have hspos : 0 < s := lt_trans (by norm_num) hs
    have : s ≤ s * (f x - f y) := by nlinarith
    dsimp
    nlinarith
  · intro hstrict
    by_contra hxy
    have hd := hbound hxy
    have hdiff_lt : f x - f y < 1 := by
      have := not_le.mp (mt (hf x y).mpr hxy)
      linarith
    have hspos : 0 < s := lt_trans (by norm_num) hs
    have hscaled : s * (f x - f y) ≤ 1 := by
      by_cases hd0 : f x - f y ≤ 0
      · nlinarith
      · have hprod : s * (f x - f y) ≤ s * M :=
          mul_le_mul_of_nonneg_left hd hspos.le
        dsimp [s] at hprod ⊢
        nlinarith [mul_nonneg (sub_nonneg.mpr hM_nonneg)
          (sub_nonneg.mpr (le_of_lt hM_lt_one))]
    dsimp at hstrict
    nlinarith

end Scott1964.MeasurementStructures
```

B.4 Differences

B.4.1 Pair.lean

Lean 4 source

```
import Scott1964.MeasurementStructures.FinHead
import Scott1964.MeasurementStructures.LinearInequalities.ScottTheorems

/-!
# Additive utility for pairs

This is Scott 1964, Theorem 3.1.
-/

namespace Scott1964.MeasurementStructures

open scoped BigOperators
open scoped Classical

/-- The incidence vector of a pair, with one coordinate in each summand. -/
noncomputable def pairVector {A : Type u} {A' : Type v} (x : A) (x' : A') :
    Sum A A' → ℝ :=
  Pi.single (Sum.inl x) 1 + Pi.single (Sum.inr x') 1

theorem pairVector_injective {A : Type u} {A' : Type v} :
    Function.Injective (Function.uncurry (@pairVector A A')) := by
```

```
  rintro ⟨x, x'⟩ ⟨y, y'⟩ h
  have hx := congrFun h (Sum.inl x)
  have hx' := congrFun h (Sum.inr x')
  simp [pairVector, Pi.single_apply] at hx hx'
  exact Prod.ext hx hx'

/-- Equality of sums of incidence vectors gives the two independent
permutations of coordinates used in condition `(2_V)`. -/
theorem pairVector_sum_eq_permutations {A : Type u} {A' : Type v}
    {ι : Type*} [Fintype ι] (x y : ι → A) (x' y' : ι → A')
    (h : ∑ i, pairVector (x i) (x' i) =
      ∑ i, pairVector (y i) (y' i)) :
    ∃ π σ : Equiv.Perm ι,
      (∀ i, y i = x (π i)) ∧ (∀ i, y' i = x' (σ i)) := by
  classical
  have hleft (a : A) :
      Fintype.card {i // y i = a} = Fintype.card {i // x i = a} := by
    have ha := congrFun h (Sum.inl a)
    simp only [Fintype.sum_apply, pairVector, Pi.add_apply, Pi.single_apply,
      Sum.inl.injEq, Sum.inl_ne_inr, ↓reduceIte, add_zero] at ha
    have hy :
        Fintype.card {i // y i = a} =
          (Finset.univ.filter fun i ↦ y i = a).card := by
      rw [← Fintype.card_coe]
      exact Fintype.card_congr
        (Equiv.subtypeEquivRight fun i ↦ by simp)
    have hx :
        Fintype.card {i // x i = a} =
          (Finset.univ.filter fun i ↦ x i = a).card := by
      rw [← Fintype.card_coe]
      exact Fintype.card_congr
        (Equiv.subtypeEquivRight fun i ↦ by simp)
    rw [hy, hx]
    rw [Finset.card_filter, Finset.card_filter]
    exact_mod_cast (by simpa [eq_comm] using ha.symm)
  have hright (a : A') :
      Fintype.card {i // y' i = a} = Fintype.card {i // x' i = a} := by
    have ha := congrFun h (Sum.inr a)
    simp only [Fintype.sum_apply, pairVector, Pi.add_apply, Pi.single_apply,
      Sum.inr.injEq, Sum.inr_ne_inl, ↓reduceIte, zero_add] at ha
    have hy :
        Fintype.card {i // y' i = a} =
          (Finset.univ.filter fun i ↦ y' i = a).card := by
      rw [← Fintype.card_coe]
      exact Fintype.card_congr
        (Equiv.subtypeEquivRight fun i ↦ by simp)
    have hx :
        Fintype.card {i // x' i = a} =
          (Finset.univ.filter fun i ↦ x' i = a).card := by
      rw [← Fintype.card_coe]
      exact Fintype.card_congr
        (Equiv.subtypeEquivRight fun i ↦ by simp)
    rw [hy, hx]
    rw [Finset.card_filter, Finset.card_filter]
    exact_mod_cast (by simpa [eq_comm] using ha.symm)
  let π : Equiv.Perm ι :=
```

```
    Equiv.ofFiberEquiv fun a ↦ Fintype.equivOfCardEq (hleft a)
  let σ : Equiv.Perm ι :=
    Equiv.ofFiberEquiv fun a ↦ Fintype.equivOfCardEq (hright a)
  refine ⟨π, σ, ?_, ?_⟩
  · intro i
    change y i = x ((Equiv.ofFiberEquiv
      (fun a ↦ Fintype.equivOfCardEq (hleft a))) i)
    exact ((Fintype.equivOfCardEq (hleft (y i)))
      ((Equiv.sigmaFiberEquiv y).symm i).2).2.symm
  · intro i
    change y' i = x' ((Equiv.ofFiberEquiv
      (fun a ↦ Fintype.equivOfCardEq (hright a))) i)
    exact ((Fintype.equivOfCardEq (hright (y' i)))
      ((Equiv.sigmaFiberEquiv y').symm i).2).2.symm

/-- A relation on pairs is represented by the sum of two utility functions. -/
def RealizableUtilityPair {A : Type u} {A' : Type v}
    (V : A → A' → A → A' → Prop) : Prop :=
  ∃ f : A → ℝ, ∃ f' : A' → ℝ,
    ∀ x x' y y', V x x' y y' ↔ f x + f' x' ≥ f y + f' y'

/-- Scott 1964, condition `(1_V)`. -/
def PairTotal {A : Type u} {A' : Type v}
    (V : A → A' → A → A' → Prop) : Prop :=
  ∀ x x' y y', V x x' y y' ∨ V y y' x x'

/-- Scott 1964, condition `(2_V)`, indexed at length `n + 1` so that the
distinguished index `0` always exists. -/
def PairPermutation {A : Type u} {A' : Type v}
    (V : A → A' → A → A' → Prop) : Prop :=
  ∀ (n : ℕ) (x : Fin (n + 1) → A) (x' : Fin (n + 1) → A')
    (π σ : Equiv.Perm (Fin (n + 1))),
    (∀ i, i ≠ finHead n → V (x i) (x' i) (x (π i)) (x' (σ i))) →
      V (x (π (finHead n))) (x' (σ (finHead n))) (x (finHead n)) (x' (finHead n))

/-- Scott's observation following `(2_V)`: the permutation condition already
implies transitivity. -/
theorem PairPermutation.transitive {A : Type u} {A' : Type v}
    {V : A → A' → A → A' → Prop} (hV : PairPermutation V) :
    ∀ p q r : A × A',
      V p.1 p.2 q.1 q.2 → V q.1 q.2 r.1 r.2 → V p.1 p.2 r.1 r.2 := by
  rintro ⟨x, x'⟩ ⟨y, y'⟩ ⟨z, z'⟩ hxy hyz
  let p : Equiv.Perm (Fin 3) :=
    Equiv.ofBijective ![1, 2, 0] (by native_decide)
  apply hV 2 ![z, x, y] ![z', x', y'] p p
  intro i hi
  have hi0 : i ≠ 0 := by simpa [finHead_eq_zero] using hi
  fin_cases i
  · contradiction
  · simpa [p] using hxy
  · simpa [p] using hyz

theorem RealizableUtilityPair.total {A : Type u} {A' : Type v}
    {V : A → A' → A → A' → Prop} (hV : RealizableUtilityPair V) :
    PairTotal V := by
  rcases hV with ⟨f, f', hrep⟩
```

```
  intro x x' y y'
  rcases le_total (f y + f' y') (f x + f' x') with h | h
  · exact Or.inl ((hrep x x' y y').2 h)
  · exact Or.inr ((hrep y y' x x').2 h)

theorem RealizableUtilityPair.permutation {A : Type u} {A' : Type v}
    {V : A → A' → A → A' → Prop} (hV : RealizableUtilityPair V) :
    PairPermutation V := by
  rcases hV with ⟨f, f', hrep⟩
  intro n x x' π σ h
  let s : Fin (n + 1) → ℝ := fun i ↦ f (x i) + f' (x' i)
  let t : Fin (n + 1) → ℝ := fun i ↦ f (x (π i)) + f' (x' (σ i))
  have hπ : ∑ i, f (x (π i)) = ∑ i, f (x i) :=
    Fintype.sum_equiv π _ _ (fun _ ↦ rfl)
  have hσ : ∑ i, f' (x' (σ i)) = ∑ i, f' (x' i) :=
    Fintype.sum_equiv σ _ _ (fun _ ↦ rfl)
  have hsum : ∑ i, s i = ∑ i, t i := by
    simp only [s, t, Finset.sum_add_distrib]
    rw [hπ, hσ]
  have hrest :
      ∑ i ∈ (Finset.univ.erase (finHead n)), t i ≤
        ∑ i ∈ (Finset.univ.erase (finHead n)), s i := by
    apply Finset.sum_le_sum
    intro i hi
    have hi0 : i ≠ finHead n := by
      simpa using (Finset.mem_erase.mp hi).1
    exact (hrep _ _ _ _).1 (h i hi0)
  apply (hrep _ _ _ _).2
  have hzero : finHead n ∈ Finset.univ := Finset.mem_univ _
  have hs := Finset.sum_erase_add _ s hzero
  have ht := Finset.sum_erase_add _ t hzero
  dsimp only [s, t] at *
  linarith

theorem RealizableUtilityPair.necessary {A : Type u} {A' : Type v}
    {V : A → A' → A → A' → Prop} (hV : RealizableUtilityPair V) :
    PairTotal V ∧ PairPermutation V :=
  ⟨hV.total, hV.permutation⟩

/-- **Scott 1964, Theorem 3.1 (additive utility for pairs).** -/
theorem theorem_3_1 {A : Type u} {A' : Type v}
    [Fintype A] [Nonempty A] [Fintype A'] [Nonempty A']
    (V : A → A' → A → A' → Prop) :
    RealizableUtilityPair V ↔ PairTotal V ∧ PairPermutation V := by
  constructor
  · exact RealizableUtilityPair.necessary
  · rintro ⟨htotal, hperm⟩
    let Y : Set (Sum A A' → ℝ) :=
      Set.range (Function.uncurry (@pairVector A A'))
    let R : (Sum A A' → ℝ) → (Sum A A' → ℝ) → Prop :=
      fun z w ↦ ∀ x x' y y',
        z = pairVector x x' → w = pairVector y y' → V x x' y y'
    have hR_pair (x : A) (x' : A') (y : A) (y' : A') :
        R (pairVector x x') (pairVector y y') ↔ V x x' y y' := by
      constructor
      · intro h
```

```
    exact h x x' y y' rfl rfl
  · intro h a a' b b' ha hb
    have hxa : (x, x') = (a, a') :=
      pairVector_injective ha
    have hyb : (y, y') = (b, b') :=
      pairVector_injective hb
    have hxa₁ : x = a := congrArg Prod.fst hxa
    have hxa₂ : x' = a' := congrArg Prod.snd hxa
    have hyb₁ : y = b := congrArg Prod.fst hyb
    have hyb₂ : y' = b' := congrArg Prod.snd hyb
    simpa [← hxa₁, ← hxa₂, ← hyb₁, ← hyb₂] using h
have hYfinite : Y.Finite := Set.finite_range _
have hYrat : LinearInequalities.IsRationalSet Y := by
  rintro z ⟨⟨x, x'⟩, rfl⟩ s
  rcases s with a | a'
  · by_cases ha : a = x
    · exact ⟨1, by simp [pairVector, ha]⟩
    · exact ⟨0, by simp [pairVector, ha]⟩
  · by_cases ha : a' = x'
    · exact ⟨1, by simp [pairVector, ha]⟩
    · exact ⟨0, by simp [pairVector, ha]⟩
have hcomplete : LinearInequalities.RelationComplete Y R := by
  rintro z ⟨⟨x, x'⟩, rfl⟩ w ⟨⟨y, y'⟩, rfl⟩
  change R (pairVector x x') (pairVector y y') ∨
    R (pairVector y y') (pairVector x x')
  rw [hR_pair, hR_pair]
  exact htotal x x' y y'
have hcancel : LinearInequalities.RelationSequenceCancellation Y R := by
  intro n z w hz hw hR hsum
  choose p hp using hz
  choose q hq using hw
  let x : Fin (n + 1) → A := fun i ↦ (p i).1
  let x' : Fin (n + 1) → A' := fun i ↦ (p i).2
  let y : Fin (n + 1) → A := fun i ↦ (q i).1
  let y' : Fin (n + 1) → A' := fun i ↦ (q i).2
  have hx (i) : z i = pairVector (x i) (x' i) := by
    have hi := hp i
    change pairVector (p i).1 (p i).2 = z i at hi
    simpa [x, x'] using hi.symm
  have hy (i) : w i = pairVector (y i) (y' i) := by
    have hi := hq i
    change pairVector (q i).1 (q i).2 = w i at hi
    simpa [y, y'] using hi.symm
  have hsum' :
      ∑ i, pairVector (x i) (x' i) =
        ∑ i, pairVector (y i) (y' i) := by
    simpa only [hx, hy] using hsum
  obtain ⟨π, σ, hπ, hσ⟩ :=
    pairVector_sum_eq_permutations x y x' y' hsum'
  intro i
  let τ : Equiv.Perm (Fin (n + 1)) := Equiv.swap (finHead n) i
  let π' : Equiv.Perm (Fin (n + 1)) := τ.trans (π.trans τ)
  let σ' : Equiv.Perm (Fin (n + 1)) := τ.trans (σ.trans τ)
  have hconcl := hperm n (fun j ↦ x (τ j)) (fun j ↦ x' (τ j)) π' σ' (by
    intro j hj
    have hτj : τ j ≠ i := by
```

```
          intro heq
          have : j = finHead n := τ.injective (by simpa [τ] using heq)
          exact hj this
        have hvR := hR (τ j)
        rw [hx (τ j), hy (τ j), hR_pair] at hvR
        have hv := hvR
        simpa [π', σ', τ, hπ (τ j), hσ (τ j)] using hv)
      rw [hy i, hx i, hR_pair]
      simpa [π', σ', τ, hπ i, hσ i] using hconcl
    obtain ⟨φ, hφ⟩ :=
      (LinearInequalities.scott_theorem_1_3 hYfinite hYrat).2
        ⟨hcomplete, hcancel⟩
    refine ⟨fun x ↦ φ (Pi.single (Sum.inl x) 1),
      fun x' ↦ φ (Pi.single (Sum.inr x') 1), ?_⟩
    intro x x' y y'
    have hxy := hφ (pairVector x x') ⟨(x, x'), rfl⟩
      (pairVector y y') ⟨(y, y'), rfl⟩
    rw [hR_pair] at hxy
    simpa only [pairVector, map_add] using hxy

end Scott1964.MeasurementStructures
```

B.4.2 Ordered.lean

Lean 4 source

```
import Scott1964.MeasurementStructures.Differences.Pair

/-!
# Ordered differences

Scott's reduction of the one-utility problem to Theorem 3.1 is entirely
algebraic.  The key point is that reversal lets the two utilities be combined.
-/

namespace Scott1964.MeasurementStructures

/-- Representation of comparisons of utility differences. -/
def RealizableDifference {A : Type u} (D : A → A → A → A → Prop) : Prop :=
  ∃ f : A → ℝ, ∀ x y z w, D x y z w ↔ f x - f y ≥ f z - f w

/-- Scott 1964, condition `(1_D)`. -/
def DiffTotal {A : Type u} (D : A → A → A → A → Prop) : Prop :=
  ∀ x y z w, D x y z w ∨ D z w x y

/-- Scott 1964, condition `(2_D)`. -/
def DiffPermutation {A : Type u} (D : A → A → A → A → Prop) : Prop :=
  ∀ (n : ℕ) (x y : Fin (n + 1) → A) (π σ : Equiv.Perm (Fin (n + 1))),
    (∀ i, i ≠ finHead n → D (x i) (y i) (x (π i)) (y (σ i))) →
      D (x (π (finHead n))) (y (σ (finHead n))) (x (finHead n)) (y (finHead n))

/-- Scott 1964, condition `(3_D)`. -/
def DiffReversal {A : Type u} (D : A → A → A → A → Prop) : Prop :=
  ∀ x y z w, D x y z w → D w z y x
```

```
theorem RealizableDifference.utilityPair {A : Type u}
    {D : A → A → A → A → Prop} (hD : RealizableDifference D) :
    RealizableUtilityPair D := by
  rcases hD with ⟨f, hf⟩
  refine ⟨f, fun y ↦ -f y, ?_⟩
  intro x y z w
  rw [hf]
  constructor <;> intro h <;> dsimp at h ⊢ <;> linarith

theorem RealizableDifference.reversal {A : Type u}
    {D : A → A → A → A → Prop} (hD : RealizableDifference D) :
    DiffReversal D := by
  rcases hD with ⟨f, hf⟩
  intro x y z w h
  rw [hf] at h ⊢
  linarith

theorem RealizableDifference.necessary {A : Type u}
    {D : A → A → A → A → Prop} (hD : RealizableDifference D) :
    DiffTotal D ∧ DiffPermutation D ∧ DiffReversal D := by
  have hp := hD.utilityPair.necessary
  exact ⟨hp.1, hp.2, hD.reversal⟩

/-- The combining-utilities argument in the proof of Theorem 3.2. -/
theorem difference_of_pair_and_reversal {A : Type u}
    {D : A → A → A → A → Prop} (hp : RealizableUtilityPair D)
    (hr : DiffReversal D) : RealizableDifference D := by
  rcases hp with ⟨g, q, hrep⟩
  refine ⟨fun x ↦ g x - q x, ?_⟩
  intro x y z w
  constructor
  · intro h
    have h₁ := (hrep x y z w).1 h
    have h₂ := (hrep w z y x).1 (hr x y z w h)
    dsimp
    linarith
  · intro hdiff
    by_contra hn
    have hstrict : g x + q y < g z + q w :=
      lt_of_not_ge (fun h ↦ hn ((hrep x y z w).2 h))
    have hreverse : D z w x y := (hrep z w x y).2 (le_of_lt hstrict)
    have h₂ := (hrep y x w z).1 (hr z w x y hreverse)
    dsimp at hdiff
    linarith

/-- The exact logical reduction used for Theorem 3.2. -/
theorem realizableDifference_iff_pair_reversal {A : Type u}
    (D : A → A → A → A → Prop) :
    RealizableDifference D ↔ RealizableUtilityPair D ∧ DiffReversal D := by
  exact ⟨fun h ↦ ⟨h.utilityPair, h.reversal⟩,
    fun h ↦ difference_of_pair_and_reversal h.1 h.2⟩

/-- Once Theorem 3.1 is available, its sufficiency immediately yields Scott's
Theorem 3.2 through the preceding combining lemma. -/
theorem difference_sufficient_of_pair_sufficient {A : Type u}
```

```
    (pairSufficient :
      ∀ V : A → A → A → A → Prop,
        PairTotal V ∧ PairPermutation V → RealizableUtilityPair V)
    {D : A → A → A → A → Prop}
    (hD : DiffTotal D ∧ DiffPermutation D ∧ DiffReversal D) :
    RealizableDifference D :=
  difference_of_pair_and_reversal (pairSufficient D ⟨hD.1, hD.2.1⟩) hD.2.2

/-- **Scott 1964, Theorem 3.2 (ordered differences).** -/
theorem theorem_3_2 {A : Type u} [Fintype A] [Nonempty A]
    (D : A → A → A → A → Prop) :
    RealizableDifference D ↔ DiffTotal D ∧ DiffPermutation D ∧ DiffReversal D := by
  constructor
  · exact RealizableDifference.necessary
  · rintro ⟨htotal, hperm, hreversal⟩
    exact difference_of_pair_and_reversal
      ((theorem_3_1 D).2 ⟨htotal, hperm⟩) hreversal

end Scott1964.MeasurementStructures
```

B.5 Probability

B.5.1 Basic.lean

Lean 4 source

```
import Mathlib
import Scott1964.MeasurementStructures.FinHead

/-!
# Finitely additive probabilities on Boolean algebras

Scott's probabilities are real-valued and finitely additive.  This file deliberately
does not use Mathlib's countably additive `MeasureTheory.Measure`: the source works
with an arbitrary Boolean algebra and only finite disjoint unions.
-/

namespace Scott1964.MeasurementStructures

open scoped BigOperators Classical

/-- A normalized finitely additive probability on a Boolean algebra. -/
structure IsProbability {B : Type u} [BooleanAlgebra B] (μ : B → ℝ) : Prop where
  bot : μ ⊥ = 0
  top : μ ⊤ = 1
  additive : ∀ x y, Disjoint x y → μ (x ⊔ y) = μ x + μ y
  nonnegative : ∀ x, 0 ≤ μ x

/-- A finitely additive signed charge.  Unlike `IsProbability`, no
nonnegativity or normalization is imposed. -/
structure IsSignedCharge {B : Type u} [BooleanAlgebra B] (μ : B → ℝ) : Prop where
  bot : μ ⊥ = 0
  additive : ∀ x y, Disjoint x y → μ (x ⊔ y) = μ x + μ y
```

```
/-- Nonnegativity of a finitely additive set function. -/
def IsNonnegative {B : Type u} [BooleanAlgebra B] (μ : B → ℝ) : Prop :=
  ∀ x, 0 ≤ μ x

/-- Compatibility name for a finitely additive probability. -/
def IsFinitelyAdditiveProbability {B : Type u} [BooleanAlgebra B] (μ : B → ℝ) : Prop
    :=
  IsProbability μ

/-- A qualitative ordering is represented exactly by a normalized additive function.
    -/
def RealizableProbability {B : Type u} [BooleanAlgebra B] (R : B → B → Prop) : Prop
    :=
  ∃ μ : B → ℝ, IsProbability μ ∧ ∀ x y, R x y ↔ μ x ≥ μ y

/-- A qualitative relation is represented by a finitely additive signed
charge, with no normalization or positivity requirement. -/
def RealizableSignedCharge {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop) : Prop :=
  ∃ μ : B → ℝ, IsSignedCharge μ ∧ ∀ x y, R x y ↔ μ x ≥ μ y

/-- Scott's strict notation: `x ≻ y` means that `y` is not weakly preferred to `x`. -/
def StrictlyPreferred {B : Type u} (R : B → B → Prop) (x y : B) : Prop :=
  ¬R y x

/-- Scott 1964, condition `(1_B)`: the certain event is strictly above the null event.
    -/
def ProbNontrivial {B : Type u} [BooleanAlgebra B] (R : B → B → Prop) : Prop :=
  R ⊤ ⊥ ∧ ¬R ⊥ ⊤

/-- Scott 1964, condition `(2_B)`: every event is at least as likely as null. -/
def ProbNonneg {B : Type u} [BooleanAlgebra B] (R : B → B → Prop) : Prop :=
  ∀ x, R x ⊥

/-- Scott 1964, condition `(3_B)`: qualitative probabilities are comparable. -/
def ProbTotal {B : Type u} [BooleanAlgebra B] (R : B → B → Prop) : Prop :=
  ∀ x y, R x y ∨ R y x

/-- Scott 1964, condition `(4_B)`.  The equality says that every atom occurs
equally often on the two sides. -/
def ProbCancellation {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) : Prop :=
  ∀ (n : ℕ) (x y : Fin (n + 1) → B),
    (∀ a : B, IsAtom a →
        (∑ i, if a ≤ x i then 1 else 0) =
          (∑ i, if a ≤ y i then 1 else 0 : ℕ)) →
      (∀ i, i ≠ finHead n → R (x i) (y i)) → R (y (finHead n)) (x (finHead n))

/-- Transitivity, de Finetti's condition (iv). -/
def ProbTransitive {B : Type u} (R : B → B → Prop) : Prop :=
  ∀ {{x y z}}, R x y → R y z → R x z

/-- Invariance under adjoining one event disjoint from both sides, de Finetti's
condition (v). -/
def ProbDisjointUnionInvariant {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop) : Prop :=
```

```
  ∀ {{x y z}}, Disjoint z x → Disjoint z y →
    (R x y ↔ R (x ⊔ z) (y ⊔ z))

/-- De Finetti's five qualitative-probability axioms.  Condition (i) is
literally `¬ R ⊥ ⊤`; it does not include Scott's additional `R ⊤ ⊥`
conjunct from `(1_B)`. -/
structure DeFinettiAxioms {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop) : Prop where
  nontrivial : ¬R ⊥ ⊤
  bottom : ∀ x, R x ⊥
  total : ProbTotal R
  transitive : ProbTransitive R
  disjointUnionInvariant : ProbDisjointUnionInvariant R

theorem IsProbability.compl {B : Type u} [BooleanAlgebra B] {μ : B → ℝ}
    (hμ : IsProbability μ) (x : B) : μ xᶜ = 1 - μ x := by
  have hd : Disjoint x xᶜ := disjoint_compl_right
  have h := hμ.additive x xᶜ hd
  rw [sup_compl_eq_top, hμ.top] at h
  linarith

theorem IsProbability.mono_of_nonnegative {B : Type u} [BooleanAlgebra B]
    {μ : B → ℝ} (hμ : IsProbability μ) (hn : IsNonnegative μ)
    {{x y : B}} (hxy : x ≤ y) : μ x ≤ μ y := by
  have hd : Disjoint x (y \ x) := by
    rw [disjoint_iff_inf_le]
    simp
  have hy : x ⊔ (y \ x) = y := by
    exact sup_sdiff_cancel_right hxy
  rw [← hy, hμ.additive _ _ hd]
  exact le_add_of_nonneg_right (hn _)

theorem IsProbability.mono {B : Type u} [BooleanAlgebra B]
    {μ : B → ℝ} (hμ : IsProbability μ) {{x y : B}} (hxy : x ≤ y) :
    μ x ≤ μ y :=
  hμ.mono_of_nonnegative hμ.nonnegative hxy

theorem IsProbability.le_one_of_nonnegative {B : Type u} [BooleanAlgebra B]
    {μ : B → ℝ} (hμ : IsProbability μ) (hn : IsNonnegative μ) (x : B) :
    μ x ≤ 1 := by
  simpa [hμ.top] using hμ.mono_of_nonnegative hn (show x ≤ (⊤ : B) from le_top)

theorem IsProbability.le_one {B : Type u} [BooleanAlgebra B]
    {μ : B → ℝ} (hμ : IsProbability μ) (x : B) : μ x ≤ 1 :=
  hμ.le_one_of_nonnegative hμ.nonnegative x

theorem RealizableProbability.nonnegative {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop} (hR : RealizableProbability R) (_h0 : ProbNonneg R) :
    ∃ μ : B → ℝ, IsFinitelyAdditiveProbability μ ∧
      ∀ x y, R x y ↔ μ x ≥ μ y := by
  rcases hR with ⟨μ, hμ, hrep⟩
  exact ⟨μ, hμ, hrep⟩

theorem RealizableProbability.probNonneg {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop} (hR : RealizableProbability R) :
    ProbNonneg R := by
```

```
  rcases hR with ⟨μ, hμ, hrep⟩
  intro x
  apply (hrep x ⊥).2
  simpa [hμ.bot] using hμ.nonnegative x

theorem RealizableProbability.probTotal {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop} (hR : RealizableProbability R) : ProbTotal R := by
  rcases hR with ⟨μ, -, hrep⟩
  intro x y
  rcases le_total (μ y) (μ x) with h | h
  · exact Or.inl ((hrep x y).2 h)
  · exact Or.inr ((hrep y x).2 h)

theorem RealizableProbability.probNontrivial {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop} (hR : RealizableProbability R) :
    ProbNontrivial R := by
  rcases hR with ⟨μ, hμ, hrep⟩
  constructor
  · exact (hrep ⊤ ⊥).2 (by rw [hμ.top, hμ.bot]; norm_num)
  · intro h
    have := (hrep ⊥ ⊤).1 h
    rw [hμ.top, hμ.bot] at this
    norm_num at this

theorem RealizableProbability.probTransitive {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop} (hR : RealizableProbability R) : ProbTransitive R := by
  rcases hR with ⟨μ, -, hrep⟩
  intro x y z hxy hyz
  exact (hrep x z).2 (((hrep y z).1 hyz).trans ((hrep x y).1 hxy))

theorem RealizableProbability.disjointUnionInvariant {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop} (hR : RealizableProbability R) :
    ProbDisjointUnionInvariant R := by
  rcases hR with ⟨μ, hμ, hrep⟩
  intro x y z hzx hzy
  rw [hrep x y, hrep (x ⊔ z) (y ⊔ z)]
  rw [hμ.additive x z hzx.symm, hμ.additive y z hzy.symm]
  constructor <;> intro h <;> linarith

end Scott1964.MeasurementStructures
```

B.5.2 Atoms.lean

Lean 4 source

```
import Mathlib.Order.Atoms
import Scott1964.MeasurementStructures.Probability.Basic

/-!
# Atom vectors for finite Boolean algebras

Scott identifies an event with its `0`/`1` characteristic vector on the atoms.
The cancellation identity in Theorem 4.1 is equality of sums of these vectors.
-/
```

```
namespace Scott1964.MeasurementStructures

open scoped BigOperators Classical

noncomputable section

/-- The real characteristic vector of an event on Boolean atoms. -/
def atomVector {B : Type u} [BooleanAlgebra B] (x : B) :
    {a : B // IsAtom a} → ℝ :=
  fun a ↦ if (a : B) ≤ x then 1 else 0

@[simp]
theorem atomVector_apply {B : Type u} [BooleanAlgebra B] (x : B)
    (a : {a : B // IsAtom a}) :
    atomVector x a = if (a : B) ≤ x then 1 else 0 :=
  rfl

theorem atomVector_bot {B : Type u} [BooleanAlgebra B] :
    atomVector (⊥ : B) = 0 := by
  funext a
  simp [atomVector, a.property.1]

theorem atomVector_top {B : Type u} [BooleanAlgebra B] :
    atomVector (⊤ : B) = 1 := by
  funext a
  simp [atomVector]

theorem atomVector_sup_of_disjoint {B : Type u} [BooleanAlgebra B]
    {x y : B} (hxy : Disjoint x y) :
    atomVector (x ⊔ y) = atomVector x + atomVector y := by
  funext a
  have hatom := a.property
  by_cases hx : (a : B) ≤ x
  · have hny : ¬(a : B) ≤ y := by
      intro hy
      have : (a : B) ≤ x ⊓ y := le_inf hx hy
      rw [hxy.eq_bot] at this
      exact hatom.1 (bot_unique this)
    have hsup : (a : B) ≤ x ⊔ y := hx.trans le_sup_left
    simp [atomVector, hx, hny, hsup]
  · by_cases hy : (a : B) ≤ y
    · have hsup : (a : B) ≤ x ⊔ y := hy.trans le_sup_right
      simp [atomVector, hx, hy, hsup]
    · have hsup : ¬(a : B) ≤ x ⊔ y := by
        intro ha
        have hix : (a : B) ⊓ x = ⊥ := by
          rcases hatom.le_iff.mp inf_le_left with hbot | heq
          · exact hbot
          · exact False.elim (hx (by rw [← heq]; exact inf_le_right))
        have hiy : (a : B) ⊓ y = ⊥ := by
          rcases hatom.le_iff.mp inf_le_left with hbot | heq
          · exact hbot
          · exact False.elim (hy (by rw [← heq]; exact inf_le_right))
        have haeq : (a : B) ⊓ (x ⊔ y) = a := inf_eq_left.mpr ha
        rw [inf_sup_left, hix, hiy, bot_sup_eq] at haeq
```

```
        exact hatom.1 haeq.symm
      simp [atomVector, hx, hy, hsup]

theorem atomVector_injective {B : Type u} [BooleanAlgebra B] [Fintype B] :
    Function.Injective (atomVector : B → ({a : B // IsAtom a} → ℝ)) := by
  let _ : IsAtomic B := isAtomic_of_orderBot_wellFounded_lt wellFounded_lt
  intro x y hxy
  apply BooleanAlgebra.eq_iff_atom_le_iff.mpr
  intro a ha
  have h := congrFun hxy ⟨a, ha⟩
  simp only [atomVector_apply] at h
  constructor <;> intro hle
  · by_contra hn
    simp [hle, hn] at h
  · by_contra hn
    simp [hle, hn] at h

/-- Distinct Boolean atoms are disjoint. -/
theorem disjoint_atoms_of_ne {B : Type u} [BooleanAlgebra B]
    (a b : {a : B // IsAtom a}) (hab : a ≠ b) :
    Disjoint (a : B) b := by
  rw [disjoint_iff_inf_le]
  rcases a.property.le_iff.mp inf_le_left with hbot | heq
  · exact le_of_eq hbot
  · have hab_le : (a : B) ≤ b := by
      rw [← heq]
      exact inf_le_right
    rcases b.property.le_iff.mp hab_le with ha0 | heqab
    · exact False.elim (a.property.1 ha0)
    · exact False.elim (hab (Subtype.ext heqab))

theorem disjoint_atom_finset_sup {B : Type u} [BooleanAlgebra B]
    (a : {a : B // IsAtom a}) (s : Finset {a : B // IsAtom a})
    (ha : a ∉ s) :
    Disjoint (a : B) (s.sup fun b ↦ (b : B)) := by
  induction s using Finset.induction_on with
  | empty => simp
  | @insert b s hb ih =>
      rw [Finset.sup_insert]
      exact (disjoint_sup_right).2
        ⟨disjoint_atoms_of_ne a b (by
            intro hab
            subst b
            exact ha (Finset.mem_insert_self _ _)),
          ih (fun has ↦ ha (Finset.mem_insert_of_mem has))⟩

/-- A signed charge evaluates a supremum of distinct atoms as the sum of
their charges. -/
theorem IsSignedCharge.finset_sup_atoms
    {B : Type u} [BooleanAlgebra B] {μ : B → ℝ}
    (hμ : IsSignedCharge μ) (s : Finset {a : B // IsAtom a}) :
    μ (s.sup fun a ↦ (a : B)) = ∑ a ∈ s, μ a := by
  induction s using Finset.induction_on with
  | empty => simp [hμ.bot]
  | @insert a s ha ih =>
      rw [Finset.sup_insert, hμ.additive _ _ (disjoint_atom_finset_sup a s ha),
```

```
        Finset.sum_insert ha, ih]

/-- Finite additivity evaluates a supremum of distinct atoms as the sum of
their masses. -/
theorem IsProbability.finset_sup_atoms
    {B : Type u} [BooleanAlgebra B] {μ : B → ℝ}
    (hμ : IsProbability μ) (s : Finset {a : B // IsAtom a}) :
    μ (s.sup fun a ↦ (a : B)) = ∑ a ∈ s, μ a := by
  induction s using Finset.induction_on with
  | empty => simp [hμ.bot]
  | @insert a s ha ih =>
      rw [Finset.sup_insert, hμ.additive _ _ (disjoint_atom_finset_sup a s ha),
        Finset.sum_insert ha, ih]

/-- The atoms below an event. -/
def atomsBelow {B : Type u} [BooleanAlgebra B] [Fintype B] (x : B) :
    Finset {a : B // IsAtom a} :=
  Finset.univ.filter fun a ↦ (a : B) ≤ x

theorem sup_atomsBelow {B : Type u} [BooleanAlgebra B] [Fintype B] (x : B) :
    (atomsBelow x).sup (fun a ↦ (a : B)) = x := by
  let _ : IsAtomic B := isAtomic_of_orderBot_wellFounded_lt wellFounded_lt
  apply BooleanAlgebra.eq_iff_atom_le_iff.mpr
  intro a ha
  constructor
  · intro h
    exact h.trans (Finset.sup_le fun b hb ↦
      (Finset.mem_filter.mp hb).2)
  · intro h
    exact Finset.le_sup (f := fun b : {a : B // IsAtom a} ↦ (b : B))
      (show ⟨a, ha⟩ ∈ atomsBelow x by simp [atomsBelow, h])

/-- Every finitely additive signed charge is the sum of its atom charges. -/
theorem IsSignedCharge.eq_sum_atoms
    {B : Type u} [BooleanAlgebra B] [Fintype B] {μ : B → ℝ}
    (hμ : IsSignedCharge μ) (x : B) :
    μ x = ∑ a ∈ atomsBelow x, μ a := by
  calc
    μ x = μ ((atomsBelow x).sup fun a ↦ (a : B)) := by
      rw [sup_atomsBelow]
    _ = ∑ a ∈ atomsBelow x, μ a := hμ.finset_sup_atoms (atomsBelow x)

/-- Every finitely additive probability is the sum of its atom masses. -/
theorem IsProbability.eq_sum_atoms
    {B : Type u} [BooleanAlgebra B] [Fintype B] {μ : B → ℝ}
    (hμ : IsProbability μ) (x : B) :
    μ x = ∑ a ∈ atomsBelow x, μ a := by
  calc
    μ x = μ ((atomsBelow x).sup fun a ↦ (a : B)) := by
      rw [sup_atomsBelow]
    _ = ∑ a ∈ atomsBelow x, μ a := hμ.finset_sup_atoms (atomsBelow x)

/-- Pointwise equality of sums of event vectors is exactly Scott's
atom-count identity. -/
theorem sum_atomVector_eq_iff {B : Type u} [BooleanAlgebra B]
    (n : ℕ) (x y : Fin n → B) :
```

```
    (∑ i, atomVector (x i)) = ∑ i, atomVector (y i) ↔
      ∀ a : B, IsAtom a →
        (∑ i, if a ≤ x i then 1 else 0) =
          (∑ i, if a ≤ y i then 1 else 0 : ℕ) := by
  constructor
  · intro h a ha
    have haeq := congrFun h ⟨a, ha⟩
    simp only [Finset.sum_apply, atomVector_apply] at haeq
    exact_mod_cast haeq
  · intro h
    funext a
    simp only [Finset.sum_apply, atomVector_apply]
    exact_mod_cast h a a.property

/-- Scott 1964, literal condition `(4_B)`: cancellation is stated using
equality of sums of the characteristic vectors themselves. -/
def ProbVectorCancellation {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop) : Prop :=
  ∀ (n : ℕ) (x y : Fin (n + 1) → B),
    (∑ i, atomVector (x i)) = ∑ i, atomVector (y i) →
      (∀ i, i ≠ 0 → R (x i) (y i)) → R (y 0) (x 0)

/-- The literal vector-sum and atom-count formulations of `(4_B)` agree. -/
theorem probVectorCancellation_iff_probCancellation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} :
    ProbVectorCancellation R ↔ ProbCancellation R := by
  constructor
  · intro h n x y hcount hweak
    exact h n x y ((sum_atomVector_eq_iff (n + 1) x y).2 hcount) hweak
  · intro h n x y hsum hweak
    exact h n x y ((sum_atomVector_eq_iff (n + 1) x y).1 hsum) hweak

end

end Scott1964.MeasurementStructures
```

B.5.3 Finite.lean

Lean 4 source (lines 1–400)

```
import Scott1964.MeasurementStructures.Probability.Atoms
import Scott1964.MeasurementStructures.LinearInequalities.ScottTheorems

/-!
# Finite qualitative probability

This file isolates the last step of Scott's proof: a separating functional on
atom vectors is normalized to a finitely additive probability.
-/

namespace Scott1964.MeasurementStructures

open scoped BigOperators Classical
```

```
noncomputable section

open LinearInequalities

/-- Linear-functional representation of a qualitative relation on event
characteristic vectors. -/
def AtomLinearRepresentation {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop) : Prop :=
  ∃ φ : ({a : B // IsAtom a} → ℝ) →ₗ[ℝ] ℝ,
    ∀ x y, R x y ↔ φ (atomVector x) ≥ φ (atomVector y)

/-- Integration of atom vectors against the atom masses of a finite
probability. -/
def atomWeightFunctional {B : Type u} [BooleanAlgebra B] [Fintype B]
    (μ : B → ℝ) : ({a : B // IsAtom a} → ℝ) →ₗ[ℝ] ℝ where
  toFun v := ∑ a, v a * μ a
  map_add' u v := by
    simp only [Pi.add_apply, add_mul, Finset.sum_add_distrib]
  map_smul' c v := by
    simp only [Pi.smul_apply, smul_eq_mul, RingHom.id_apply]
    symm
    rw [Finset.mul_sum]
    apply Finset.sum_congr rfl
    intro a ha
    ring

@[simp]
theorem atomWeightFunctional_apply
    {B : Type u} [BooleanAlgebra B] [Fintype B] (μ : B → ℝ)
    (v : {a : B // IsAtom a} → ℝ) :
    atomWeightFunctional μ v = ∑ a, v a * μ a :=
  rfl

theorem atomWeightFunctional_atomVector
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {μ : B → ℝ} (hμ : IsProbability μ) (x : B) :
    atomWeightFunctional μ (atomVector x) = μ x := by
  rw [hμ.eq_sum_atoms x]
  simp only [atomWeightFunctional_apply, atomVector_apply, atomsBelow]
  rw [Finset.sum_filter]
  apply Finset.sum_congr rfl
  intro a ha
  by_cases hax : (a : B) ≤ x <;> simp [hax]

/-- Atom-weight integration also recovers an arbitrary signed charge. -/
theorem atomWeightFunctional_atomVector_signed
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {μ : B → ℝ} (hμ : IsSignedCharge μ) (x : B) :
    atomWeightFunctional μ (atomVector x) = μ x := by
  rw [hμ.eq_sum_atoms x]
  simp only [atomWeightFunctional_apply, atomVector_apply, atomsBelow]
  rw [Finset.sum_filter]
  apply Finset.sum_congr rfl
  intro a ha
  by_cases hax : (a : B) ≤ x <;> simp [hax]
```

```
theorem RealizableProbability.atomLinearRepresentation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : RealizableProbability R) :
    AtomLinearRepresentation R := by
  rcases hR with ⟨μ, hμ, hrep⟩
  refine ⟨atomWeightFunctional μ, ?_⟩
  intro x y
  rw [atomWeightFunctional_atomVector hμ,
    atomWeightFunctional_atomVector hμ]
  exact hrep x y

theorem RealizableSignedCharge.atomLinearRepresentation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : RealizableSignedCharge R) :
    AtomLinearRepresentation R := by
  rcases hR with ⟨μ, hμ, hrep⟩
  refine ⟨atomWeightFunctional μ, ?_⟩
  intro x y
  rw [atomWeightFunctional_atomVector_signed hμ,
    atomWeightFunctional_atomVector_signed hμ]
  exact hrep x y

/-- A linear representation on atom vectors induces a signed charge directly.
No division by the charge of `⊤` is needed. -/
theorem signedCharge_of_atomLinearRepresentation
    {B : Type u} [BooleanAlgebra B] {R : B → B → Prop}
    (hR : AtomLinearRepresentation R) :
    RealizableSignedCharge R := by
  rcases hR with ⟨φ, hrep⟩
  let μ : B → ℝ := fun x ↦ φ (atomVector x)
  refine ⟨μ, ?_, ?_⟩
  · refine ⟨?_, ?_⟩
    · simp [μ, atomVector_bot]
    · intro x y hxy
      simp only [μ, atomVector_sup_of_disjoint hxy, map_add]
  · exact hrep

/-- A nondegenerate atom-functional representation normalizes to a finitely
additive probability without changing any comparisons. -/
theorem probability_of_atomLinearRepresentation {B : Type u} [BooleanAlgebra B]
    {R : B → B → Prop}
    (hR : AtomLinearRepresentation R)
    (hnt : ProbNontrivial R) (hn : ProbNonneg R) :
    RealizableProbability R := by
  rcases hR with ⟨φ, hrep⟩
  have hbot : φ (atomVector (⊥ : B)) = 0 := by rw [atomVector_bot, map_zero]
  have htop : φ (atomVector (⊤ : B)) > 0 := by
    have hge := (hrep ⊤ ⊥).1 hnt.1
    have hnle := fun h ↦ hnt.2 ((hrep ⊥ ⊤).2 h)
    rw [hbot] at hge hnle
    exact lt_of_not_ge hnle
  let μ : B → ℝ := fun x ↦ φ (atomVector x) / φ (atomVector (⊤ : B))
  refine ⟨μ, ?_, ?_⟩
  · refine ⟨?_, ?_, ?_, ?_⟩
    · simp [μ, atomVector_bot]
    · simp [μ, ne_of_gt htop]
```

```
    · intro x y hxy
      change φ (atomVector (x ⊔ y)) / φ (atomVector (⊤ : B)) =
        φ (atomVector x) / φ (atomVector (⊤ : B)) +
          φ (atomVector y) / φ (atomVector (⊤ : B))
      rw [atomVector_sup_of_disjoint hxy, map_add, add_div]
    · intro x
      have hx := (hrep x ⊥).1 (hn x)
      rw [hbot] at hx
      exact div_nonneg hx htop.le
  · intro x y
    rw [hrep]
    change φ (atomVector y) ≤ φ (atomVector x) ↔
      φ (atomVector y) / φ (atomVector (⊤ : B)) ≤
        φ (atomVector x) / φ (atomVector (⊤ : B))
    exact (div_le_div_iff_of_pos_right htop).symm

/-- Condition `(2_B)` makes the normalized signed measure nonnegative. -/
theorem nonnegative_probability_of_atomLinearRepresentation
    {B : Type u} [BooleanAlgebra B] {R : B → B → Prop}
    (hR : AtomLinearRepresentation R) (hnt : ProbNontrivial R)
    (hn : ProbNonneg R) :
    ∃ μ : B → ℝ, IsFinitelyAdditiveProbability μ ∧
      ∀ x y, R x y ↔ μ x ≥ μ y :=
  probability_of_atomLinearRepresentation hR hnt hn

/-- Every atom-functional representation satisfies Scott's cancellation
scheme.  This is the easy direction of the finite separation theorem. -/
theorem AtomLinearRepresentation.cancellation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : AtomLinearRepresentation R) :
    ∀ (n : ℕ) (x y : Fin (n + 1) → B),
      (∀ a : B, IsAtom a →
        (∑ i, if a ≤ x i then 1 else 0) =
          (∑ i, if a ≤ y i then 1 else 0 : ℕ)) →
      (∀ i, i ≠ finHead n → R (x i) (y i)) →
        R (y (finHead n)) (x (finHead n)) := by
  rcases hR with ⟨φ, hrep⟩
  intro n x y hcount hweak
  have hvectors :
      (∑ i, atomVector (x i)) = ∑ i, atomVector (y i) :=
    (sum_atomVector_eq_iff (n + 1) x y).2 hcount
  have hsums :
      ∑ i, φ (atomVector (x i)) = ∑ i, φ (atomVector (y i)) := by
    simpa only [map_sum] using congrArg φ hvectors
  have hrest :
      ∑ i ∈ (Finset.univ.erase (finHead n)),
          φ (atomVector (y i)) ≤
        ∑ i ∈ (Finset.univ.erase (finHead n)),
          φ (atomVector (x i)) := by
    apply Finset.sum_le_sum
    intro i hi
    have hi0 : i ≠ finHead n := by simpa using (Finset.mem_erase.mp hi).1
    exact (hrep (x i) (y i)).1 (hweak i hi0)
  apply (hrep (y (finHead n)) (x (finHead n))).2
  have hzero : finHead n ∈ Finset.univ := Finset.mem_univ _
  have hx := Finset.sum_erase_add _ (fun i ↦ φ (atomVector (x i))) hzero
```

```
  have hy := Finset.sum_erase_add _ (fun i ↦ φ (atomVector (y i))) hzero
  linarith

theorem AtomLinearRepresentation.probCancellation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : AtomLinearRepresentation R) :
    ProbCancellation R :=
  hR.cancellation

/-- The atom-count formulation can equivalently be supplied as an equality
of sums of characteristic vectors. -/
theorem ProbCancellation.of_atomVector_sum
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : ProbCancellation R)
    (n : ℕ) (x y : Fin (n + 1) → B)
    (hsum : (∑ i, atomVector (x i)) = ∑ i, atomVector (y i))
    (hweak : ∀ i, i ≠ finHead n → R (x i) (y i)) :
    R (y (finHead n)) (x (finHead n)) :=
  hR n x y ((sum_atomVector_eq_iff (n + 1) x y).1 hsum) hweak

/-- Scott cancellation alone implies reflexivity. -/
theorem ProbCancellation.refl
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : ProbCancellation R) (x : B) :
    R x x := by
  apply hR.of_atomVector_sum 0 (fun _ ↦ x) (fun _ ↦ x)
  · rfl
  · intro i hi
    exact False.elim (hi (by simpa [finHead_eq_zero] using Fin.eq_zero i))

/-- Scott cancellation implies de Finetti transitivity. -/
theorem ProbCancellation.transitive
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : ProbCancellation R) :
    ProbTransitive R := by
  intro x y z hxy hyz
  let xs : Fin 3 → B := ![z, x, y]
  let ys : Fin 3 → B := ![x, y, z]
  apply hR.of_atomVector_sum 2 xs ys
  · simp [xs, ys, Fin.sum_univ_succ]
    abel
  · intro i hi
    fin_cases i
    · exact False.elim (hi (finHead_eq_zero _).symm)
    · exact hxy
    · exact hyz

/-- Scott cancellation gives both directions of de Finetti's invariance
under adjoining a common disjoint event. -/
theorem ProbCancellation.disjointUnionInvariant
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : ProbCancellation R) :
    ProbDisjointUnionInvariant R := by
  intro x y z hzx hzy
  have hxz : atomVector (x ⊔ z) = atomVector x + atomVector z :=
    atomVector_sup_of_disjoint hzx.symm
```

```
  have hyz : atomVector (y ⊔ z) = atomVector y + atomVector z :=
    atomVector_sup_of_disjoint hzy.symm
  constructor
  · intro hxy
    let xs : Fin 2 → B := ![y ⊔ z, x]
    let ys : Fin 2 → B := ![x ⊔ z, y]
    apply hR.of_atomVector_sum 1 xs ys
    · simp [xs, ys, Fin.sum_univ_succ, hxz, hyz]
      abel
    · intro i hi
      fin_cases i
      · exact False.elim (hi (finHead_eq_zero _).symm)
      · exact hxy
  · intro hxy
    let xs : Fin 2 → B := ![y, x ⊔ z]
    let ys : Fin 2 → B := ![x, y ⊔ z]
    apply hR.of_atomVector_sum 1 xs ys
    · simp [xs, ys, Fin.sum_univ_succ, hxz, hyz]
      abel
    · intro i hi
      fin_cases i
      · exact False.elim (hi (finHead_eq_zero _).symm)
      · exact hxy

/-! ## Reduction to Scott's explicit §1 relation theorem -/

/-- The event represented by an atom vector in the range of `atomVector`.
Outside that range its value is immaterial. -/
def eventOfAtomVector {B : Type u} [BooleanAlgebra B]
    (v : {a : B // IsAtom a} → ℝ) : B :=
  if h : ∃ x, atomVector x = v then Classical.choose h else ⊥

@[simp]
theorem eventOfAtomVector_atomVector
    {B : Type u} [BooleanAlgebra B] [Fintype B] (x : B) :
    eventOfAtomVector (atomVector x) = x := by
  unfold eventOfAtomVector
  split
  · rename_i h
    exact atomVector_injective (Classical.choose_spec h)
  · rename_i h
    exact False.elim (h ⟨x, rfl⟩)

/-- The relation transported from events to their characteristic vectors. -/
def AtomVectorRelation {B : Type u} [BooleanAlgebra B]
    (R : B → B → Prop)
    (u v : {a : B // IsAtom a} → ℝ) : Prop :=
  R (eventOfAtomVector u) (eventOfAtomVector v)

@[simp]
theorem atomVectorRelation_atomVectors
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) (x y : B) :
    AtomVectorRelation R (atomVector x) (atomVector y) ↔ R x y := by
  simp [AtomVectorRelation]
```

```
theorem atomVector_range_finite
    {B : Type u} [BooleanAlgebra B] [Fintype B] :
    (Set.range (atomVector :
      B → ({a : B // IsAtom a} → ℝ))).Finite :=
  Set.finite_range _

theorem atomVector_range_rational
    {B : Type u} [BooleanAlgebra B] :
    IsRationalSet (Set.range (atomVector :
      B → ({a : B // IsAtom a} → ℝ))) := by
  rintro v ⟨x, rfl⟩ a
  by_cases h : (a : B) ≤ x
  · exact ⟨1, by simp [atomVector, h]⟩
  · exact ⟨0, by simp [atomVector, h]⟩

theorem atomVectorRelation_complete
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : ProbTotal R) :
    RelationComplete
      (Set.range (atomVector :
        B → ({a : B // IsAtom a} → ℝ)))
      (AtomVectorRelation R) := by
  rintro u ⟨x, rfl⟩ v ⟨y, rfl⟩
  simpa only [atomVectorRelation_atomVectors] using hR x y

theorem atomVectorRelation_sequenceCancellation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : ProbCancellation R) :
    RelationSequenceCancellation
      (Set.range (atomVector :
        B → ({a : B // IsAtom a} → ℝ)))
      (AtomVectorRelation R) := by
  intro n u v hu hv huv hsum j
  let e : Equiv.Perm (Fin (n + 1)) := Equiv.swap (finHead n) j
  let x : Fin (n + 1) → B := fun i ↦ eventOfAtomVector (u (e i))
  let y : Fin (n + 1) → B := fun i ↦ eventOfAtomVector (v (e i))
  have hxu : ∀ i, atomVector (x i) = u (e i) := by
    intro i
    rcases hu (e i) with ⟨xi, hxi⟩
    change atomVector (eventOfAtomVector (u (e i))) = u (e i)
    rw [← hxi]
    simp
  have hyv : ∀ i, atomVector (y i) = v (e i) := by
    intro i
    rcases hv (e i) with ⟨yi, hyi⟩
    change atomVector (eventOfAtomVector (v (e i))) = v (e i)
    rw [← hyi]
    simp
  change R (eventOfAtomVector (v j)) (eventOfAtomVector (u j))
  apply hR.of_atomVector_sum n x y
  · simp only [hxu, hyv]
    rw [Equiv.sum_comp e, Equiv.sum_comp e, hsum]
  · intro i hi
    change R (eventOfAtomVector (u (e i))) (eventOfAtomVector (v (e i)))
    exact huv (e i)
```

```
theorem atomLinearRepresentation_of_relationRealizable
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop}
    (hreal : RelationRealizable
      (Set.range (atomVector :
        B → ({a : B // IsAtom a} → ℝ)))
      (AtomVectorRelation R)) :
    AtomLinearRepresentation R := by
  rcases hreal with ⟨φ, hφ⟩
  refine ⟨φ, ?_⟩
  intro x y
  simpa only [atomVectorRelation_atomVectors] using
    hφ (atomVector x) (Set.mem_range_self x)
      (atomVector y) (Set.mem_range_self y)

/-- Scott's `(3_B)` and `(4_B)` produce a linear representation on atom
vectors, without any positivity or nondegeneracy assumptions. -/
theorem atomLinearRepresentation_of_total_cancellation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop}
    (htotal : ProbTotal R) (hcancel : ProbCancellation R) :
    AtomLinearRepresentation R := by
  have hreal : RelationRealizable
      (Set.range (atomVector :
        B → ({a : B // IsAtom a} → ℝ)))
      (AtomVectorRelation R) :=
    (scott_theorem_1_3 atomVector_range_finite atomVector_range_rational).2
      ⟨atomVectorRelation_complete htotal,
        atomVectorRelation_sequenceCancellation hcancel⟩
  exact atomLinearRepresentation_of_relationRealizable hreal

theorem RealizableProbability.probCancellation
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : RealizableProbability R) :
    ProbCancellation R :=
  hR.atomLinearRepresentation.probCancellation

theorem RealizableProbability.finiteNecessary
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : RealizableProbability R) :
    ProbNontrivial R ∧ ProbNonneg R ∧ ProbTotal R ∧ ProbCancellation R :=
  ⟨hR.probNontrivial, hR.probNonneg, hR.probTotal, hR.probCancellation⟩

theorem RealizableSignedCharge.finiteNecessary
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop} (hR : RealizableSignedCharge R) :
    ProbTotal R ∧ ProbCancellation R := by
  have hlin := hR.atomLinearRepresentation
  constructor
  · rcases hlin with ⟨φ, hrep⟩
```

Lean 4 source (lines 401–463)

```
    intro x y
    exact (le_total (φ (atomVector y)) (φ (atomVector x))).imp
      ((hrep x y).2) ((hrep y x).2)
  · exact hlin.probCancellation
```

```
/-- Scott's p.15 signed-measure claim: `(3_B)` and `(4_B)` are exactly the
conditions for representation by a finitely additive signed charge. -/
theorem scott_p15_signed_charge
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) :
    RealizableSignedCharge R ↔ ProbTotal R ∧ ProbCancellation R := by
  constructor
  · exact RealizableSignedCharge.finiteNecessary
  · rintro ⟨htotal, hcancel⟩
    exact signedCharge_of_atomLinearRepresentation
      (atomLinearRepresentation_of_total_cancellation htotal hcancel)

/-- Scott's p.15 claim with literal vector-sum cancellation `(4_B)`. -/
theorem scott_p15_signed_charge_vector
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) :
    RealizableSignedCharge R ↔ ProbTotal R ∧ ProbVectorCancellation R := by
  rw [scott_p15_signed_charge,
    probVectorCancellation_iff_probCancellation]

theorem probability_of_finite_axioms
    {B : Type u} [BooleanAlgebra B] [Fintype B]
    {R : B → B → Prop}
    (hnt : ProbNontrivial R) (hn : ProbNonneg R)
    (htotal : ProbTotal R) (hcancel : ProbCancellation R) :
    RealizableProbability R := by
  have hreal : RelationRealizable
      (Set.range (atomVector :
        B → ({a : B // IsAtom a} → ℝ)))
      (AtomVectorRelation R) :=
    (scott_theorem_1_3 atomVector_range_finite atomVector_range_rational).2
      ⟨atomVectorRelation_complete htotal,
        atomVectorRelation_sequenceCancellation hcancel⟩
  exact probability_of_atomLinearRepresentation
    (atomLinearRepresentation_of_relationRealizable hreal) hnt hn

/-- **Scott 1964, Theorem 4.1 (subjective probability).** -/
theorem theorem_4_1 {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧ ProbTotal R ∧ ProbCancellation R := by
  constructor
  · exact RealizableProbability.finiteNecessary
  · rintro ⟨hnt, hn, htotal, hcancel⟩
    exact probability_of_finite_axioms hnt hn htotal hcancel

/-- **Scott 1964, Theorem 4.1**, with `(4_B)` stated literally as equality
of sums of atom vectors. -/
theorem theorem_4_1_vector {B : Type u} [BooleanAlgebra B] [Fintype B]
    (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧ ProbTotal R ∧
        ProbVectorCancellation R := by
  rw [theorem_4_1, probVectorCancellation_iff_probCancellation]
```

```
end

end Scott1964.MeasurementStructures
```

B.5.4 KPSCounterexample.lean

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.Probability.Finite

/-!
# The Kraft-Pratt-Seidenberg five-atom counterexample

This file formalizes the simple ordering displayed in Section 4 of
Kraft-Pratt-Seidenberg, *Intuitive Probability on Finite Sets* (1959).
The five atoms `p q r s t` are represented by `0 1 2 3 4 : Fin 5`.

The rank table below is the paper's displayed order, from the empty event
through the full event.  Its four decisive comparisons are

`qs < p`, `pq < rs`, `ps < tq`, and `rt < spq`.

The finite de Finetti properties are checked by evaluating all events.
The non-representability argument itself is symbolic: finite additivity
turns the four strict event comparisons into four incompatible linear
inequalities between singleton weights.
-/

namespace Scott1964.MeasurementStructures.Probability.KPSCounterexample

abbrev Atom := Fin 5
abbrev Event := Finset Atom

def p : Atom := 0
def q : Atom := 1
def r : Atom := 2
def s : Atom := 3
def t : Atom := 4

/-- Binary code for an event, with `p q r s t` as bits `0 1 2 3 4`. -/
def eventCode (A : Event) : Nat :=
  ∑ i ∈ A, 2 ^ i.val

/-- Position in the strict 32-event order printed by KPS. -/
def kpsRank (A : Event) : Nat :=
  match eventCode A with
  | 0  => 0   -- 1 (the empty event)
  | 2  => 1   -- q
  | 4  => 2   -- r
```

```
  | 8  => 3   -- s
  | 6  => 4   -- qr
  | 10 => 5   -- qs
  | 1  => 6   -- p
  | 3  => 7   -- pq
  | 12 => 8   -- rs
  | 16 => 9   -- t
  | 14 => 10  -- qrs
  | 5  => 11  -- rp
  | 9  => 12  -- ps
  | 18 => 13  -- tq
  | 7  => 14  -- qrp
  | 20 => 15  -- rt
  | 11 => 16  -- spq
  | 24 => 17  -- st
  | 13 => 18  -- rsp
  | 22 => 19  -- qrt
  | 26 => 20  -- qst
  | 17 => 21  -- pt
  | 15 => 22  -- qrsp
  | 19 => 23  -- qpt
  | 28 => 24  -- rst
  | 30 => 25  -- qrst
  | 21 => 26  -- rpt
  | 25 => 27  -- spt
  | 23 => 28  -- qrpt
  | 27 => 29  -- qspt
  | 29 => 30  -- rspt
  | 31 => 31  -- pqrst
  | _  => 0   -- unreachable for an event on five atoms

def KPSLe (A B : Event) : Prop := kpsRank A ≤ kpsRank B
def KPSLt (A B : Event) : Prop := kpsRank A < kpsRank B

/-- The KPS order oriented as "at least as probable", matching the shared
qualitative-probability API. -/
def KPSGe (A B : Event) : Prop := KPSLe B A

instance : DecidableRel KPSLe := by
  intro A B
  unfold KPSLe
  infer_instance

instance : DecidableRel KPSLt := by
  intro A B
  unfold KPSLt
  infer_instance

instance : DecidableRel KPSGe := by
  intro A B
  unfold KPSGe
  infer_instance

/-- The table gives distinct ranks to all 32 events. -/
theorem kpsRank_injective : Function.Injective kpsRank := by
  native_decide +revert
```

```
/-- Comparability (C). -/
theorem comparable (A B : Event) : KPSLe A B ∨ KPSLe B A := by
  simp only [KPSLe]
  omega

/-- Transitivity (T). -/
theorem transitive {A B C : Event} :
    KPSLe A B → KPSLe B C → KPSLe A C := by
  simp only [KPSLe]
  omega

/-- The empty event is no more likely than every event. -/
theorem empty_le (A : Event) : KPSLe ∅ A := by
  native_decide +revert

/-- De Finetti's disjoint-union invariance (A), exhaustively verified. -/
theorem disjoint_union_invariance (A B C : Event)
    (hCA : Disjoint C A) (hCB : Disjoint C B) :
    KPSLe A B ↔ KPSLe (A ∪ C) (B ∪ C) := by
  native_decide +revert

theorem qs_lt_p : KPSLt {q, s} {p} := by native_decide
theorem pq_lt_rs : KPSLt {p, q} {r, s} := by native_decide
theorem ps_lt_tq : KPSLt {p, s} {t, q} := by native_decide
theorem rt_lt_spq : KPSLt {r, t} {s, p, q} := by native_decide

/-- Finite additivity determines every event from its singleton values. -/
theorem measure_eq_sum_singletons {μ : Event → ℝ}
    (hμ : IsProbability μ) (A : Event) :
    μ A = ∑ i ∈ A, μ {i} := by
  classical
  induction A using Finset.induction_on with
  | empty =>
      simpa using hμ.bot
  | @insert a A ha ih =>
      calc
        μ (insert a A) = μ ({a} ∪ A) := by rw [Finset.singleton_union]
        _ = μ {a} + μ A := hμ.additive _ _
          (Finset.disjoint_singleton_left.mpr ha)
        _ = μ {a} + ∑ i ∈ A, μ {i} := by rw [ih]
        _ = ∑ i ∈ insert a A, μ {i} := by simp [ha]

/-- Strict numerical representation of the KPS strict order. -/
def StrictlyRepresents (μ : Event → ℝ) : Prop :=
  ∀ A B, KPSLt A B ↔ μ A < μ B

/--
No real finitely additive probability strictly represents the displayed
Kraft-Pratt-Seidenberg order.
-/
theorem no_strictly_representing_probability :
    ¬ ∃ μ : Event → ℝ,
      IsProbability μ ∧ StrictlyRepresents μ := by
  rintro ⟨μ, hμ, hrep⟩
  have h1 : μ {q, s} < μ {p} := (hrep _ _).mp qs_lt_p
```

```
  have h2 : μ {p, q} < μ {r, s} := (hrep _ _).mp pq_lt_rs
  have h3 : μ {p, s} < μ {t, q} := (hrep _ _).mp ps_lt_tq
  have h4 : μ {r, t} < μ {s, p, q} := (hrep _ _).mp rt_lt_spq
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h1
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h2
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h3
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h4
  simp [p, q, r, s, t] at h1 h2 h3 h4
  linarith

theorem kpsGe_nontrivial : ¬KPSGe (⊥ : Event) ⊤ := by
  native_decide

theorem kpsGe_bottom (A : Event) : KPSGe A ⊥ :=
  empty_le A

theorem kpsGe_total : ProbTotal KPSGe := by
  intro A B
  exact (comparable B A).imp id id

theorem kpsGe_transitive : ProbTransitive KPSGe := by
  intro A B C hAB hBC
  exact transitive hBC hAB

theorem kpsGe_disjointUnionInvariant :
    ProbDisjointUnionInvariant KPSGe := by
  intro A B C hCA hCB
  exact disjoint_union_invariance (A := B) (B := A) (C := C) hCB hCA

/-- The exact KPS order satisfies de Finetti's five axioms, with condition
(i) in its original weak form `¬ R ⊥ ⊤`. -/
theorem kpsGe_deFinettiAxioms : DeFinettiAxioms KPSGe where
  nontrivial := kpsGe_nontrivial
  bottom := kpsGe_bottom
  total := kpsGe_total
  transitive := kpsGe_transitive
  disjointUnionInvariant := kpsGe_disjointUnionInvariant

theorem measure_strict_of_kpsLt {μ : Event → ℝ}
    (hrep : ∀ A B, KPSGe A B ↔ μ A ≥ μ B)
    {A B : Event} (hAB : KPSLt A B) :
    μ A < μ B := by
  have hBA : KPSGe B A := by
    exact hAB.le
  have hnAB : ¬KPSGe A B := by
    simp only [KPSGe, KPSLe, KPSLt] at hAB ⊢
    omega
  have hle := (hrep B A).1 hBA
  have hnle : ¬μ A ≥ μ B := fun h ↦ hnAB ((hrep A B).2 h)
  linarith

/-- No shared-API probability weakly represents the exact KPS “at least as
probable” order.  The contradiction uses the paper's four decisive
inequalities. -/
theorem not_realizableProbability_kpsGe :
    ¬RealizableProbability KPSGe := by
```

```
  rintro ⟨μ, hμ, hrep⟩
  have h1 : μ {q, s} < μ {p} :=
    measure_strict_of_kpsLt hrep qs_lt_p
  have h2 : μ {p, q} < μ {r, s} :=
    measure_strict_of_kpsLt hrep pq_lt_rs
  have h3 : μ {p, s} < μ {t, q} :=
    measure_strict_of_kpsLt hrep ps_lt_tq
  have h4 : μ {r, t} < μ {s, p, q} :=
    measure_strict_of_kpsLt hrep rt_lt_spq
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h1
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h2
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h3
  rw [measure_eq_sum_singletons hμ, measure_eq_sum_singletons hμ] at h4
  simp [p, q, r, s, t] at h1 h2 h3 h4
  linarith

/-- De Finetti's five axioms are insufficient for finite additive-probability
representation. -/
theorem deFinetti_axioms_insufficient :
    DeFinettiAxioms KPSGe ∧ ¬RealizableProbability KPSGe :=
  ⟨kpsGe_deFinettiAxioms, not_realizableProbability_kpsGe⟩

end Scott1964.MeasurementStructures.Probability.KPSCounterexample
```

B.5.5 Infinite

B.5.5.1 EventSpace.lean

Lean 4 source (lines 1–400)

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.Probability.Basic
import Mathlib.Order.PrimeSeparator
import Mathlib.Topology.Bornology.BoundedOperation
import Mathlib.Topology.ContinuousMap.Bounded.Basic

/-!
# Universal event spaces for an infinite Boolean algebra

This file supplies two complementary universal realizations of a Boolean algebra.

* `PrimePoint B` and `stoneIndicator` are the usual Stone representation by
  characteristic functions on prime ideals.  This representation is faithful and is
  convenient for stating Scott's finite-sum cancellation condition.
* `ProbabilityPoint B` and `eventVector` evaluate an event at every finitely additive
  probability.  Their real span is a normed order-unit space on which probabilities
  are exactly the point-evaluation positive normalized continuous linear functionals.

The second realization is used for the elementary, choice-free state correspondence
below.  Constructing the analogous functional on the Stone-indicator span requires
```

```
the separate simple-function integration theorem for arbitrary finitely additive
charges.
-/

namespace Scott1964.MeasurementStructures.Probability.Infinite

open scoped BoundedContinuousFunction
open Set

noncomputable section

universe u

variable (B : Type u) [BooleanAlgebra B]

/-! ## The probability-evaluation event space -/

noncomputable instance instBoundedAddReal : BoundedAdd ℝ := inferInstance

/-- A point of the universal probability-evaluation space. -/
def ProbabilityPoint :=
  {μ : B → ℝ // IsFinitelyAdditiveProbability μ}

instance instTopologicalSpaceProbabilityPoint : TopologicalSpace (ProbabilityPoint B)
    := ⊥

theorem probabilityPoint_discrete_eq_bot :
    instTopologicalSpaceProbabilityPoint B = ⊥ :=
  rfl

instance instDiscreteTopologyProbabilityPoint : DiscreteTopology (ProbabilityPoint B)
    where
  eq_bot := probabilityPoint_discrete_eq_bot B

noncomputable instance instAddCommMonoidProbabilityBCF :
    AddCommMonoid (ProbabilityPoint B →ᵇ ℝ) := inferInstance

noncomputable instance instModuleProbabilityBCF :
    Module ℝ (ProbabilityPoint B →ᵇ ℝ) := inferInstance

noncomputable instance instAddCommGroupProbabilityBCF :
    AddCommGroup (ProbabilityPoint B →ᵇ ℝ) := inferInstance

theorem probabilityPoint_nonnegative (p : ProbabilityPoint B) (a : B) :
    0 ≤ p.1 a :=
  p.2.nonnegative a

theorem probabilityPoint_le_one (p : ProbabilityPoint B) (a : B) :
    p.1 a ≤ 1 :=
  p.2.le_one a

theorem eventVector_dist_le_one (a : B) (p q : ProbabilityPoint B) :
    dist (p.1 a) (q.1 a) ≤ 1 := by
  rw [Real.dist_eq, abs_sub_le_iff]
  constructor <;>
    linarith [probabilityPoint_nonnegative B p a,
```

```
      probabilityPoint_nonnegative B q a,
      probabilityPoint_le_one B p a,
      probabilityPoint_le_one B q a]

/-- The universal evaluation vector `μ ↦ μ(a)` of an event. -/
def eventVector (a : B) : ProbabilityPoint B →ᵇ ℝ :=
  BoundedContinuousFunction.mkOfDiscrete (fun p => p.1 a) 1
    (eventVector_dist_le_one B a)

/-- The real normed space generated by universal event-evaluation vectors. -/
def EventSpan : Submodule ℝ (ProbabilityPoint B →ᵇ ℝ) :=
  Submodule.span ℝ (Set.range (eventVector B))

noncomputable instance instNormedAddCommGroupEventSpan :
    NormedAddCommGroup (EventSpan B) := inferInstance

noncomputable instance instNormedSpaceEventSpan :
    NormedSpace ℝ (EventSpan B) := inferInstance

noncomputable instance instLocallyConvexSpaceEventSpan :
    LocallyConvexSpace ℝ (EventSpan B) := inferInstance

/-- The event vector as an element of `EventSpan`. -/
def event (a : B) : EventSpan B :=
  ⟨eventVector B a, Submodule.subset_span (Set.mem_range_self a)⟩

/-! ## Stone points and indicator vectors -/

/-- A Stone point of `B`, presented as a prime (equivalently maximal) ideal. -/
structure PrimePoint where
  ideal : Order.Ideal B
  isPrime : ideal.IsPrime

instance : TopologicalSpace (PrimePoint B) := ⊥

instance : DiscreteTopology (PrimePoint B) := ⟨rfl⟩

/-- The `0`/`1` value of an event at a Stone point. -/
def stoneValue (a : B) (p : PrimePoint B) : ℝ :=
  @ite ℝ (a ∈ p.ideal) (Classical.propDecidable (a ∈ p.ideal)) 0 1

/-- The characteristic vector of an event on the Stone points of `B`.

The convention is that a point lies in the event `a` exactly when its prime ideal
does not contain `a`.
-/
def stoneIndicator (a : B) : PrimePoint B →ᵇ ℝ :=
  by
    classical
    exact BoundedContinuousFunction.mkOfDiscrete
      (stoneValue B a) 1 (by
        intro p q
        simp only [stoneValue, Real.dist_eq]
        split <;> split <;> norm_num)

@[simp]
```

```
theorem stoneIndicator_apply (a : B) (p : PrimePoint B) :
    stoneIndicator B a p = stoneValue B a p :=
  rfl

@[simp]
theorem stoneIndicator_bot : stoneIndicator B (⊥ : B) = 0 := by
  classical
  ext p
  simp [stoneIndicator, stoneValue]

@[simp]
theorem stoneIndicator_top : stoneIndicator B (⊤ : B) = 1 := by
  classical
  ext p
  have hp : (⊤ : B) ∉ p.ideal := p.isPrime.toIsProper.top_notMem
  simp [stoneIndicator, stoneValue, hp]

@[simp]
theorem stoneIndicator_sup_of_disjoint {a b : B} (h : Disjoint a b) :
    stoneIndicator B (a ⊔ b) = stoneIndicator B a + stoneIndicator B b := by
  classical
  ext p
  have hp := p.isPrime
  by_cases ha : a ∈ p.ideal
  · by_cases hb : b ∈ p.ideal
    · simp [stoneIndicator, stoneValue, ha, hb, Order.Ideal.sup_mem ha hb]
    · have hab : a ⊔ b ∉ p.ideal := fun hs => hb (p.ideal.lower le_sup_right hs)
      simp [stoneIndicator, stoneValue, ha, hb, hab]
  · by_cases hb : b ∈ p.ideal
    · have hab : a ⊔ b ∉ p.ideal := fun hs => ha (p.ideal.lower le_sup_left hs)
      simp [stoneIndicator, stoneValue, ha, hb, hab]
    · have hab : a ⊔ b ∉ p.ideal := fun hs => ha (p.ideal.lower le_sup_left hs)
      have hnotboth : ¬(a ∉ p.ideal ∧ b ∉ p.ideal) := by
        intro hn
        have hi : a ⊓ b ∈ p.ideal := by
          rw [h.eq_bot]
          exact p.ideal.bot_mem
        exact (hp.mem_or_mem hi).elim hn.1 hn.2
      exact (hnotboth ⟨ha, hb⟩).elim

/-- Stone indicators separate events, so the Stone representation is faithful. -/
theorem stoneIndicator_injective : Function.Injective (stoneIndicator B) := by
  classical
  intro a b hab
  apply le_antisymm
  · by_contra hle
    have hd : Disjoint
        ((Order.PFilter.principal a : Order.PFilter B) : Set B)
        ((Order.Ideal.principal b : Order.Ideal B) : Set B) := by
      rw [Set.disjoint_left]
      intro x hax hxb
      exact hle ((Order.PFilter.mem_principal.mp hax).trans
        (Order.Ideal.mem_principal.mp hxb))
    obtain ⟨J, hJ, hbJ, hdis⟩ :=
      DistribLattice.prime_ideal_of_disjoint_filter_ideal hd
    let p : PrimePoint B := ⟨J, hJ⟩
```

```
    have hbin : b ∈ J := hbJ Order.Ideal.mem_principal_self
    have hain : a ∉ J := by
      intro haJ
      exact Set.disjoint_left.mp hdis
        (show a ∈ Order.PFilter.principal a by
          rw [Order.PFilter.mem_principal]) haJ
    have := congrArg (fun f : PrimePoint B →ᵇ ℝ => f p) hab
    simp [stoneIndicator, stoneValue, p, hbin, hain] at this
  · by_contra hle
    have hd : Disjoint
        ((Order.PFilter.principal b : Order.PFilter B) : Set B)
        ((Order.Ideal.principal a : Order.Ideal B) : Set B) := by
      rw [Set.disjoint_left]
      intro x hbx hxa
      exact hle ((Order.PFilter.mem_principal.mp hbx).trans
        (Order.Ideal.mem_principal.mp hxa))
    obtain ⟨J, hJ, haJ, hdis⟩ :=
      DistribLattice.prime_ideal_of_disjoint_filter_ideal hd
    let p : PrimePoint B := ⟨J, hJ⟩
    have hain : a ∈ J := haJ Order.Ideal.mem_principal_self
    have hbin : b ∉ J := by
      intro hbJ
      exact Set.disjoint_left.mp hdis
        (show b ∈ Order.PFilter.principal b by
          rw [Order.PFilter.mem_principal]) hbJ
    have := congrArg (fun f : PrimePoint B →ᵇ ℝ => f p) hab
    simp [stoneIndicator, stoneValue, p, hbin, hain] at this

/-- Equality of two finite sums of event indicators. -/
def IndicatorSumEqual {ι : Type*} [Fintype ι] (x y : ι → B) : Prop :=
  ∑ i, stoneIndicator B (x i) = ∑ i, stoneIndicator B (y i)

theorem indicatorSumEqual_iff {ι : Type*} [Fintype ι] (x y : ι → B) :
    IndicatorSumEqual B x y ↔
      ∀ p : PrimePoint B,
        ∑ i, stoneValue B (x i) p =
          ∑ i, stoneValue B (y i) p := by
  classical
  constructor
  · intro h p
    have := congrArg (fun f : PrimePoint B →ᵇ ℝ => f p) h
    simpa [IndicatorSumEqual, stoneIndicator, stoneValue] using this
  · intro h
    apply BoundedContinuousFunction.ext
    intro p
    simpa [IndicatorSumEqual, stoneIndicator, stoneValue] using h p

/-- The two-valued finitely additive probability associated with a Stone point. -/
def primeProbability (p : PrimePoint B) : ProbabilityPoint B := by
  refine ⟨fun a => stoneValue B a p, ?_⟩
  refine
    { bot := by simp [stoneValue]
      top := by
        have hp : (⊤ : B) ∉ p.ideal := p.isPrime.toIsProper.top_notMem
        simp [stoneValue, hp]
      additive := ?_
```

```
      nonnegative := ?_ }
  .
    intro a b h
    have hs := congrArg (fun f : PrimePoint B →ᵇ ℝ => f p)
      (stoneIndicator_sup_of_disjoint B h)
    simpa [stoneIndicator_apply] using hs
  .
    intro a
    simp only [stoneValue]
    split <;> norm_num

@[simp]
theorem primeProbability_apply (p : PrimePoint B) (a : B) :
    (primeProbability B p).1 a = stoneValue B a p :=
  rfl

@[simp]
theorem eventVector_apply (a : B) (p : ProbabilityPoint B) :
    eventVector B a p = p.1 a :=
  rfl

@[simp]
theorem eventVector_bot : eventVector B (⊥ : B) = 0 := by
  ext p
  exact p.2.bot

@[simp]
theorem eventVector_top : eventVector B (⊤ : B) = 1 := by
  ext p
  exact p.2.top

@[simp]
theorem eventVector_sup_of_disjoint {a b : B} (h : Disjoint a b) :
    eventVector B (a ⊔ b) = eventVector B a + eventVector B b := by
  ext p
  exact p.2.additive a b h

/-- Probability-evaluation vectors are also a faithful representation of events. -/
theorem eventVector_injective : Function.Injective (eventVector B) := by
  intro a b hab
  apply stoneIndicator_injective B
  ext p
  have h := congrArg (fun f : ProbabilityPoint B →ᵇ ℝ =>
    f (primeProbability B p)) hab
  exact h

@[simp]
theorem event_apply (a : B) (p : ProbabilityPoint B) :
    (event B a : ProbabilityPoint B →ᵇ ℝ) p = p.1 a :=
  rfl

@[simp]
theorem event_bot : event B (⊥ : B) = 0 := by
  apply Subtype.ext
  exact eventVector_bot B
```

```
@[simp]
theorem event_top_coe :
    ((event B (⊤ : B) : EventSpan B) : ProbabilityPoint B →ᵇ ℝ) = 1 :=
  eventVector_top B

@[simp]
theorem event_sup_of_disjoint {a b : B} (h : Disjoint a b) :
    event B (a ⊔ b) = event B a + event B b := by
  apply Subtype.ext
  exact eventVector_sup_of_disjoint B h

/-- Pointwise positivity for a functional on the event span. -/
def IsPositive (L : EventSpan B →L[ℝ] ℝ) : Prop :=
  ∀ f : EventSpan B,
    (∀ p : ProbabilityPoint B, 0 ≤ (f : ProbabilityPoint B →ᵇ ℝ) p) →
      0 ≤ L f

/-- A positive normalized bounded linear functional on the universal event span. -/
structure PositiveState where
  functional : EventSpan B →L[ℝ] ℝ
  positive : IsPositive B functional
  normalized : functional (event B ⊤) = 1

/-- Evaluation at a finitely additive probability is a positive normalized state. -/
def probabilityToState (p : ProbabilityPoint B) : PositiveState B where
  functional :=
    (BoundedContinuousFunction.evalCLM ℝ p).comp (EventSpan B).subtypeL
  positive := by
    intro f hf
    exact hf p
  normalized := by
    exact p.2.top

@[ext]
theorem PositiveState.ext {S T : PositiveState B}
    (h : S.functional = T.functional) : S = T := by
  cases S
  cases T
  simp_all

/-- A positive normalized state restricts to a finitely additive probability on events
    . -/
def stateToProbability (S : PositiveState B) : ProbabilityPoint B := by
  refine ⟨fun a => S.functional (event B a), ?_⟩
  refine
    { bot := by simp
      top := S.normalized
      additive := ?_
      nonnegative := ?_ }
  .
    intro a b h
    rw [event_sup_of_disjoint B h, map_add]
  .
    intro a
    apply S.positive
    intro p
```

```
    exact p.2.nonnegative a

@[simp]
theorem stateToProbability_apply (S : PositiveState B) (a : B) :
    (stateToProbability B S).1 a = S.functional (event B a) :=
  rfl

@[simp]
theorem stateToProbability_probabilityToState (p : ProbabilityPoint B) :
    stateToProbability B (probabilityToState B p) = p := by
  apply Subtype.ext
  funext a
  rfl

theorem probabilityToState_stateToProbability (S : PositiveState B) :
    probabilityToState B (stateToProbability B S) = S := by
  cases S with
  | mk L hpos hnorm =>
      apply PositiveState.ext
      apply ContinuousLinearMap.ext
      intro f
      change f.1 (stateToProbability B ⟨L, hpos, hnorm⟩) =
        L f
      refine Submodule.span_induction
        (p := fun g hg =>
          g (stateToProbability B ⟨L, hpos, hnorm⟩) =
            L ⟨g, hg⟩) ?_ ?_ ?_ ?_ f.2
      · intro g hg
        rcases hg with ⟨a, rfl⟩
        rfl
      · change 0 = L 0
        simp
      · intro x y hx hy hx' hy'
        change x (stateToProbability B ⟨L, hpos, hnorm⟩) +
            y (stateToProbability B ⟨L, hpos, hnorm⟩) =
          L (⟨x, hx⟩ + ⟨y, hy⟩)
        rw [map_add, hx', hy']
      · intro c x hx hx'
        change c * x (stateToProbability B ⟨L, hpos, hnorm⟩) =
          L (c • ⟨x, hx⟩)
        rw [map_smul, hx']
        simp [smul_eq_mul]
```

Lean 4 source (lines 401–412)

```
/-- Finitely additive probabilities and positive normalized states on `EventSpan`
are canonically equivalent. -/
def probabilityStateEquiv : ProbabilityPoint B ≃ PositiveState B where
  toFun := probabilityToState B
  invFun := stateToProbability B
  left_inv := stateToProbability_probabilityToState B
  right_inv := probabilityToState_stateToProbability B

end

end Scott1964.MeasurementStructures.Probability.Infinite
```

B.5.5.2 HahnBanach.lean

Lean 4 source

```
import Mathlib.Analysis.LocallyConvex.Separation
import Mathlib.Analysis.Convex.Cone.Basic

/-!
# Hahn--Banach separation above a closed cone

This is the analytic step used in the reconstructed infinite version of
Scott's Theorem 4.1.  A closed convex set which is stable under translation
by a closed cone and avoids zero admits a functional which is nonnegative on
the cone and strictly positive on the set.
-/

namespace Scott1964.MeasurementStructures.Probability.Infinite

open Set

noncomputable section

variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [LocallyConvexSpace ℝ E]

/-- `K` is upward closed under translation by the cone `C`. -/
def ConeUpperSet (C : ProperCone ℝ E) (K : Set E) : Prop :=
  ∀ {x c}, x ∈ K → c ∈ C → x + c ∈ K

/-- The Hahn--Banach separation lemma in the exact form needed for Kelley's
countable-cover argument. -/
theorem exists_positive_separator_of_coneUpper
    (C : ProperCone ℝ E) {K : Set E}
    (hKne : K.Nonempty) (hKconv : Convex ℝ K) (hKclosed : IsClosed K)
    (hKzero : (0 : E) ∉ K) (hupper : ConeUpperSet C K) :
    ∃ f : StrongDual ℝ E,
      (∀ c ∈ C, 0 ≤ f c) ∧ ∀ x ∈ K, 0 < f x := by
  obtain ⟨f, u, hfu, hfK⟩ :=
    geometric_hahn_banach_point_closed hKconv hKclosed hKzero
  have hu : 0 < u := by simpa using hfu
  rcases hKne with ⟨x₀, hx₀⟩
  have hcone : ∀ c ∈ C, 0 ≤ f c := by
    intro c hc
    by_contra hneg
    have hfc : f c < 0 := lt_of_not_ge hneg
    have hden : 0 < -f c := neg_pos.mpr hfc
    obtain ⟨n : ℕ, hn⟩ := exists_nat_gt ((f x₀ - u) / (-f c))
    have hn' : f x₀ - u < (n : ℝ) * (-f c) :=
      (div_lt_iff₀ hden).mp hn
    have hnc : (n : ℝ) • c ∈ C :=
      C.smul_mem hc (Nat.cast_nonneg n)
    have hxnc : x₀ + (n : ℝ) • c ∈ K := hupper hx₀ hnc
    have hsep := hfK _ hxnc
    rw [map_add, map_smul, smul_eq_mul] at hsep
    linarith
  refine ⟨f, hcone, ?_⟩
```

```
  intro x hx
  exact hu.trans (hfK x hx)

end

end Scott1964.MeasurementStructures.Probability.Infinite
```

B.5.5.3 Kelley.lean

Lean 4 source

```
import Scott1964.MeasurementStructures.Probability.Infinite.HahnBanach
import Mathlib.Analysis.Normed.Module.Dual

/-!
# A generalized Kelley cover

The countable cover below is the explicit Archimedean hypothesis used in the
reconstructed infinite theorem.  Each layer is a closed convex upper set over
the weak-comparison cone and is bounded away from zero.  Hahn--Banach supplies
one positive separator per nonempty layer.
-/

namespace Scott1964.MeasurementStructures.Probability.Infinite

open Set

noncomputable section

variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [LocallyConvexSpace ℝ E]

/-- Countable Kelley cover of strict-comparison vectors above a closed weak
comparison cone. -/
structure KelleyCover (C : ProperCone ℝ E) (S : Set E) where
  layer : ℕ → Set E
  strictInCone : S ⊆ C
  covers : S ⊆ ⋃ n, layer n
  convex : ∀ n, Convex ℝ (layer n)
  closed : ∀ n, IsClosed (layer n)
  avoidsZero : ∀ n, (0 : E) ∉ layer n
  upper : ∀ n, ConeUpperSet C (layer n)

/-- Hahn--Banach produces a positive separator for every layer, with the zero
functional used harmlessly on empty layers. -/
theorem KelleyCover.exists_separator_family {C : ProperCone ℝ E} {S : Set E}
    (h : KelleyCover C S) :
    ∃ f : ℕ → StrongDual ℝ E,
      (∀ n c, c ∈ C → 0 ≤ f n c) ∧
      (∀ n x, x ∈ h.layer n → 0 < f n x) := by
  classical
  have hex : ∀ n, ∃ f : StrongDual ℝ E,
      (∀ c, c ∈ C → 0 ≤ f c) ∧
      (∀ x, x ∈ h.layer n → 0 < f x) := by
```

```
    intro n
    by_cases hne : (h.layer n).Nonempty
    · obtain ⟨f, hfC, hfK⟩ :=
        exists_positive_separator_of_coneUpper C hne
          (h.convex n) (h.closed n) (h.avoidsZero n) (h.upper n)
      exact ⟨f, hfC, hfK⟩
    · refine ⟨0, by simp, ?_⟩
      intro x hx
      exact (hne ⟨x, hx⟩).elim
  choose f hf using hex
  exact ⟨f, fun n c hc ↦ (hf n).1 c hc, fun n x hx ↦ (hf n).2 x hx⟩

/-- A summable positive combination of the layer separators is nonnegative on
the weak cone and strictly positive on every covered strict vector. -/
theorem KelleyCover.exists_combined_separator
    {C : ProperCone ℝ E} {S : Set E} (h : KelleyCover C S) :
    ∃ F : StrongDual ℝ E,
      (∀ c ∈ C, 0 ≤ F c) ∧ ∀ x ∈ S, 0 < F x := by
  classical
  obtain ⟨f, hfC, hfK⟩ := h.exists_separator_family
  let w : ℕ → ℝ := fun n ↦ (1 / 2 : ℝ) ^ n / (1 + ‖f n‖)
  let g : ℕ → StrongDual ℝ E := fun n ↦ w n • f n
  have hwpos : ∀ n, 0 < w n := by
    intro n
    exact div_pos (pow_pos (by norm_num) _) (by positivity)
  have hgnorm : ∀ n, ‖g n‖ ≤ (1 / 2 : ℝ) ^ n := by
    intro n
    have hb : 0 ≤ (1 / 2 : ℝ) ^ n := (pow_nonneg (by norm_num) _)
    have hd : 0 < 1 + ‖f n‖ := by positivity
    have hr : ‖f n‖ / (1 + ‖f n‖) ≤ 1 :=
      (div_le_one hd).2 (by linarith [norm_nonneg (f n)])
    change ‖w n • f n‖ ≤ (1 / 2 : ℝ) ^ n
    rw [norm_smul, Real.norm_eq_abs, abs_of_pos (hwpos n)]
    calc
      w n * ‖f n‖ =
          (1 / 2 : ℝ) ^ n * (‖f n‖ / (1 + ‖f n‖)) := by
            dsimp [w]
            ring
      _ ≤ (1 / 2 : ℝ) ^ n * 1 := mul_le_mul_of_nonneg_left hr hb
      _ = (1 / 2 : ℝ) ^ n := mul_one _
  have hgeom : Summable (fun n : ℕ ↦ (1 / 2 : ℝ) ^ n) :=
    summable_geometric_of_norm_lt_one (by norm_num)
  have hnormsum : Summable (fun n ↦ ‖g n‖) :=
    Summable.of_nonneg_of_le (fun _ ↦ norm_nonneg _) hgnorm hgeom
  have hgsum : Summable g := Summable.of_norm hnormsum
  let F : StrongDual ℝ E := ∑' n, g n
  refine ⟨F, ?_, ?_⟩
  · intro c hc
    have happly := ((ContinuousLinearMap.apply ℝ ℝ) c).map_tsum hgsum
    change F c = ∑' n, g n c at happly
    rw [show F c = ∑' n, g n c from happly]
    apply tsum_nonneg
    intro n
    change 0 ≤ w n * f n c
    exact mul_nonneg (hwpos n).le (hfC n c hc)
  · intro x hx
```

```
    rcases Set.mem_iUnion.mp (h.covers hx) with ⟨n, hxn⟩
    have heval : Summable (fun k ↦ g k x) := by
      have hm := hgsum.map ((ContinuousLinearMap.apply ℝ ℝ) x)
        ((ContinuousLinearMap.apply ℝ ℝ) x).continuous
      change Summable (fun k ↦ g k x) at hm
      exact hm
    have hterm : 0 < g n x := by
      change 0 < w n * f n x
      exact mul_pos (hwpos n) (hfK n x hxn)
    have hle : g n x ≤ ∑' k, g k x :=
      heval.le_tsum n (fun k _ ↦ by
        change 0 ≤ w k * f k x
        exact mul_nonneg (hwpos k).le (hfC k x (h.strictInCone hx)))
    have happly := ((ContinuousLinearMap.apply ℝ ℝ) x).map_tsum hgsum
    change F x = ∑' k, g k x at happly
    rw [show F x = ∑' k, g k x from happly]
    exact hterm.trans_le hle

end

end Scott1964.MeasurementStructures.Probability.Infinite
```

B.5.5.4 Reconstructed.lean

Lean 4 source

```
import Scott1964.MeasurementStructures.Probability.Infinite.EventSpace
import Scott1964.MeasurementStructures.Probability.Infinite.Kelley

/-!
# A modern reconstruction of the infinite probability theorem

Scott's published 1964 paper states and proves the finite theorem, then only
announces an unstated infinite extension. The result below adapts Kelley's
method and is not presented as Scott's unpublished theorem. Based on the
literature audit in the report, Section 3.5.10, it provides the first
explicit, self-contained realization of Scott's announced infinite extension
in the published literature.

Weak comparisons generate a closed cone in the universal event span, expressed
here by its continuous dual polar.  The additional generalized Kelley condition
gives a countable cover of the strict comparisons by closed convex upper sets.
This adapts the countable-cover/separation method of J. L. Kelley, *Measures on
Boolean Algebras* (1959), but the event-span formulation, condition, and theorem
statement are not attributed to Kelley. The real Hahn--Banach theorem in
`HahnBanach.lean`, through
`KelleyCover.exists_combined_separator`, then supplies one functional which is
nonnegative on every weak comparison and positive on every strict comparison.
-/

namespace Scott1964.MeasurementStructures.Probability.Infinite

open Set
```

```
noncomputable section

universe u

variable {B : Type u} [BooleanAlgebra B]

/-- The signed event-span vector associated with the comparison of `x` to `y`. -/
def comparisonVector (B : Type u) [BooleanAlgebra B] (x y : B) : EventSpan B :=
  event B x - event B y

/-- The closed weak-comparison cone, defined as the intersection of all closed
dual half-spaces which contain the declared weak comparisons. -/
def weakComparisonCone (R : B → B → Prop) : ProperCone ℝ (EventSpan B) :=
  ⋂ (L : StrongDual ℝ (EventSpan B))
    (_hL : ∀ x y, R x y → 0 ≤ L (comparisonVector B x y)),
      (ProperCone.positive ℝ ℝ).comap L

theorem mem_weakComparisonCone_iff {R : B → B → Prop} {v : EventSpan B} :
    v ∈ weakComparisonCone R ↔
      ∀ L : StrongDual ℝ (EventSpan B),
        (∀ x y, R x y → 0 ≤ L (comparisonVector B x y)) →
          0 ≤ L v := by
  simp [weakComparisonCone]

theorem comparisonVector_mem_weakComparisonCone {R : B → B → Prop}
    {x y : B} (hxy : R x y) :
    comparisonVector B x y ∈ weakComparisonCone R := by
  rw [mem_weakComparisonCone_iff]
  intro L hL
  exact hL x y hxy

/-- The set of vectors corresponding to source-strict comparisons
`x ≻ y`, namely `¬ R y x`. -/
def strictComparisonSet (R : B → B → Prop) : Set (EventSpan B) :=
  {v | ∃ x y, StrictlyPreferred R x y ∧ v = comparisonVector B x y}

theorem comparisonVector_mem_strictComparisonSet {R : B → B → Prop}
    {x y : B} (hxy : StrictlyPreferred R x y) :
    comparisonVector B x y ∈ strictComparisonSet R :=
  ⟨x, y, hxy, rfl⟩

/-- Concrete generalized Kelley condition used by this reconstruction: strict
comparison vectors admit a countable Kelley cover above the closed weak cone. -/
def GeneralizedKelleyCondition (R : B → B → Prop) : Prop :=
  Nonempty (KelleyCover (weakComparisonCone R) (strictComparisonSet R))

private theorem exists_reciprocal_le {r : ℝ} (hr : 0 < r) :
    ∃ n : ℕ, (1 : ℝ) / (n + 1) ≤ r := by
  obtain ⟨n : ℕ, hn⟩ := exists_nat_gt (1 / r)
  refine ⟨n, ?_⟩
  have hnpos : (0 : ℝ) < n + 1 := by positivity
  rw [div_le_iff₀ hnpos]
  have hr' : (1 : ℝ) < (n + 1) * r := by
    have := (div_lt_iff₀ hr).mp hn
    nlinarith
  nlinarith
```

```
private theorem realizable_generalizedKelley
    {R : B → B → Prop} (hR : RealizableProbability R) :
    GeneralizedKelleyCondition R := by
  rcases hR with ⟨μ, hμ, hrep⟩
  let p : ProbabilityPoint B := ⟨μ, hμ⟩
  let L : StrongDual ℝ (EventSpan B) :=
    (BoundedContinuousFunction.evalCLM ℝ p).comp (EventSpan B).subtypeL
  let K : ℕ → Set (EventSpan B) :=
    fun n => {v | (1 : ℝ) / (n + 1) ≤ L v}
  refine ⟨{
    layer := K
    strictInCone := ?_
    covers := ?_
    convex := ?_
    closed := ?_
    avoidsZero := ?_
    upper := ?_
  }⟩
  · rintro v ⟨x, y, hstrict, rfl⟩
    apply comparisonVector_mem_weakComparisonCone
    apply (hrep x y).2
    exact le_of_not_gt (fun h => hstrict ((hrep y x).2 h.le))
  · rintro v ⟨x, y, hstrict, rfl⟩
    have hpos : 0 < L (comparisonVector B x y) := by
      change 0 < μ x - μ y
      apply sub_pos.mpr
      exact lt_of_not_ge (fun h => hstrict ((hrep y x).2 h))
    obtain ⟨n, hn⟩ := exists_reciprocal_le hpos
    apply Set.mem_iUnion.mpr
    exact ⟨n, hn⟩
  · intro n
    exact convex_halfSpace_ge L.toLinearMap.isLinear _
  · intro n
    exact isClosed_le continuous_const L.continuous
  · intro n hz
    change (1 : ℝ) / (n + 1) ≤ L 0 at hz
    rw [map_zero] at hz
    have : (0 : ℝ) < 1 / (n + 1) := by positivity
    linarith
  · intro n v c hv hc
    dsimp [K] at hv ⊢
    rw [map_add]
    apply hv.trans
    apply le_add_of_nonneg_right
    rw [mem_weakComparisonCone_iff] at hc
    apply hc L
    intro x y hxy
    change 0 ≤ μ x - μ y
    exact sub_nonneg.mpr ((hrep x y).1 hxy)

private theorem generalizedKelley_realizable
    {R : B → B → Prop}
    (hnt : ProbNontrivial R) (hnn : ProbNonneg R) (htot : ProbTotal R)
    (hK : GeneralizedKelleyCondition R) :
    RealizableProbability R := by
```

```
rcases hK with ⟨cover⟩
obtain ⟨F, hFweak, hFstrict⟩ := cover.exists_combined_separator
have htopStrict : StrictlyPreferred R (⊤ : B) ⊥ := hnt.2
have htopSet :
    comparisonVector B (⊤ : B) ⊥ ∈ strictComparisonSet R :=
  comparisonVector_mem_strictComparisonSet htopStrict
have htopPos : 0 < F (event B (⊤ : B)) := by
  have := hFstrict _ htopSet
  have hvec :
      event B (⊤ : B) - event B (⊥ : B) = event B (⊤ : B) := by
    rw [event_bot]
    exact sub_zero (event B (⊤ : B))
  rwa [comparisonVector, hvec] at this
let μ : B → ℝ := fun a => F (event B a) / F (event B (⊤ : B))
have hμ : IsProbability μ := by
  refine {
    bot := ?_
    top := ?_
    additive := ?_
    nonnegative := ?_
  }
  · simp [μ]
  · exact div_self htopPos.ne'
  · intro x y hxy
    simp only [μ]
    rw [event_sup_of_disjoint B hxy, map_add, add_div]
  · intro x
    have hxcone :
        comparisonVector B x ⊥ ∈ weakComparisonCone R :=
      comparisonVector_mem_weakComparisonCone (hnn x)
    have hxnonneg := hFweak _ hxcone
    have : 0 ≤ F (event B x) := by
      have hvec : event B x - event B (⊥ : B) = event B x := by
        rw [event_bot]
        exact sub_zero (event B x)
      rwa [comparisonVector, hvec] at hxnonneg
    exact div_nonneg this htopPos.le
refine ⟨μ, hμ, ?_⟩
intro x y
constructor
· intro hxy
  have hcone :
      comparisonVector B x y ∈ weakComparisonCone R :=
    comparisonVector_mem_weakComparisonCone hxy
  have hnonneg := hFweak _ hcone
  change F (event B x) / F (event B ⊤) ≥
    F (event B y) / F (event B ⊤)
  exact (div_le_div_iff_of_pos_right htopPos).2 (by
    have hmap := map_sub F (event B x) (event B y)
    change 0 ≤ F (event B x - event B y) at hnonneg
    linarith)
· intro hμxy
  rcases htot x y with hxy | hyx
  · exact hxy
  · by_contra hnxy
    have hstrict : StrictlyPreferred R y x := hnxy
```

```
      have hset :
          comparisonVector B y x ∈ strictComparisonSet R :=
        comparisonVector_mem_strictComparisonSet hstrict
      have hpos := hFstrict _ hset
      have hlt : μ x < μ y := by
        change F (event B x) / F (event B ⊤) <
          F (event B y) / F (event B ⊤)
        rw [div_lt_div_iff_of_pos_right htopPos]
        have hmap := map_sub F (event B y) (event B x)
        change 0 < F (event B y - event B x) at hpos
        linarith
      exact (not_lt_of_ge hμxy) hlt

/-- If-and-only-if characterization of an infinite analogue of Scott's Theorem 4.1,
adapting Kelley's 1959 separation method. It is not an attribution of an
unpublished theorem to Scott. Based on the literature audit in the report,
Section 3.5.10, it provides the first explicit, self-contained realization of
Scott's announced infinite extension in the published literature. -/
theorem reconstructed_infinite_theorem_4_1 (R : B → B → Prop) :
    RealizableProbability R ↔
      ProbNontrivial R ∧ ProbNonneg R ∧ ProbTotal R ∧
        GeneralizedKelleyCondition R := by
  constructor
  · intro hR
    exact ⟨hR.probNontrivial, hR.probNonneg, hR.probTotal,
      realizable_generalizedKelley hR⟩
  · rintro ⟨hnt, hnn, htot, hK⟩
    exact generalizedKelley_realizable hnt hnn htot hK

end

end Scott1964.MeasurementStructures.Probability.Infinite
```

B.6 Basic.lean

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/

import Scott1964.MeasurementStructures.FinHead
import Scott1964.MeasurementStructures.LinearInequalities.Definitions
import Scott1964.MeasurementStructures.LinearInequalities.Separation
import Scott1964.MeasurementStructures.LinearInequalities.Rationalization
import Scott1964.MeasurementStructures.LinearInequalities.Sequences
import Scott1964.MeasurementStructures.LinearInequalities.ScottTheorems
import Scott1964.MeasurementStructures.LinearInequalities.OrderedGroup
import Scott1964.MeasurementStructures.Preference.Direct
import Scott1964.MeasurementStructures.Preference.Cycle
import Scott1964.MeasurementStructures.Preference.Intransitive
import Scott1964.MeasurementStructures.Differences.Pair
```

```
import Scott1964.MeasurementStructures.Differences.Ordered
import Scott1964.MeasurementStructures.Probability.Basic
import Scott1964.MeasurementStructures.Probability.Atoms
import Scott1964.MeasurementStructures.Probability.Finite
import Scott1964.MeasurementStructures.Probability.KPSCounterexample
import Scott1964.MeasurementStructures.Probability.Infinite.EventSpace
import Scott1964.MeasurementStructures.Probability.Infinite.HahnBanach
import Scott1964.MeasurementStructures.Probability.Infinite.Kelley
import Scott1964.MeasurementStructures.Probability.Infinite.Reconstructed

/-!
# Scott 1964 -- Measurement Structures and Linear Inequalities

Primary source: Dana S. Scott, *Measurement Structures and Linear
Inequalities*, Journal of Mathematical Psychology 1 (1964), 233-247.
Working transcription: `sources/ScottMeasurement1964_vision.md`.

The paper applies the general solvability criterion for finite systems of
linear inequalities (Theorems 1.1-1.4) to three measurement problems:
intransitive indifference (Theorem 2.1), ordered differences (Theorems 3.1
and 3.2), and subjective probability (Theorem 4.1).

This module re-exports the complete sorry-free development imported by
`Solution.lean`: Scott's eight published theorems, the finite KPS
counterexample, and the separately labelled modern reconstruction of the
infinite probability theorem.
-/

namespace Scott1964.MeasurementStructures

end Scott1964.MeasurementStructures
```

C Scott 1964 source paper

The following pages reproduce the working source used throughout this report: Dana S. Scott, *Measurement Structures and Linear Inequalities*, Journal of Mathematical Psychology **1** (1964), 233–247, from `sources/ScottMeasurement1964.pdf`. The journal PDF is not licensed under this repository's Apache-2.0 terms; see `NOTICE` and `sources/README.md`.

JOURNAL OF MATHEMATICAL PSYCHOLOGY: 1, 233-247 (1964)

Measurement Structures and Linear Inequalities

DANA SCOTT

Stanford University, Stanford, California

The general mathematical criterion for the solvability of finite systems of linear inequalities is applied to some specific situations from measurement theory. Three examples are treated in detail, and in each case the necessary and sufficient conditions for existence of a suitable real-valued (utility) function on a finite structure are obtained.

In establishing a system of measurement one notices first that a class of objects (or events) has a certain inherent structure. Then the next step is to find a method of assigning (real) numbers to the objects in such a way that the observed structure corresponds exactly to reasonably simple arithmetical relationships involving the assigned numbers. In this paper some theoretical aspects of this subject will be discussed particularly in connection with the following specific problems:

PROBLEM I. *(Intransitive Indifference)*

Let A be a finite set and let P be a binary relation on A. Under what conditions on P will there exist a real function f on A such that

$$xPy \quad \text{if and only if} \quad f(x) \geqslant f(y) + 1,$$

for all $x, y \in A$?

PROBLEM II. *(Ordered Differences)*

Let A be a finite set and let D be a quaternary relation on A. Under what conditions on D will there exist a real function f on A such that

$$xy\,Dzw \quad \text{if and only if} \quad f(x) - f(y) \geqslant f(z) - f(w),$$

for all $x, y, z, w \in A$?

PROBLEM III. *(Subjective Probability)*

Let B be a finite Boolean algebra and let $\succcurlyeq$ be a binary relation on B. Under what conditions on $\succcurlyeq$ will there exist a probability measure μ on B such that

$$x \succcurlyeq y \quad \text{if and only if} \quad \mu(x) \geqslant \mu(y),$$

for all $x, y \in B$.

Problem I was first considered in Luce (1956), where a partial solution was given. The complete solution was given in Scott and Suppes (1958) and the proof, as well as a

comprehensive survey of related problems, is also presented in Suppes and Zinnes (1963). For motivation imagine that the relation xPy means that the object x is *definitely preferred* to the object y. Hence if neither xPy nor yPx hold, then x and y are *indifferent*. Unfortunately human powers of discrimination often lead to cases where indifference is intransitive. The question is then to find numerical assignments in which the boundary between preference and indifference is made explicit: the arithmetic relation $\alpha \geqslant \beta + 1$ is an obvious candidate for performing this service (note that the constant 1 could be replaced by any other convenient positive constant by a change of units.) The solution in Scott and Suppes (1958), though direct, is a quite tedious and clumsy proof exploiting special properties of the particular situation. The method to be presented in Section I and applied in Section II uses very well-known theorems on the existence of solutions of linear inequalities. As a special virtue of the approach we find that analysis of the conditions from the general result specialized to the particular instance leads us to *discover* the required properties of the relation P in an almost mechanical way. This virtue will be illustrated in the discussion of the other problems as well.

Problem III was solved in Kraft, Pratt, and Seidenberg (1959). Those authors suggest the kind of method used here, but carry out a different, more direct approach. Their conditions are a little hard to digest because they write certain formulas *multiplicatively* when an *additive* notation is more suggestive and more natural in application to Boolean algebras. For this reason, and for purely expository reasons, the author decided to include the details in Section IV; the proof, however, is quite short given the material of Section I.

A solution to Problem II has, to the author's best knowledge, not been previously published. (After this paper was prepared for publication, the referee informed the author that E. W. Adams had independently obtained similar results by pursuing the method of Adams and Fagot (1956). More specifically, Adams obtained results including those given below in Theorem 3.1 and 4.1. The details are contained in a dittoed report "Remarks on inexact additive measurement." The author is happy to acknowledge priority to Professor Adams. The purpose of the present paper is, in any case, to show how to apply a general method to many problems of this type.) For motivation the reader is referred to Luce and Tukey (1964) and Suppes and Zinnes (1963).

I. THE GENERAL METHOD

Let L be a finite-dimensional real linear vector space. A subset $X \subseteq L$ is *symmetric* if $X = -X = \{-x : x \in X\}$. A subset $N \subseteq X$ is called *realizable* in X if there is a linear functional φ on L such that for all $x \in X$

$$x \in N \quad \text{if and only if} \quad \varphi(x) \geqslant 0.$$

(Recall that a linear functional is a real-valued, homogeneous, additive function defined on L. Thus in case $N \neq X$ the condition means that there is a half-space H of L separating the sets N and $X \sim N$ so that $X \cap H = N \cap H$. If $N = X$, the trivial 0 functional shows that N is realizable.) We shall employ a more suggestive notation writing $x \succcurlyeq 0$ to mean $x \in N$. Further we write $x \preccurlyeq 0$ to mean $-x \succcurlyeq 0$, and $x \succ 0$ for *not* $x \preccurlyeq 0$. The whole approach can be summed up by a simple, general theorem.

THEOREM 1.1. *Let X be a finite, symmetric subset of L. For a subset $\{x \in X : x \succcurlyeq 0\}$ to be realizable in X it is necessary and sufficient that the conditions*

$$x \succcurlyeq 0 \quad \text{or} \quad x \preccurlyeq 0, \tag{1}$$

$$\sum_{i<n} \lambda_i x_i = 0 \quad \text{implies} \quad x_0 \preccurlyeq 0, \tag{2}$$

hold for all $x \in X$ and all sequences $x_0, \cdots, x_{n-1} \in X$, and all scalars $\lambda_0, \cdots, \lambda_{n-1}$, where $\lambda_i > 0$ and $x_i \succcurlyeq 0$, for $i < n$, and $n > 0$.

PROOF. The necessity of (1) is clear. The necessity of (2) becomes at once clear when it is considered that a realizing functional makes $\Sigma_{i<n} \lambda_i x_i$ nonnegative because $\lambda_i > 0$ and $x_i \succcurlyeq 0$, for $i < n$; and since the vector sum is actually 0, the functional cannot make x_0 (or any other x_i, for that matter) strictly positive.

To prove the sufficiency, assume that the two conditions hold. Let Q be the convex polyhedral cone generated by the set $\{x \in X : x \preccurlyeq 0\}$ and let P be the convex polyhedron generated by (the convex closure of) the set $\{x \in X : x \succ 0\}$. We can assume P is nonempty, since otherwise $x \preccurlyeq 0$ holds for all $x \in X$; and therefore $x \succcurlyeq 0$ would hold for all $x \in X$, because X is symmetric. If we can show that P and Q are *disjoint*, it follows at once (see Theorem 2, p. 50 of the book Kuhn and Tucker 1956, for example) that there is a linear functional φ on L such that for all $x \in L$,

$$x \in P \quad \text{implies} \quad \varphi(x) > 0$$

$$x \in Q \quad \text{implies} \quad \varphi(x) \leqslant 0.$$

Thus if $x \in X$ and $x \succcurlyeq 0$, then $-x \preccurlyeq 0$, $-x \in Q$, $\varphi(-x) \leqslant 0$, and $\varphi(x) \geqslant 0$. If $\varphi(x) \geqslant 0$, then $\varphi(-x) \not> 0$, so $-x \notin P$, and so $-x \preccurlyeq 0$ and $x \succcurlyeq 0$. Hence, φ is the required functional.

Let us then suppose that there is a vector $z \in P \cap Q$. Now

$$z = \sum_{i<l} \lambda_i x_i = \sum_{i<m} \lambda_i' x_i',$$

where $x_i \succ 0$, $\lambda_i \geqslant 0$, for $i > l$, and $\Sigma_{i<l}\lambda_i = 1$, and where $x_i' \preccurlyeq 0$, $\lambda_i \geqslant 0$, for $i < m$. By condition (1), $x_i \geqslant 0$ holds for $i < l$. Thus

$$\sum_{i<l} \lambda_i x_i + \sum_{i<m} \lambda_i'(-x_i') = 0.$$

We can assume that all the scalars are strictly positive, and because $\Sigma_{i<l}\lambda_i = 1$, we know $l > 0$. Since $-x_i' \geqslant 0$ for $i < m$, we can apply (2) to conclude that $x_0 \preccurlyeq 0$. But this contradicts the assumption that $x_0 \succ 0$, and the proof is complete.

There is a case where condition (2) of Theorem 1.1 can be simplified. Let S be a finite set and let $L = L(S)$ be the vector space of all real-valued functions defined on S (the ordinary S-dimensional vector space). A *vector* (function) in L is called *rational* if all its coordinates (values) are rational numbers. A *set* of rational vectors is also called rational. With these conventions Theorem 1.1 becomes more combinatorial.

THEOREM 1.2. *Let X be a finite, rational, symmetric subset of L. For a subset $\{x \in X : x \geqslant 0\}$ to be realizable it is necessary and sufficient that the conditions*

$$x \geqslant 0 \quad \text{or} \quad x \preccurlyeq 0, \tag{3}$$

$$\sum_{i<n} x_i = 0 \quad \text{implies} \quad x_0 \preccurlyeq 0, \tag{4}$$

hold for all $x \in X$ and all sequences $x_0, \cdots, x_{n-1} \in X$, where $x_i \geqslant 0$ for $i < n$, and $n > 0$.

PROOF. Suppose that $\Sigma_{i<n}\lambda_i x_i = 0$, where $n > 0$, and $\lambda_i > 0$, $x_i \geqslant 0$ for $i < n$. In other words, the λ_i are positive solutions to a system of homogeneous linear equations with *rational* coefficients (the coordinates of the x_i). Hence there must also be a *rational* set of positive λ_i satisfying the equation. By clearing fractions and replacing integral multiples by repetitions, we can apply (4) to conclude that $x_0 \preccurlyeq 0$. Thus condition (2) is verified and the result follows by Theorem 1.1.

Suppose next that Y is any subset of L. Suppose further that $\geqslant$ is a *binary* relation on Y. We shall write $x \preccurlyeq y$ for $y \geqslant x$. We shall say that $\geqslant$ is *realizable* if there exists a linear functional φ on L such that for all $x, y \in Y$ we have

$$x \geqslant y \quad \text{if and only if} \quad \varphi(x) \geqslant \varphi(y).$$

Conditions for realizability are easily deduced from Theorem 1.2.

THEOREM 1.3. *Let Y be a finite rational subset of L. For a binary relation $\geqslant$ on Y to be realizable it is necessary and sufficient that the conditions*

$$x \geqslant y \quad \text{or} \quad x \preccurlyeq y, \tag{5}$$

$$\sum_{i<n} x_i = \sum_{i<n} y_i \quad \text{implies} \quad x_0 \preccurlyeq y_0, \tag{6}$$

hold for all $x, y \in Y$ *and all sequences* $x_0, \cdots, x_{n-1}, y_0, \cdots, y_{n-1} \in Y$, *where* $x_i \succcurlyeq y_i$ *for* $i < n$ *and* $n > 0$.

PROOF. Assume (5) and (6). Let $X = Y - Y = \{x - y : x, y \in Y\}$. Clearly X is finite, rational, and symmetric. Define $x - y \succcurlyeq 0$ to mean that $x \succcurlyeq y$. This is justified because if $x - y = x' - y'$, x, y, x', y', then by (6), $x \succcurlyeq y$ if and only if $x' \succcurlyeq y'$. The desired result is now a direct consequence of Theorem 1.2.

Let Z be a subset of L which is closed under addition ($Z + Z \subseteq Z$). A binary relation $\succcurlyeq$ on Z will be called *strictly monotonic* if the following three conditions are satisfied:

(i) $x \succcurlyeq y$ or $x \preccurlyeq y$,
(ii) $x_0 \succcurlyeq y_0$ and $x_1 \succcurlyeq y_1$ imply $x_0 + x_1 \succcurlyeq y_0 + y_1$,
(iii) $x_0 + x_1 = y_0 + y_1$ and $x_1 \succcurlyeq y_1$ imply $x_0 \preccurlyeq y_0$,

for all $x, y, x_0, y_0, x_1, y_1 \in Z$. For a subset $Y \subseteq L$, we let Y^+, the additive closure of Y, denote the least set $Z \supseteq Y$ closed under addition. Using this terminology, we can restate Theorem 1.3.

THEOREM 1.4. *Let* Y *be a finite rational subset of* L. *For a binary relation* $\succcurlyeq$ *on* Y *to be realizable it is necessary and sufficient that it be extendable to a strictly monotonic relation on* Y^+.

PROOF. Suppose $\succcurlyeq$ on Y is realizable. Let φ be a functional that realizes $\succcurlyeq$. Define $\succcurlyeq$ on Y^+ by the condition

$$x \succcurlyeq y \qquad \text{if and only if} \qquad \varphi(x) \geqslant \varphi(y)$$

for all $x, y \in Y^+$. Then the new $\succcurlyeq$ is obviously an extension of the old $\succcurlyeq$ on Y, and the linearity of φ makes it trivial to verify (i), (ii), and (iii). Conversely, if we have the extension $\succcurlyeq$ to Y^+, we need only verify condition (6). Thus suppose that $x_i, y_i \in Y$, $x_i \succcurlyeq y_i$ for $i < n$, $n > 0$ and that $\Sigma_{i<n} x_i = \Sigma_{i<n} y_i$. If $n = 1$, then $x_0 = y_0$ and so $x_0 \preccurlyeq y_0$. If $n > 1$, then $\Sigma_{0<i<n} x_i \succcurlyeq \Sigma_{0<i<n} y_i$ in view of (ii). But

$$x_0 + \sum_{0<1<n} x_i = y_0 + \sum_{0<1<n} y_i,$$

so $x_0 \preccurlyeq y_0$ by virtue of (iii).

One should not suppose that a strictly monotonic relation on Y^+ is realizable *on* Y^+. We have only shown that the *restriction* of $\succcurlyeq$ is realizable on the *finite* set Y. (Example: Let $L = E^2$, the two dimensional space, and let $Y = \{(1, 0), (0, 1)\}$. Let $\succcurlyeq$ be defined on Y^+ by the stipulation

$$(n_0, m_0) \succcurlyeq (n_1, m_1) \qquad \text{if and only if} \qquad n_0 > n_1 \quad \text{or} \quad n_0 = n_1 \quad \text{and} \quad m_0 \geqslant m_1.$$

This is a non-Archimedean ordering of Y^+.)

The content of Theorem 1.4 amounts to this well-known fact from algebra: *Every finite subset of an ordered abelian group can be isomorphically embedded in the additive group of reals.* Proof: let G be the abelian group with $\oplus$ as the group operation and $\oslash$ as the ordering. Let $S = \{g_0, \cdots, g_{m-1}\}$ be the finite subset of G. For each $i < m$ let U_i be the characteristic function of the subset $\{g_i\}$ of S. So $U_i \in L = L(S)$, and $Y = \{U_0, \cdots, U_{m-1}\}$ is rational. Further Y^+ has only *integer-valued* functions as elements. Define $\succcurlyeq$ on Y^+ by the condition that for $x, y \in Y^+$

$$x \succcurlyeq y \qquad \text{if and only if}$$

$$x(g_0) \cdot g_0 \oplus \cdots \oplus x(g_{m-1}) \cdot g_{m-1} \oslash y(g_0) \cdot g_0 \oplus \cdots \oplus y(g_{m-1}) \cdot g_{m-1}.$$

It is obvious from the axioms for ordered abelian groups that $\succcurlyeq$ is a strictly monotonic relation on Y^+. Let φ be some linear functional which realizes $\succcurlyeq$ on Y. The function f on S where

$$f(g_i) = \varphi(U_i)$$

is well-defined and, by construction, preserves the addition and order relations *within* the finite set S.

This trick of passing from an abstract structure to the "more concrete" vectors in a finite-dimensional linear space will now be consistently exploited for the solution of the three problems from measurement theory.

II. SOLUTION TO THE PROBLEM OF INTRANSITIVE INDIFFERENCE

Let A be a finite nonempty set and let P be a binary relation on A. For the purposes of this section we shall call P *realizable* if there is a real-valued function f on A such that

$$xPy \qquad \text{if and only if} \qquad f(x) \geqslant f(y) + 1$$

for all $x, y \in A$. Realizable relations are obviously irreflexive, and so we shall assume P is irreflexive.

Let e be an element not in the finite set A and set $S = A \cup \{e\}$. Each element $x \in S$ determines a vector in $L = L(S)$: namely, the characteristic function of $\{x\}$. To avoid clumsy notation, we shall identify each $x \in S$ with its corresponding vector and pretend that $S \subseteq L$. Thus S becomes an *independent basis* for the S-dimensional space L.

We let X be the subset of L containing just the vectors of the forms $x - y - e$ or $y + e - x$, where $x, y \in A$. We define $\succcurlyeq 0$ by these stipulations:

(i) $x - y - e \succcurlyeq 0$ if and only if xPy;
(ii) $y + e - x \succcurlyeq 0$ if and only if not xPy;

for all $x, y \in A$. This is justified because P is irreflexive. Obviously X is a finite, rational, symmetric set. Further $u \succcurlyeq 0$ or $-u \succcurlyeq 0$ holds for all $u \in X$. We are now going to find necessary conditions on P that will verify the conditions of Theorem 1.2 to make $\succcurlyeq 0$ realizable on X. Using such a realization we will then be able to show that the conditions are sufficient to make P realizable in the sense of the present section.

Since P is irreflexive, we see that this means that $e \succcurlyeq 0$ holds while $-e \succcurlyeq 0$ does not hold, in other words $e \succ 0$. Hence, assuming that $\succcurlyeq 0$ is realizable on X, we would have a linear functional φ realizing $\succcurlyeq 0$ such that $\varphi(e) > 0$. Multiplying by a positive scalar, we can assume that $\varphi(e) = 1$. It follows at once that

$$xPy \qquad \text{if and only if} \qquad \varphi(x) \geqslant \varphi(y) + 1,$$

for $x, y \in A$. Hence φ restricted to A gives the realization of P. To complete the picture, we need now only find further necessary conditions on P that will imply condition (2) of 1.2 for $\succcurlyeq 0$ as defined on X.

Suppose, therefore, that we have sequences $x_0, \cdots, x_{k-1}, y_0, \cdots, y_{k-1}, x'_0, \cdots, x'_{m-1}$, $y'_0, \cdots, y'_{m-1} \in A$ where $x_i P y_i$ holds for $i < k$, but *not* $x'_i P y'_i$ holds for $i < m$. We assume $k + m = n > 0$. If we can establish that in this situation the equation

$$\sum_{i<k} (x_i - y_i - e) + \sum_{i<m} (y'_i + e - x'_i) = 0$$

is ***never valid***, then we will have indeed verified what amounts to condition (4) of Theorem 1.2.

Thus, by way of contradiction, let us assume that this last equation does hold for the indicated system of elements of A. We can rewrite the equation as

$$\sum_{i<k} (x_i - y_i) + \sum_{i<m} (y'_i - x'_i) + (m - k) \cdot e = 0.$$

Now the vector $e \in L$ is known to be ***independent*** of vectors in A, thus $m - k = 0$ and $m = k > 0$. In the equation

$$\sum_{i<k} (x_i - y_i) + \sum_{i<m} (y'_i - x'_i) = 0$$

only vectors from A appear, and A is an independent set of vectors. Unfortunately in the equation vectors can occur several times with both positive and negative coefficients. The best we can say in view of the independence of A is that the two arrays of vectors

$$(x_0, \cdots, x_{k-1}, y'_0, \cdots, y'_{m-1})$$

and

$$(y_0, \cdots, y_{k-1}, x'_0, \cdots, x'_{m-1})$$

are alike in that one is a *permutation* of the other. Let us make a tour through these arrays following out the permutation.

Start with y_0 in the lower array. Since P is irreflexive, $x_0 \neq y_0$, so y_0 coincides with some other element in the upper array. Take the element in the lower array *directly under* the element in the upper array just considered. It must coincide with *some other* element in the upper array. Proceeding in this manner, passing from an element in the lower array to an equal element in the upper and then dropping to the corresponding element of the lower directly below this upper element, we shall eventually come back to our starting element y_0. We shall have traced out a *cycle*. There are two cases.

CASE 1. *The cycle never leaves the unprimed elements.* By a simple permutation of subscripts we can assume that we have an $l < k$, $l > 0$, such that

$$y_0 = x_1, y_1 = x_2, \cdots, y_{l-1} = x_l, y_l = x_0,$$

and that $x_i P y_i$ holds for all $i < k$, thus we have a cycle in P-relationships:

$$x_0 P x_1 P x_2 \cdots x_l P x_0.$$

This can easily be ruled out by assuming that P is *transitive*, so that $x_0 P x_0$ would follow contradicting the irreflexivity of P. If P is realizable, then P is necessarily transitive, because $\alpha \geqslant \beta + 1$ is obviously a transitive relation between real numbers.

CASE 2. *The cycle passes through primed and unprimed elements.* If the cycle starting with y_0 has *more* primed than unprimed elements, forget it. It cannot contain all the unprimed elements, because $k = m$. Start with an unused element y_i. *Its* cycle we may assume has both primed and unprimed elements, otherwise we are back in Case 1. Again, if there are more primed elements, start a new cycle. By this argument we may be sure that the cycle we obtain finally has *at least as many* unprimed elements as primed. Let us use the notation yQx as shorthand for *not* xPy. This time we will get a cycle of relationships which, after a change of subscripts, we can assume to be of the form

$$x_0 P x_1 \cdots x_l P y_0' Q y_1' \cdots y_j' Q x_{l+1} \cdots x_0.$$

The number of P's is the same as the number of x's and the Q's as the y's; hence, there are at least as many P's as Q's. A degenerate case is $x_0 P y_0' Q x_0$, which is the only case where there is only one P. This case is impossible because $y_0' Q x_0$ *means* that *not* $x_0 P y_0'$. Thus we may assume there are at least two P's.

Suppose there are two P's adjacent in the cycle. The situation looks like

$$zPxPyQw$$

or like

$$wQzPxPy,$$

(where the letters x, y, z, w now no longer refer to the arrays from which the cycle was obtained.) Assuming that P is realizable by a function f we would have in the first situation

$$f(z) \geqslant f(x) + 1, \qquad f(x) \geqslant f(y) + 1, \qquad \text{and} \qquad f(y) + 1 > f(w).$$

Therefore $f(z) \geqslant f(w) + 1$, and zPw would follow. In the second situation by a similar argument wPy would follow. These *two* transitivity conditions are equivalent and are equivalent to the following implication

$$zPxPy \quad \text{implies} \quad wPy \quad \text{or} \quad zPw,$$

for $x, y, z, w \in A$. If we assume that P has this necessary property, then our cycle of P, Q-relationships can be reduced to a *shorter* cycle where there are still as many P's as Q's.

Applying the above reasoning, we shall reduce the cycle either to one P and one Q (which is impossible) or to a cycle where there are no adjacent P's. Since there are as many P's as Q's, there can be no pair of adjacent Q's either. Thus a part of the cycle must look like

$$xPyQzPw.$$

Assuming that f realizes P on A we have

$$f(x) \geqslant f(y) + 1 > f(z) \geqslant f(w) + 1,$$

whence $f(x) \geqslant f(w) + 1$ and so xPw. This means that if we assume about P the necessary implication that

$$xPy \quad \text{and} \quad zPw \quad \text{implies} \quad xPw \quad \text{or} \quad zPy,$$

for $x, y, z, w \in A$, then our cycle can be reduced step by step to xPx which is already assumed impossible. This completes the impossibility argument.

Notice in this last implication that if we substitute y for z and eliminate the false yPy, then P is found to satisfy the transitive law. We can thus summarize this lengthy discussion as a theorem.

THEOREM 2.1. *Let A be a finite nonempty set and let P be a binary relation on A. For P to be realizable it is necessary and sufficient that the conditions*

(1_P) *not* xPx,
(2_P) xPy *and* zPw *imply* xPw *or* zPy,
(3_P) xPy *and* zPx *imply* wPy *or* zPw,

hold for all $x, y, z, w \in A$.

It will be noted that in our argument we did not make full use of some *strict* inequalities that turned up. The reason is that we did not require the more definite information; indeed, the method gives a realization f of P where for all $x, y \in A$,

$$f(x) \geqslant f(y) + 1 \qquad \text{if and only if} \qquad f(x) > f(y) + 1$$

holds for all $x, y \in A$.

The author does not claim that the proof of Theorem 2.1 given here is particularly attractive. But he does feel that the relentless application of Theorem 1.2 gave the required solution without having to know the answer in advance—that by elementary combinatorial analysis the result was systematically uncovered.

III. SOLUTION TO THE PROBLEM OF ORDERED DIFFERENCES

Before presenting the solution to Problem II, we shall treat a related, but simpler, question which also has independent interest.

Let A and A^* be two finite nonempty sets, and let V be a quaternary relation which we treat as a binary relation on the cartesian product $A \times A^*$. Think of A and A^* as sets of two different kinds of commodities, and interpret $x\, x^*\, V y\, y^*$, where $x, y \in A$, $x^*, y^* \in A^*$, to mean that the combination of x and x^* is *more valuable than* the combination of y and y^*. The question about these structures is whether there exist real-valued functions f and f^* on A and A^*, respectively, that realize V in the sense that

$$x\, x^*\, V y\, y^* \qquad \text{if and only if} \qquad f(x) + f^*(x) \geqslant f^*(y) + f^*(y^*)$$

for all $x, y \in A$, $x^*, y^* \in A^*$. Such a pair f, f^* of functions is called a pair of *utility* functions for V. Thinking of f and f^* as assigning values to the commodities, the interpretation of V as an ordering of the combinations is clear. Of course, we are treating the situation with the philosophy that "the whole is equal to the sum of its parts"; hence, a straightforward adding of utilities in finding the utility of a combination is assumed to be subtle enough for the present theory.

We shall now apply our general Theorem 1.3 to characterize the relations V realizable in the above sense. To this end let $S = A \cup A^*$ be the finite set determining the linear space $L = L(S)$. As in Section II we identify S with the set of basis vectors for L in the obvious way. We let

$$Y = \{x + x^* : x \in A, x^* \in A^*\}.$$

Without loss of generality we can assume that the sets A and A^* are *disjoint*, so that Y is in a one-one correspondence with the set $A \times A^*$. We define $\geqslant$ on Y in the inescapable way for $x, y \in A$, $x, x^* \in A^*$:

$$x + x^* \geqslant y + y^* \qquad \text{if and only if} \qquad x\, x^*\, V y\, y^*.$$

It is immediate that the binary relation $\succcurlyeq$ is realizable on the finite rational set Y (in the sense of Section I) if and only if V is realizable on A by a pair of utility functions.

Considering Theorem 1.3, we need now only transcribe conditions (5) and (6) into equivalent conditions about V. There is no problem about (5): we must assume the relation V has the property that $x\, x^*\, Vy\, y^*$ or $y\, y^*\, Vx\, x^*$ holds for all $x, y \in A$, $x^*, y^* \in A^*$.

For property (6), assume we have elements satisfying the equation

$$\sum_{i<n} (x_i + x_i^*) = \sum_{i<n} (y_i + y_i^*)$$

where $x_i y_i \in A$, $x_i^*, y_i^* \in A^*$ and $x_i x_i^*\, Vy_i\, y_i^*$ holds for $i < n$ and $n > 0$. We require conditions on V so that $y_0\, y_0^*\, Vx_0\, x_0^*$ follows from these assumptions.

Now in the linear space L, remember that the vectors in $S = A \cup A^*$ are independent of each other. Hence, for the above equation to be valid, we must have

$$\sum_{i<n} x_i = \sum_{i<n} y_i \quad \text{and} \quad \sum_{i<n} x_i^* = \sum_{i<n} y_i^*,$$

because the sets A and A^* are assumed disjoint. In other words, there must exist a pair π, σ of permutations of the indices $\{0, 1, \cdots, n-1\}$ such that the equations

$$y_i = x_{\pi(i)} \quad \text{and} \quad y_i^* = x_{\sigma(i)}^*$$

hold for $i < n$. Therefore, the required assumption about V states that $x_i\, x_i^*\, V\, x_{\pi(i)}\, x_{\sigma(i)}$; for $i < n$, implies $x_{\pi(0)}\, x_{\sigma(0)}^*\, V\, x_0\, x_0^*$ where $n > 0$, π and σ are permutations of $\{0, 1, \cdots, n-1\}$, and $x_i \in A$, $x_i^* \in A^*$ for $i < n$. (Actually the hypothesis of this implication is too strong. In view of the other assumption on V, we need only take $i > 0$ in the hypothesis of the second assumption.) It should be clear to everyone that these two conditions are necessary for V to be realizable, and that 1.3 shows conversely the sufficiency. We can therefore state the desired theorem.

THEOREM 3.1. *Let A and A^* be two finite nonempty sets and let V be a binary relation on $A \times A^*$. For V to be realizable by a pair of utility functions on A and A^* it is necessary and sufficient that the conditions*

(1_V) $x\, x^*\, Vy\, y^*$ *or* $y\, y^*\, V\, x\, x^*$,

(2_V) $x_i\, x_i^*\, V\, x_{\pi(i)}\, x_{\sigma(i)}^*$ *for* $i < n, i > 0$, *implies* $x_{\pi(0)}\, x_{\sigma(0)}^*\, V\, x_0\, x_0^*$,

hold for all $x, y \in A$, $x^*, y^* \in A^*$ and all sequences $x_0, \cdots, x_{n-1} \in A$, $x_0^*, \cdots, x_{n-1}^* \in A^*$ and all permutations π, σ of $\{0, \cdots, n-1\}$, where $n > 0$.

We do not need to assume that A and A^* are disjoint. That assumption was only a technical device in the proof to obtain *two* utility functions from *one* linear functional on $L = L(S) = L(A \cup A^*)$.

It is mildly amusing to note how condition (2_V) implies that V is transitive. Suppose $x\,x^*\,V\,y\,y^*$ and $y\,y^*\,V\,z\,z^*$ both hold. Let $x_0 = z$, $x_1 = x$, $x_2 = y$, and $x_0^* = z^*$, $x_1^* = x^*$, $x_2^* = y^*$. Let $\pi(i) = \sigma(i) = i + 1 \pmod 3$. Then by (2_V) we see that $x_1\,x_1^*\,V\,x_0\,x_0^*$ holds; that is, $x\,x^*\,V\,z\,z^*$ holds as required.

Turning now to the ordered differences, we have in this case only *one* finite non-empty set A and a quaternary relation D on A (that is, a binary relation on $A \times A$). The problem is to characterize those D realizable by a single utility function f on A such that

$$xyDzw \qquad \text{if and only if} \qquad f(x) - f(y) \geqslant f(z) - f(w)$$

for all $x, y, z, w \in A$.

Note that if we define a relation V in terms of D by the condition

$$xwVzy \qquad \text{if and only if} \qquad xyDzw$$

for all $x, y, z, w \in A$, then D is also definable in terms of V. Further, if D is realizable in the sense just defined, then V is realizable in the sense of Theorem 3.1 where $A = A^*$ and the two utility functions coincide. Suppose we assume (1_V) and (2_V) of Theorem 3.1, which can be equivalently rewritten in terms of D. (By the remark just made, these are necessary conditions for D to be realizable.) In view of Theorem 3.1 there must exist a *pair* of utility functions on A, which we shall call g and h, such that

$$xyDzw \qquad \text{if and only if} \qquad g(x) - h(y) \geqslant g(z) - h(w)$$

for all $x, y, z, w \in A$. Unfortunately, g and h need not be the same function, so the problem is not yet solved.

To complete the solution, we note that not all necessary conditions for D to be realizable by a single utility function have yet been appreciated. In particular, there is an obvious *commutativity* condition that D must satisfy to be realizable:

$$xyDzw \qquad \text{implies} \qquad wzDyx$$

for all $x, y, zw \in A$. Assuming this property we see that if $xyDzw$ holds, then

$$g(x) - h(y) \geqslant g(z) - h(w)$$

and

$$g(w) - h(z) \geqslant g(y) - h(x).$$

Therefore,

$$h(x) - g(y) \geqslant h(z) - g(w),$$

and so, by adding inequalities,

$$[g(x) + h(x)] - [g(y) + h(y)] \geqslant [g(z) + h(z)] - [g(w) + h(w)].$$

Similarly, if $xyDzw$ does *not* hold, then neither does $wzDyx$. Whence, we find that

$$g(x) - h(y) < g(z) - h(w),$$

and

$$g(w) - h(z) < g(y) - h(x),$$

and so

$$[g(x) + h(x)] - [g(y) + h(y)] < [g(z) + h(z)] - [g(w) + h(w)].$$

In other words, define the function f on A so that $f(x) = g(x) + h(x)$ for all $x \in A$, then f realizes D in the required sense. This discussion leads directly to our main result.

THEOREM 3.2. *Let A be a finite nonempty set and let D be a quaternary relation on A. For D to be realizable by a single utility function on A it is necessary and sufficient that the conditions*

(1_D) $xyDzw$ *or* $zwDxy$,
(2_D) $x_iy_iDx_{\pi(i)}y_{\sigma(i)}$ *for* $i < n$, $i > 0$, *implies* $x_{\pi(0)}y_{\sigma(0)}Dx_0y_0$,
(3_D) $xyDzw$ *implies* $wzDyx$,

hold for all x, y, z, $w \in A$ and all sequences $x_0, \cdots, x_{n-1}, y_0, \cdots, y_{n-1} \in A$ and all permutations π, σ of $\{0, \cdots, n-1\}$, where $n > 0$.

It should be remarked that (2_D) is an infinite bundle of conditions (for each $n > 0$, each π, σ), and it was shown in Scott and Suppes (1958), that no finite number of them could be sufficient.

IV. SOLUTION TO THE PROBLEM OF SUBJECTIVE PROBABILITIES

Let B be a finite Boolean algebra that we can call the algebra of *events*. A binary relation $\succcurlyeq$ is given on B, where $x \succcurlyeq y$ means that the event x is *more probable than* the event y in some subjective sense. We wonder if these subjective decisions about probabilities are *rational* in the sense that $\succcurlyeq$ is *realizable* by some real-valued probability measure μ on B where

$$x \succcurlyeq y \quad \text{if and only if} \quad \mu(x) \geqslant \mu(y)$$

for all $x, y \in B$.

It seems that de Finetti first raised this question and proposed these obviously necessary axioms for $\succcurlyeq$ for all $x, y, z, w \in B$:

(i) not $0 \succcurlyeq 1$,
(ii) $x \succcurlyeq 0$,
(iii) $x \succcurlyeq y$ or $y \succcurlyeq x$,
(iv) $x \succcurlyeq y \succcurlyeq z$ implies $x \succcurlyeq z$,
(v) $x \succcurlyeq y$ if and only if $x \cup z \succcurlyeq y \cup z$,

in case z is disjoint from both x and y.

It was shown in Kraft, Pratt, and Seidenberg (1959) by an ingenious counter-example in a 32-element Boolean algebra that the axioms (i)-(v) are *not* sufficient. The proper strengthening of the axioms was also given in Theorem 2 of that paper and we shall derive what amounts to their result as a consequence of Theorem 1.3.

Let S be the (finite) set of atoms of B. In the well-known way, we can identify B with the set of $\{0, 1\}$—valued functions on S (that is, the set of characteristic functions of subsets of S.) Thus we imagine that B is a subset of $L = L(S)$, and so the 0 element of B is the 0 vector of L, the 1 element of B is the vector with coordinates all equal to 1. Further if $x, y \in B$ are disjoint, then the vector sum $x + y = x \cup y$. Of course, if $x, y \in B$ overlap, then $x + y \notin B$. We shall write $x \succ y$ for *not* $y \succcurlyeq x$ and $x \preccurlyeq y$ for $y \succcurlyeq x$. With these conventions the desired result can be formulated as follows:

THEOREM 4.1. *Let B be a finite Boolean algebra and let $\succcurlyeq$ be a binary relation on B. For $\succcurlyeq$ to be realizable by a probability measure on B it is necessary and sufficient that the conditions*

(1_B) $1 \succ 0$,
(2_B) $x \succcurlyeq 0$,
(3_B) $x \succcurlyeq y$ *or* $x \preccurlyeq y$,
(4_B) $x_0 + x_1 + \cdots + x_{n-1} = y_0 + y_1 + \cdots + y_{n-1}$ *implies* $x_0 \preccurlyeq y_0$,

hold for all $x, y \in B$ and all sequences $x_0, \cdots, x_{n-1}, y_0, \cdots, y_{n-1} \in B$, where $x_i \succcurlyeq y_i$ for $i < n$, $i > 0$, and $n > 0$.

The unpleasant feature of (4_B) is that it is not a strictly Boolean condition: $x_0 + x_1 + \cdots + x_{n-1}$ means the *algebraic sum* of characteristic functions and *does not* stand for the *union* of the x_i. However, the equation in the hypothesis of (4_B) does have a reasonably simple interpretation in words: every point (atom) belongs to exactly the same number of the x_i as the y_i.

PROOF OF THEOREM 4.1. There is no question that the conditions are necessary. To apply Theorem 1.3, we simply take $Y = B$. Thus there will be a linear functional φ on L that realizes $\succcurlyeq$ on B. In view of (1_B) we have $\varphi(1) > 0$; while $\varphi(x) \geqslant 0$ for

$x \in B$ follows from (2_B). Then the measure μ we are looking for is defined by the equation:

$$\mu(x) = \varphi(x)/\varphi(1)$$

for all $x \in B$.

This argument shows that Theorem 4.1 is almost immediately a special case of Theorem 1.3. If we had left out (1_B) and (2_B), the best we could have said is that $\succcurlyeq$ is realized by a *signed* measure on B.

The derivation of (v) from (3_B) and (4_B) is quite easy. Suppose $x \succcurlyeq y$ but not $x + z \succcurlyeq y + z$. Then $y + z \succcurlyeq x + z$ and $(y + z) + x = (x + z) + y$. So by (4_B) $y + z \preccurlyeq x + z$ must indeed hold. The converse is analogous.

The author has obtained an extension of Theorem 4.1 to infinite Boolean algebras by using the Hahn-Banach Theorem in the same way that it was applied in Kelley (1959), where the problem of the existence of strictly positive measures was solved. If the result proves to be of interest, it will be published in a future paper.

REFERENCES

ADAMS, E. W., AND FAGOT, R. F. A model of riskless choice. *Rep. No. 4*, Applied Mathematics and Statistics Laboratory, Stanford University, 1956.

LUCE, R. D. Semi-orders and a theory of utility discrimination. *Econometrica*, 1956, **24**, 178-191.

LUCE, R. D., AND TUKEY, J. W. Simultaneous conjoint measurement: a new type of fundamental measurement. *J. math. Psychol.*, 1964, **1**, 1-27.

KELLEY, J. L. Measures on Boolean algebras. *Pacific J. Math.*, 1959, **18**, 1165-1172.

KRAFT, C. H., PRATT, J. W., AND SEIDENBERG, A. Intuitive probability on finite sets. *Ann. Math. Statist.*, 1959, **30**, 408-419.

KUHN, H. W., AND TUCKER, A. W. (Eds.) Linear inequalities and related systems. *Annals of mathematics studies*, No. 38, 1956.

SCOTT, D., AND SUPPES, P. Foundational aspects of theories of measurement. *J. Symbolic Logic*, 1958, **23**, 113-128.

SUPPES, P., AND ZINNES, J. L. Basic measurement theory. *Handbook of mathematical psychology*, Vol. 1. New York: Wiley, 1963. Pp. 1-76.

RECEIVED: September 20, 1963